BASES DE DATOS · 0484 · CFGS DAM / DAW

RA 2 · Creación de bases de datos

Crea bases de datos definiendo su estructura y las características de sus elementos según el modelo relacional. Implementación en Oracle, con SQL Server y MariaDB.

AUTOR

Alejandro Martínez Cuesta

IES Comercio · Logroño (La Rioja) · Curso 2026/2027

Índice

Inicio 3

De PixelVault al script 6

APARTADOS

1. Cómo guarda Oracle la información 10

2. Herramientas 16

3. Tipos de datos 20

4. CREATE TABLE y claves 25

5. Restricciones 31

6. Modificar y borrar tablas 37

7. Índices 41

8. Vistas 45

9. Usuarios, roles y privilegios 50

10. El diccionario de datos 58

11. El script completo de PixelVault 62

12. SQL Server y MariaDB 79

REPASO

Errores ORA- frecuentes 103

Ejercicios resueltos 107

Práctica evaluable 119

Autoevaluación 123

Glosario 130

APUNTES · CURSO 2026/2027 · RA 2

Creación de bases de datos

Del esquema relacional a una base de datos que funciona en Oracle: tipos de datos, tablas, claves, restricciones, índices, vistas, usuarios y privilegios. Todo ello creando, sentencia a sentencia, la base de datos de la tienda de videojuegos PixelVault.

Resultado de aprendizaje 2 · Bases de Datos (0484) · CFGS DAM / DAW · IES Comercio · Logroño (La Rioja)

Resultado de aprendizaje

RA 2

Crea bases de datos definiendo su estructura y las características de sus elementos según el modelo relacional.

Estos son los criterios de evaluación del RA 2 y dónde se trabaja cada uno. Verás las etiquetas **CE 2x** junto a los títulos de las secciones.

Criterio	Qué se evalúa	Dónde
CE 2a	Se ha analizado el formato de almacenamiento de la información.	Apartados 1 y 3
CE 2b	Se han creado las tablas y las relaciones entre ellas.	Apartados 4 y 11
CE 2c	Se han seleccionado los tipos de datos adecuados.	Apartado 3
CE 2d	Se han definido los campos clave en las tablas.	Apartado 4
CE 2e	Se han implantado las restricciones reflejadas en el diseño lógico.	Apartados 5 y 7
CE 2f	Se han creado vistas.	Apartado 8
CE 2g	Se han creado los usuarios y se les han asignado privilegios.	Apartado 9

Criterio	Qué se evalúa	Dónde
CE 2h	Se han utilizado asistentes, herramientas gráficas y los lenguajes de definición y control de datos.	Apartados 2, 6, 9 y 10

Qué vamos a construir

En el RA 6 (diseño) obtuvimos el esquema relacional de **PixelVault**, una tienda digital de videojuegos: 20 tablas, sus claves y un catálogo de restricciones. Aquí lo convertimos en una base de datos real en **Oracle**, con usuarios que acceden a ella con distintos permisos. Si no has trabajado ese diseño, no pasa nada: en De PixelVault al script tienes todo lo que necesitas.

→ Empieza por aquí

Contenidos

1. Cómo guarda Oracle la información

Instancia, CDB y PDB, *tablespaces*, bloques y esquemas.

2. Herramientas

SQL Developer, SQL*Plus y SQLcl; conexiones y scripts.

3. Tipos de datos

Texto, números, fechas, objetos grandes y cómo elegirlos.

4. CREATE TABLE y claves

Claves primarias y ajenas, `ON DELETE`, `IDENTITY` y secuencias.

5. Restricciones

`NOT NULL`, `UNIQUE`, `CHECK`, `DEFAULT` y restricciones diferibles.

6. Modificar y borrar tablas

`ALTER`, `DROP`, `TRUNCATE`, `RENAME`, comentarios y papelera.

7. Índices

Qué son, cuáles crea Oracle solo y cuáles hay que crear.

8. Vistas

Atributos derivados, vistas de solo lectura y `WITH CHECK OPTION`.

9. Usuarios, roles y privilegios

10. El diccionario de datos

`CREATE USER`, `GRANT`, `REVOKE` y el lenguaje de control de datos.

Las vistas `USER_`, `ALL_` y `DBA_` para saber qué hay en la BD.

11. El script completo de PixelVault

Los scripts ordenados, listos para descargar y ejecutar.

12. SQL Server y MariaDB

Comparativa y el mismo esquema en otros dos gestores.

Repaso

Errores ORA-, ejercicios, práctica evaluable, autoevaluación y glosario.

Entorno de trabajo

Herramienta	Para qué la usamos
Oracle Database XE 21c u Oracle AI Database 26ai Free	El gestor. El SQL de estos apuntes funciona en los dos; las novedades de la versión 23ai/26ai van en cajas aparte.
Oracle SQL Developer	Escribir y ejecutar scripts, y explorar el esquema con asistentes gráficos.
SQL*Plus / SQLcl	Ejecutar scripts desde la línea de comandos.
Oracle SQL Data Modeler	Generar el DDL a partir del modelo (lo viste en el RA 6).

Cómo usar estos apuntes

- Todos los bloques de código tienen un botón para **copiarlos**, y los scripts completos se pueden **descargar**.
- Las cajas marcadas como **Oracle 23ai / 26ai** explican mejoras que no existen en XE 21c.
- Al final tienes los **errores ORA-** más frecuentes, **ejercicios resueltos** y una **autoevaluación**.

RA 2 · PUNTO DE PARTIDA

De PixelVault al script

PixelVault es una tienda digital (ficticia) de videojuegos para PC y consolas: vende juegos, guarda la biblioteca de cada jugador, sus horas de juego, logros, reseñas y amigos. En el RA 6 hicimos su diseño completo, del enunciado al esquema relacional normalizado. En este RA lo **implementamos en Oracle**.

Esta página resume lo que necesitas del diseño y lo que vamos a construir.

El esquema relacional

Estas son las 20 tablas. La clave primaria va subrayada y las claves ajenas, en cursiva con una flecha hacia la tabla a la que apuntan.

USUARIO(**id_usuario**, email UNIQUE, nombre, apellidos, fecha_registro)

JUGADOR(***id_usuario → USUARIO***, nick UNIQUE, fecha_nacimiento, calle, numero, cp, ciudad, pais)

DESARROLLADOR(***id_usuario → USUARIO***, cargo, *id_estudio → ESTUDIO* NOT NULL)

ESTUDIO(**id_estudio**, nombre, pais, web)

VIDEOJUEGO(**id_juego**, titulo, fecha_lanzamiento, pegi, tipo, *id_estudio → ESTUDIO* NOT NULL, *id_juego_original → VIDEOJUEGO* UNIQUE NULL)

IDIOMA_JUEGO(***id_juego → VIDEOJUEGO***, **idioma**)

JUEGO_PAGO(***id_juego → VIDEOJUEGO***, precio)

JUEGO_F2P(***id_juego → VIDEOJUEGO***, tiene_micropagos)

GENERO(**id_genero**, nombre UNIQUE)

JUEGO_GENERO(***id_juego → VIDEOJUEGO***, ***id_genero → GENERO***)

PLATAFORMA(**id_plataforma**, nombre, fabricante)

BIBLIOTECA(***id_jugador → JUGADOR***, ***id_juego → VIDEOJUEGO***, fecha_adquisicion)

JUEGA(***id_jugador → JUGADOR***, ***id_juego → VIDEOJUEGO***, ***id_plataforma → PLATAFORMA***, horas_jugadas)

AMISTAD(***id_jugador → JUGADOR***, ***id_amigo → JUGADOR***)

LOGRO(***id_juego → VIDEOJUEGO***, **num_logro**, nombre, descripcion, puntos)

DESBLOQUEA(***id_jugador → JUGADOR***, **id_juego**, **num_logro**) — *(id_juego, num_logro) → LOGRO*

DLC(**id_dlc**, titulo, precio, fecha_lanzamiento, *id_juego → VIDEOJUEGO* NOT NULL)

PEDIDO(**num_pedido**, fecha, *id_jugador → JUGADOR* NOT NULL)

LINEA_PEDIDO(***num_pedido → PEDIDO***, ***id_juego → JUEGO_PAGO***, precio_pagado)

RESENA(**id_resena**, nota, texto, fecha, recomendado, id_jugador, id_juego) — *(id_jugador, id_juego) → BIBLIOTECA, UNIQUE, NOT NULL*

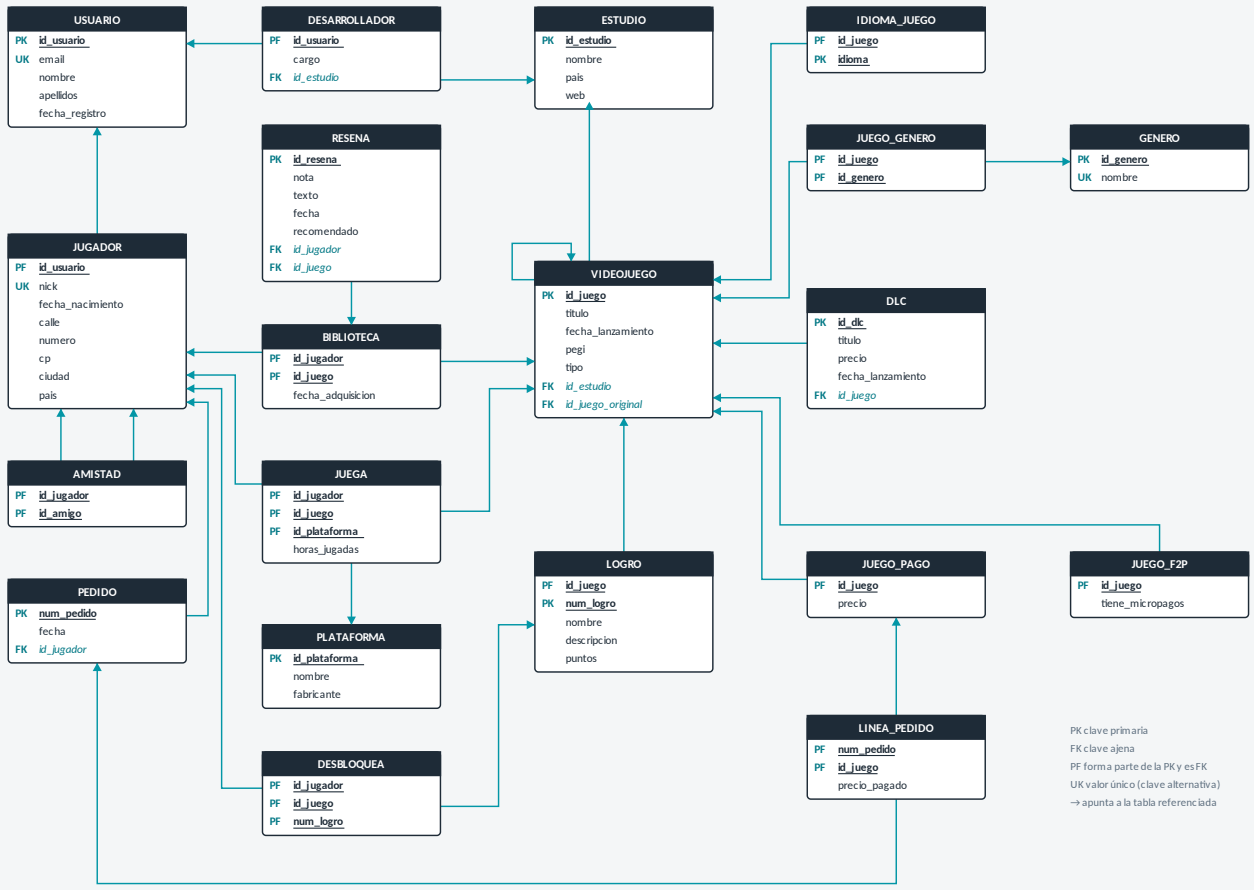


Diagrama relacional de PixelVault. Cada flecha va de la tabla con la clave ajena a la tabla referenciada. Abrir a tamaño completo.

Decisiones del diseño que hay que implantar

El diseño no son solo tablas. Estas decisiones también tienen que aparecer en el script:

Decisión del diseño	Cómo se implanta en Oracle	Apartado
Claves primarias y alternativas	`PRIMARY KEY` y `UNIQUE`	4 y 5
Claves ajenas y acción al borrar	`FOREIGN KEY ... REFERENCES ... [ON DELETE CASCADE / SET NULL]`	4
Identificadores subrogados	Columnas `GENERATED ... AS IDENTITY`	4
Valores obligatorios y por defecto	`NOT NULL` y `DEFAULT`	5
Restricciones de una sola fila (R-01 a R-07)	`CHECK`	5
Atributos derivados (edad, nota media)	Vistas	8

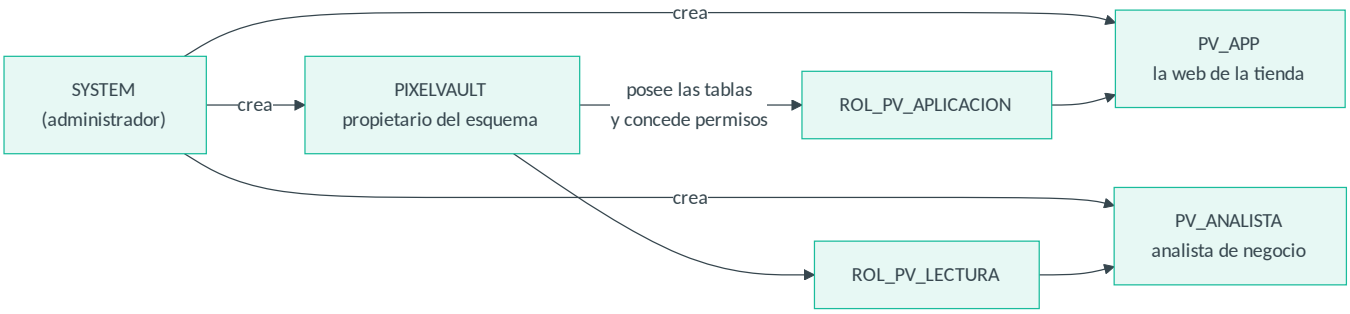
Decisión del diseño	Cómo se implanta en Oracle	Apartado
Quién puede hacer qué	Usuarios, roles y `GRANT`	9

Las restricciones del catálogo que se implantan con `CHECK` son estas; las demás (R-08 en adelante) necesitan disparadores y se harán al estudiar la programación de bases de datos.

Código	Regla	Restricción en Oracle
R-01	`pegi` solo admite 3, 7, 12, 16 o 18	`CHECK (pegi IN (3, 7, 12, 16, 18))`
R-02	`nota` entre 1 y 10	`CHECK (nota BETWEEN 1 AND 10)`
R-03	Precios y horas no negativos; puntos mayores que 0	`CHECK (precio >= 0)` …
R-04	`tipo` es `'PAGO'` o `'F2P'`	`CHECK (tipo IN ('PAGO', 'F2P'))`
R-05 y R-06	Nadie es amigo de sí mismo, y la amistad se guarda una vez	`CHECK (id_jugador < id_amigo)`
R-07	Un juego no es secuela de sí mismo	`CHECK (id_juego_original <> id_juego)`

Los usuarios de la base de datos

La base de datos la usarán tres cuentas distintas, cada una con los permisos justos (**principio de mínimo privilegio**):



Cuenta	Qué hace	Qué puede hacer
PIXELVAULT	Es el **propietario** del esquema: crea tablas, índices y vistas	Crear objetos en su esquema, con una cuota de espacio
PV_APP	La aplicación web de la tienda	Leer el catálogo; leer y escribir datos de jugadores, bibliotecas, pedidos y reseñas
PV_ANALISTA	Un analista que hace estadísticas	Solo consultar dos vistas sin datos personales

Los scripts

Todo el trabajo del RA termina en un conjunto de **scripts** que se ejecutan en orden. Los irás construyendo en los apartados y los tienes completos en el apartado 11.

Script	Quién lo ejecuta	Qué hace
`01_usuarios.sql`	SYSTEM	Crea el propietario, los roles y los usuarios
`02_tablas.sql`	PIXELVAULT	Crea las 20 tablas con sus restricciones
`03_indices.sql`	PIXELVAULT	Crea los índices adicionales
`04_vistas.sql`	PIXELVAULT	Crea las vistas
`05_privilegios.sql`	PIXELVAULT	Concede permisos sobre sus objetos a los roles
`06_datos_prueba.sql`	PIXELVAULT	Carga unas filas para probar
`07_pruebas_restricciones.sql`	PIXELVAULT	Sentencias que deben fallar, para comprobar las restricciones
`99_borrar.sql`	PIXELVAULT y SYSTEM	Lo elimina todo para empezar de cero

RA 2 · APARTADO 1

Cómo guarda Oracle la información

Antes de crear tablas conviene saber **dónde** van a quedar guardadas y **cómo**. En este apartado vemos la arquitectura de Oracle desde el punto de vista de quien crea una base de datos: dónde conectarse, en qué ficheros acaban los datos y qué es exactamente un «esquema».

1.1 Instancia y base de datos CE 2a

En Oracle se distinguen dos cosas que en el lenguaje de la calle se mezclan:

Instancia

> La parte que está **en memoria**: una zona de memoria compartida (la SGA) y un conjunto de procesos que atienden a los usuarios. Existe mientras el servidor está arrancado.

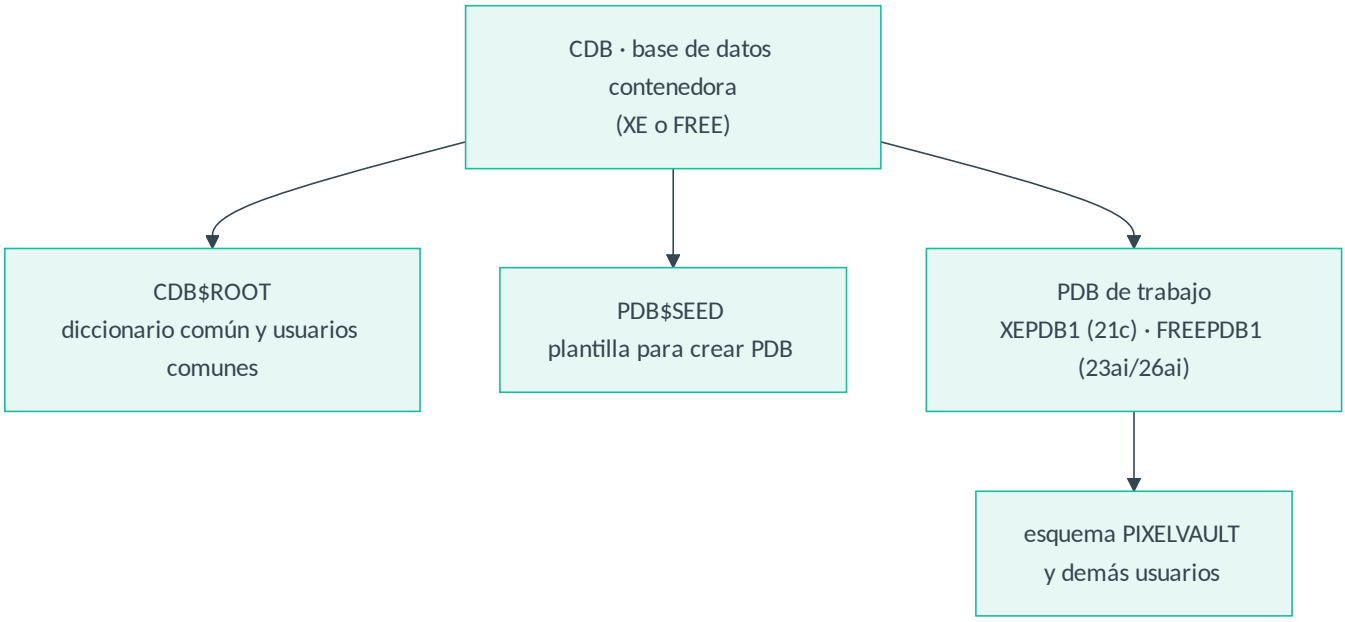
Base de datos

> La parte que está **en disco**: los ficheros de datos, los de control y los de registro (*redo log*). Existe aunque el servidor esté apagado.

Cuando «arrancamos Oracle», la instancia se crea en memoria y **abre** la base de datos, es decir, empieza a trabajar con sus ficheros.

1.2 Contenedores: CDB y PDB

Desde la versión 12c, Oracle usa una arquitectura **multiinquilino** (*multitenant*): una base de datos contenedora aloja varias bases de datos «enchufables», cada una independiente de las demás.



Contenedor	Qué es	¿Trabajamos en él?
`CDB$ROOT`	La raíz: metadatos de Oracle y usuarios comunes (`C##...`)	**No**
`PDB$SEED`	Plantilla de solo lectura para crear nuevas PDB	No
`XEPDB1` / `FREEPDB1`	La base de datos enchufable que viene creada	**Sí**: aquí creamos usuarios y tablas

Conéctate siempre a la PDB

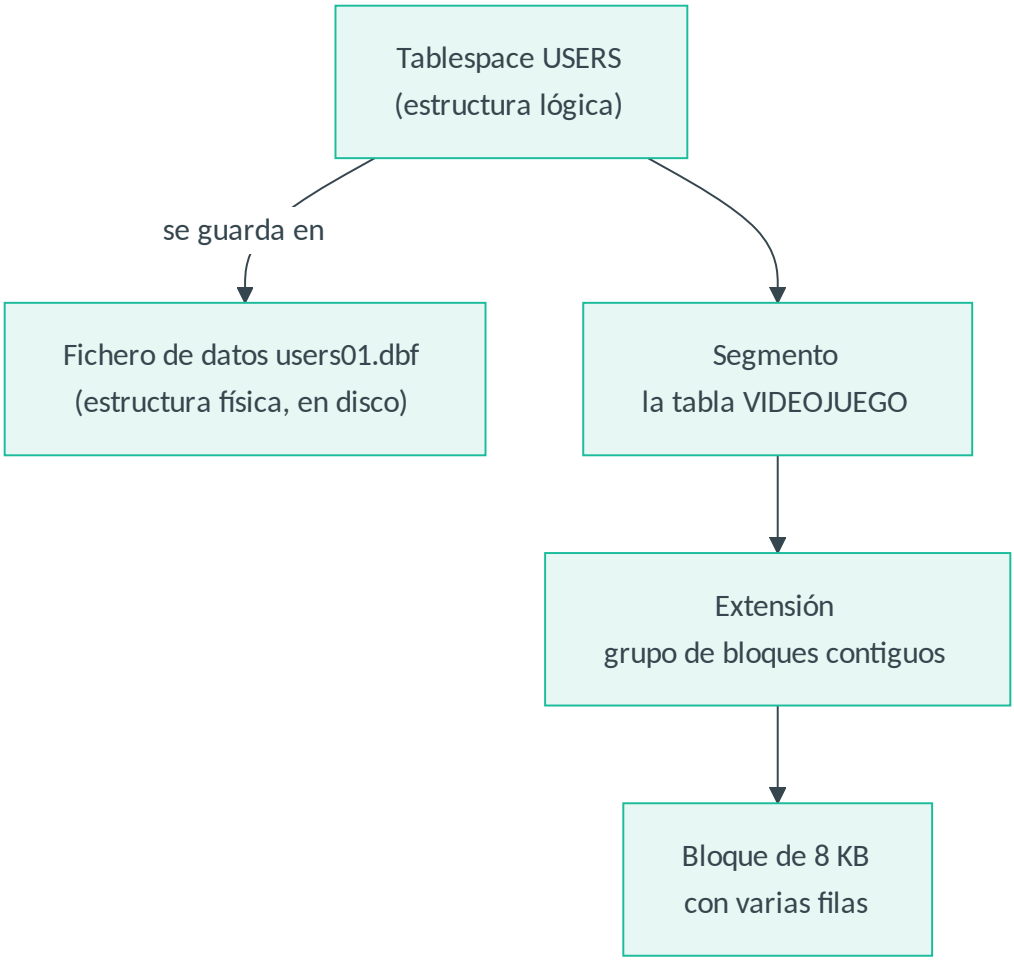
Si te conectas al servicio `XE` o `FREE` estás en la **raíz** (`CDB$ROOT`). Allí, al intentar `CREATE USER pixelvault ...` Oracle responde con **ORA-65096** (nombre de usuario común no válido), porque en la raíz solo se crean usuarios comunes cuyo nombre empieza por `C##` . La solución no es poner `C##` , sino conectarse al servicio de la PDB: `XEPDB1` en XE 21c o `FREEPDB1` en 23ai/26ai. Comprueba dónde estás con `SHOW CON_NAME` .

Ediciones gratuitas

Oracle Database XE 21c y **Oracle AI Database 26ai Free** (la antigua 23ai Free, que cambió de nombre en octubre de 2025) son gratuitas y tienen límites de uso: pocos núcleos de CPU, 2 GB de memoria y 12 GB de datos de usuario. Para aprender y para el proyecto de clase sobran.

1.3 Estructura lógica y física del almacenamiento CE 2a

Oracle organiza el espacio en dos planos: uno **lógico** (cómo lo vemos al crear objetos) y otro **físico** (qué hay en el disco).



Tablespace

: Contenedor lógico de almacenamiento. Cada tabla o índice se guarda en un tablespace, y cada tablespace se apoya en uno o varios **ficheros de datos** del disco.

Segmento

: El espacio que ocupa un objeto concreto: la tabla VIDEOJUEGO es un segmento, su índice de clave primaria es otro.

Extensión

: Un grupo de bloques contiguos que Oracle asigna a un segmento cuando necesita crecer.

Bloque

: La unidad mínima de lectura y escritura. Por defecto mide 8 KB y guarda varias filas.

Estos son los tablespaces que encontrarás en la PDB:

Tablespace	Para qué sirve
`SYSTEM`	El diccionario de datos: la descripción de todo lo que existe en la BD
`SYSAUX`	Datos auxiliares de componentes de Oracle

Tablespace	Para qué sirve
`UNDOTBS1`	Información para deshacer transacciones (`ROLLBACK`)
`TEMP`	Espacio temporal para ordenaciones y operaciones grandes
`USERS`	Los datos de los usuarios: **aquí irán las tablas de PixelVault**

Por eso, al crear el propietario del esquema le indicamos su tablespace y cuánto puede ocupar en él (su **cuota**):

```
CREATE USER pixelvault IDENTIFIED BY "PixelVault_2026"
    DEFAULT TABLESPACE users
    TEMPORARY TABLESPACE temp
    QUOTA 100M ON users;
```

Si un usuario no tiene cuota, puede crear tablas vacías, pero en cuanto intente insertar la primera fila Oracle responderá con **ORA-01950** (no tiene privilegios sobre el tablespace).

1.4 Cómo se guarda una fila CE 2a

Una tabla normal de Oracle es un **montón** (*heap*): las filas se colocan en los bloques donde haya sitio, **sin ningún orden**. Por eso, en el modelo relacional, el orden de las filas no importa, y si necesitas un orden lo pides en la consulta con `ORDER BY`.

Dentro del bloque, cada fila tiene una pequeña cabecera y, a continuación, sus columnas. Algunas consecuencias prácticas:

- Un `VARCHAR2(150)` que guarda `'Hades'` ocupa 5 bytes (más uno de longitud), no 150. En cambio, un `CHAR(150)` ocupa siempre 150, porque se rellena con espacios. Lo verás en el apartado 3.
- Las columnas `NULL` que están al final de la fila no ocupan espacio.
- Si una fila crece y ya no cabe en su bloque, Oracle la mueve a otro (fila **migrada**), lo que hace más lentas las lecturas.

Cada fila tiene una dirección física única, el **ROWID**, que indica en qué fichero, bloque y posición está. Se puede ver con la pseudocolumna del mismo nombre:

```
SELECT ROWID, id_juego, titulo
  FROM videojuego
 WHERE id_juego = 6;
```

```
ROWID              ID_JUEGO TITULO
------------------ ---------- ------
AAAR3sAAMAAAAHFAAF          6 Hades
```

Es la forma más rápida de llegar a una fila, y es lo que guardan internamente los **índices** (apartado 7).

1.5 Esquemas y usuarios

En Oracle, **un usuario y un esquema son lo mismo visto desde dos lados**:

Usuario

: Una cuenta con la que alguien se conecta a la base de datos.

Esquema

: El conjunto de objetos (tablas, vistas, índices, secuencias…) que **pertenecen** a ese usuario. Tiene su mismo nombre.

Cuando `PIXELVAULT` crea la tabla `VIDEOJUEGO`, su nombre completo es `PIXELVAULT.VIDEOJUEGO`. Otro usuario, con permiso, la consulta así:

```
SELECT titulo FROM pixelvault.videojuego;
```

En otros gestores es distinto

En SQL Server y MariaDB la «base de datos» es un objeto que se crea con `CREATE DATABASE` y que es independiente de los usuarios. En Oracle, en cambio, la base de datos ya existe (es la PDB) y lo que se crea son **usuarios**, cada uno con su esquema. Lo comparamos en el apartado 12.

Reglas para los nombres de objetos

Regla	Ejemplo válido	Ejemplo no válido
Empiezan por una letra	`juego_pago`	`2juegos`
Solo letras, dígitos, `_`, `$` y `#`	`linea_pedido`	`linea-pedido`
Hasta 128 caracteres (30 antes de la versión 12.2)	`fk_linea_pedido_juego`	—
No pueden ser palabras reservadas	`fecha`	`date`, `table`, `order`
Sin comillas, Oracle los guarda en **mayúsculas**	`videojuego` → `VIDEOJUEGO`	—

No uses comillas dobles en los nombres

Si creas una tabla como `"Videojuego"` (con comillas), Oracle respeta las mayúsculas y minúsculas y tendrás que escribirla **siempre** entre comillas y exactamente igual. Es una fuente continua de errores **ORA-00942** (la tabla o vista no existe). Escribe los nombres sin comillas, en minúsculas y con guiones bajos, sin tildes ni eñes: por eso usamos `resena` y `genero`.

Resumen

- La **instancia** es la memoria y los procesos; la **base de datos**, los ficheros.
- Oracle aloja **PDB** dentro de una **CDB**. Trabajamos siempre en la PDB (`XEPDB1` o `FREEPDB1`), nunca en `CDB$ROOT`.
- Los objetos se guardan en **tablespaces** (lógico) que se apoyan en **ficheros de datos** (físico). El espacio se organiza en segmentos, extensiones y bloques.

- Las filas no tienen orden; cada una tiene un **ROWID**.
- En Oracle **usuario = esquema**: los objetos pertenecen a un usuario y se nombran `usuario.objeto`.

RA 2 · APARTADO 2

Herramientas y lenguajes

Para crear una base de datos se pueden usar **asistentes gráficos** (que generan el SQL por nosotros) o escribir directamente las sentencias en un **script**. En este apartado conocemos las herramientas de Oracle y los sublenguajes de SQL que vamos a usar.

2.1 Los sublenguajes de SQL CE 2h

SQL agrupa sus sentencias según para qué sirven:

Sublenguaje	Para qué sirve	Sentencias	Dónde se estudia
DDL (definición de datos)	Crear, modificar y borrar la **estructura**	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`, `COMMENT`	**Este RA**
DCL (control de datos)	Gestionar **usuarios y permisos**	`GRANT`, `REVOKE` (y `CREATE USER`, `CREATE ROLE`)	**Este RA**
DML (manipulación de datos)	Consultar y cambiar el **contenido**	`SELECT`, `INSERT`, `UPDATE`, `DELETE`, `MERGE`	RA 3 y RA 4
TCL (control de transacciones)	Confirmar o deshacer cambios	`COMMIT`, `ROLLBACK`, `SAVEPOINT`	RA 4

> ⚠ **Las sentencias DDL confirman solas**
>
> En Oracle, **cada sentencia DDL hace un `COMMIT` implícito**, antes y después de ejecutarse. Si has insertado filas sin confirmar y a continuación ejecutas un `CREATE TABLE`, esas filas quedan confirmadas y ya no podrás deshacerlas con `ROLLBACK`. Por eso no se mezclan cargas de datos y cambios de estructura sin pensarlo.

2.2 Oracle SQL Developer CE 2h

Es la herramienta gráfica gratuita de Oracle y la que usaremos en clase. También existe como **extensión para Visual Studio Code** (*Oracle SQL Developer Extension for VS Code*), con las mismas conexiones y hojas de trabajo.

Crear una conexión

En el panel **Conexiones**, pulsa el botón **+** y rellena:

Campo	Valor para el administrador	Valor para el propietario
Nombre	`XE - SYSTEM`	`XE - PIXELVAULT`

Campo	Valor para el administrador	Valor para el propietario
Usuario	`system`	`pixelvault`
Contraseña	la que pusiste al instalar	`PixelVault_2026`
Nombre del host	`localhost`	`localhost`
Puerto	`1521`	`1521`
Nombre del servicio	`XEPDB1` (21c) o `FREEPDB1` (23ai/26ai)	igual

Pulsa **Probar** y, si el estado es *Correcto*, **Guardar** y **Conectar**.

> **Nombre del servicio, no SID**
>
> Marca la opción **Nombre del servicio** y escribe el de la PDB. Si eliges **SID** y pones `XE` o `FREE`, te estarás conectando a la raíz del contenedor (apartado 1.2).

Hoja de trabajo: sentencia o script

Al conectar se abre una **hoja de trabajo SQL**. Hay dos formas de ejecutar:

Acción	Tecla	Qué hace	Cuándo usarla
Ejecutar sentencia	`^ Ctrl` + `Enter ⏎`	Ejecuta **solo la sentencia** donde está el cursor y muestra el resultado en una rejilla	Probar una consulta o un `CREATE` suelto
Ejecutar script	`F5`	Ejecuta **todo** el contenido, sentencia a sentencia, y muestra la salida como texto	Lanzar un script completo

Asistentes gráficos

Con el botón derecho sobre **Tablas** en el árbol de la conexión, la opción **Nueva tabla…** abre un asistente donde se añaden columnas, tipos, clave primaria y restricciones sin escribir SQL. Tiene una pestaña **DDL** que muestra la sentencia `CREATE TABLE` que va a generar: es una forma estupenda de aprender y de comprobar la sintaxis.

Al abrir una tabla existente verás pestañas como **Columnas**, **Datos**, **Restricciones**, **Índices** y **SQL** (que muestra el DDL completo de la tabla).

> **¿Asistente o script?**
>
> Los asistentes son cómodos para explorar, pero en un proyecto real la estructura se crea siempre con **scripts** guardados en ficheros: se pueden revisar, repetir en otro servidor, guardar en un control de versiones y entregar. Usa el asistente para aprender y el script para trabajar.

2.3 SQL*Plus y SQLcl

klzzw:0026 es la herramienta de línea de comandos clásica de Oracle, disponible en cualquier instalación. **SQLcl** es su versión moderna (con autocompletado e historial) y se usa igual.

Para conectarse se usa la sintaxis `usuario@//equipo:puerto/servicio`:

```
$ sqlplus pixelvault@//localhost:1521/XEPDB1
Enter password:
Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production

SQL> SHOW USER
USER is "PIXELVAULT"
SQL> SHOW CON_NAME
CON_NAME
------------------------------
XEPDB1
```

Órdenes útiles de SQL*Plus y SQLcl (no son SQL, las interpreta la herramienta):

Orden	Qué hace
`@ruta/script.sql`	Ejecuta un script
`DESC tabla`	Describe las columnas de una tabla
`SHOW USER` / `SHOW CON_NAME`	Con qué usuario y en qué contenedor estás
`SPOOL fichero.log` … `SPOOL OFF`	Guarda en un fichero todo lo que sale por pantalla
`SET SERVEROUTPUT ON`	Muestra los mensajes de los programas PL/SQL
`CONNECT usuario@//localhost/XEPDB1`	Cambia de usuario sin salir
`EXIT`	Sale

`DESC`, `SHOW` y `@` también funcionan en la hoja de trabajo de SQL Developer al ejecutar como script (`F5`).

2.4 Cómo se escribe un buen script

Un script de creación es un fichero de texto con extensión `.sql`. Estas son las normas que seguimos en PixelVault:

1. **Cabecera** con qué hace, quién lo ejecuta y en qué orden.
2. Cada sentencia termina en **punto y coma**.
3. **Comentarios** con `--` (una línea) o `/* ... */` (varias líneas).
4. Las tablas se crean en **orden de dependencias**: primero las referenciadas y después las que las referencian. Los borrados, al revés.

5. Todas las restricciones llevan **nombre**, con un convenio fijo (`pk_` , `fk_` , `uk_` , `ck_`).

6. Palabras reservadas en MAYÚSCULAS y nombres de objetos en minúsculas.

Estructura típica de un script

```
-- ======================================================
-- 02_tablas.sql · Crea las tablas de PixelVault
-- Ejecutar como PIXELVAULT, después de 01_usuarios.sql
-- ======================================================

CREATE TABLE estudio (
    id_estudio  NUMBER(10)    GENERATED BY DEFAULT AS IDENTITY,
    nombre      VARCHAR2(100) NOT NULL,
    CONSTRAINT pk_estudio PRIMARY KEY (id_estudio)
);

/* Más tablas, siempre en orden de dependencias */
```

Resumen

- **DDL** define la estructura y **DCL** los permisos; en Oracle cada DDL hace `COMMIT` implícito.
- **SQL Developer**: conexión al **nombre del servicio** de la PDB; `^ Ctrl` + `Enter ⏎` ejecuta una sentencia y `F5` un script.
- Los **asistentes** generan el DDL y sirven para aprender; el trabajo real se hace con **scripts**.
- **SQLPlus / SQLcl*: línea de comandos, con `@script.sql` , `DESC` , `SHOW USER` y `SPOOL` .

RA 2 · APARTADO 3

Tipos de datos de Oracle

Cada columna de una tabla tiene un **tipo de dato**, que decide qué valores admite, cuánto espacio ocupan y qué operaciones se pueden hacer con ellos. Es la implementación del **dominio** del modelo relacional. Elegir mal un tipo se paga caro: fechas guardadas como texto que no se pueden ordenar, precios con errores de redondeo o nombres que no caben.

3.1 Texto CE 2c

Tipo	Qué guarda	Tamaño máximo	Uso en PixelVault
`VARCHAR2(n)`	Texto de **longitud variable**: ocupa lo que se guarda	4000 bytes (32767 si la BD tiene `MAX_STRING_SIZE = EXTENDED`)	`titulo`, `nick`, `email`…
`CHAR(n)`	Texto de **longitud fija**: se rellena con espacios hasta `n`	2000 bytes	`recomendado` (`'S'` / `'N'`)
`NVARCHAR2(n)` / `NCHAR(n)`	Como los anteriores, en el juego de caracteres nacional (Unicode)	4000 / 2000 bytes	No hace falta: la BD ya es Unicode
`CLOB`	Texto muy largo	Terabytes	Textos sin límite práctico (una guía, un manual)

⚠ **`VARCHAR`, no; `VARCHAR2`, sí**

Oracle acepta `VARCHAR`, pero hoy lo trata como sinónimo de `VARCHAR2` y su documentación advierte de que ese comportamiento podría cambiar en el futuro. Usa siempre **`VARCHAR2`**.

Bytes o caracteres

Por defecto, `VARCHAR2(10)` significa **10 bytes**, no 10 caracteres. Las bases de datos modernas usan la codificación AL32UTF8, en la que las letras con tilde o la ñ ocupan 2 bytes: `'Logroño'` tiene 7 caracteres pero ocupa 8 bytes. Si quieres contar en caracteres, indícalo:

```
ciudad VARCHAR2(60 CHAR)   -- 60 caracteres, ocupen lo que ocupen
```

En PixelVault los tamaños tienen margen de sobra, así que usamos la forma por defecto.

La cadena vacía es NULL

⚡ **Una particularidad de Oracle**

En Oracle, la cadena vacía `''` **es lo mismo que** `NULL`. Si insertas `''` en una columna `NOT NULL`, obtendrás **ORA-01400** (no se puede insertar NULL). En SQL Server, MariaDB o PostgreSQL, `''` y `NULL` son cosas distintas.

3.2 Números CE 2c

El tipo numérico de Oracle es `NUMBER(p, s)`:

- **p** (precisión): número total de dígitos significativos, hasta 38.
- **s** (escala): cuántos de ellos van detrás de la coma decimal.

Declaración	Valor que se inserta	Valor guardado
`NUMBER(6,2)`	`24.5`	`24.50`
`NUMBER(6,2)`	`59.999`	`60.00` (redondea a la escala)
`NUMBER(6,2)`	`12345.6`	**ORA-01438**: tiene 5 dígitos enteros y solo caben 4
`NUMBER(2)`	`18`	`18`
`NUMBER(2)`	`100`	**ORA-01438**
`NUMBER(2)`	`7.6`	`8` (escala 0: redondea al entero)
`NUMBER`	cualquier número	tal cual (hasta 38 dígitos)

Otros tipos numéricos:

Tipo	Qué es	Cuándo usarlo
`INTEGER` / `INT`	Sinónimo de `NUMBER(38)`	Existe por compatibilidad con el estándar; en Oracle es más claro `NUMBER(n)`
`BINARY_FLOAT` / `BINARY_DOUBLE`	Coma flotante (IEEE 754), muy rápidos en cálculos	Cálculo científico; **nunca para dinero**

El dinero, siempre con NUMBER(p,s)

Los tipos de coma flotante no pueden representar exactamente valores como 0,1. Para precios e importes usa `NUMBER` con escala 2: `precio NUMBER(6,2)` admite hasta 9999,99 €.

3.3 Fechas y horas CE 2c

Tipo	Qué guarda	Ejemplo
`DATE`	Fecha **y hora** (hasta segundos)	`fecha_lanzamiento`, `fecha_registro`

Tipo	Qué guarda	Ejemplo
`TIMESTAMP(n)`	Fecha y hora con fracciones de segundo (`n` decimales, 6 por defecto)	Registro de conexiones
`TIMESTAMP WITH TIME ZONE`	Además, la zona horaria	Eventos en varios países
`INTERVAL YEAR TO MONTH` / `INTERVAL DAY TO SECOND`	Una duración	Duración de una suscripción

DATE también guarda la hora

Aunque se llame `DATE`, guarda siempre la hora. `SYSDATE` devuelve el momento actual con horas, minutos y segundos. Si guardas `SYSDATE` y luego buscas `WHERE fecha = DATE '2026-09-25'`, no encontrarás nada, porque la hora no es 00:00:00. Para quedarte solo con el día, usa `TRUNC(SYSDATE)`.

Los valores de fecha se escriben con el **literal** `DATE` (formato ISO, independiente de la configuración) o con `TO_DATE`:

```
DATE '2023-08-03'                       -- literal ISO: año-mes-día
TO_DATE('03/08/2023', 'DD/MM/YYYY')     -- con máscara de formato
```

Error frecuente

Guardar fechas en un `VARCHAR2`. Con texto, `'10/02/2024'` es «menor» que `'9/01/2023'` (se compara letra a letra), no se pueden restar dos fechas y nada impide guardar `'31/02/2024'`. Una fecha va **siempre** en `DATE` o `TIMESTAMP`.

3.4 Otros tipos

Tipo	Qué guarda	Disponible en
`BLOB`	Datos binarios grandes (imágenes, PDF…)	Todas las versiones
`RAW(n)`	Datos binarios pequeños (hasta 2000 bytes)	Todas las versiones
`JSON`	Documentos JSON en formato binario optimizado	21c y posteriores
`BOOLEAN`	`TRUE` / `FALSE`	**Solo 23ai / 26ai**
`VECTOR`	Vectores para búsquedas de inteligencia artificial	Solo 23ai / 26ai

`LONG` y `LONG RAW` son tipos antiguos que se mantienen por compatibilidad: **no los uses** en tablas nuevas; usa `CLOB` y `BLOB`.

Valores verdadero/falso

Hasta la versión 21c, Oracle no tenía tipo booleano en las tablas. La solución clásica, que es la que usamos en PixelVault para que el script funcione en XE 21c, es una columna `CHAR(1)` con un `CHECK`:

```
recomendado CHAR(1) NOT NULL,
CONSTRAINT ck_resena_recomendado CHECK (recomendado IN ('S', 'N'))
```

Oracle 23ai / 26ai: tipo BOOLEAN

A partir de la versión 23ai existe el tipo `BOOLEAN`, que admite `TRUE`, `FALSE` y `NULL`:

```
recomendado BOOLEAN NOT NULL
```

Es más claro, pero el script dejaría de funcionar en XE 21c. Si en tu aula todos usáis 23ai/26ai, puedes cambiarlo.

3.5 Los tipos de PixelVault CE 2c

Dato	Tipo elegido	Por qué
Identificadores (`id_juego`, `id_usuario`…)	`NUMBER(10)`	Enteros de hasta 10 dígitos: más de 9 000 millones de filas
`titulo`	`VARCHAR2(150)`	Hay títulos largos, como *The Legend of Zelda: Tears of the Kingdom*
`nick`	`VARCHAR2(30)`	Nombres cortos; el límite forma parte de la regla de negocio
`email`	`VARCHAR2(120)`	Longitud variable
`pegi`	`NUMBER(2)`	Valores de 3 a 18
`precio`, `precio_pagado`	`NUMBER(6,2)`	Importes con dos decimales, hasta 9999,99
`horas_jugadas`	`NUMBER(7,1)`	Hasta 999 999,9 horas, con una décima
Fechas	`DATE`	Fecha (la hora no nos interesa, pero no molesta)
`tipo`	`VARCHAR2(4)` + `CHECK`	Solo `'PAGO'` o `'F2P'`
`recomendado`, `tiene_micropagos`	`CHAR(1)` + `CHECK`	`'S'` o `'N'`, compatible con 21c
`cp`	`VARCHAR2(10)`	**Texto**, aunque tenga cifras: no se opera con él, puede empezar por 0 (`01001`) y en otros países lleva letras

Dato	Tipo elegido	Por qué
`texto` de la reseña	`VARCHAR2(2000)`	Límite razonable para una reseña

¿Número o texto?

Pregúntate: **¿voy a hacer cuentas con este dato?** Un código postal, un teléfono o un DNI tienen cifras, pero no se suman ni se restan, pueden empezar por cero y a veces llevan letras o símbolos. Van en `VARCHAR2`.

Resumen

- Texto: **`VARCHAR2(n)`** (variable) y `CHAR(n)` (fijo). En Oracle, `''` es `NULL`.
- Números: **`NUMBER(p,s)`**; si el valor no cabe, ORA-01438. El dinero nunca en coma flotante.
- **`DATE`** guarda fecha y hora; las fechas se escriben con `DATE 'AAAA-MM-DD'` y nunca se guardan como texto.
- `BOOLEAN` solo existe desde 23ai; en 21c se usa `CHAR(1)` con `CHECK`.
- Los datos con cifras con los que no se opera (código postal, teléfono) van en `VARCHAR2`.

RA 2 · APARTADO 4

CREATE TABLE y claves

Llegamos a la sentencia central del RA: `CREATE TABLE`. En este apartado creamos tablas con sus columnas, sus **claves primarias** y sus **claves ajenas**, que son las que implementan las relaciones del diseño. Las demás restricciones (`NOT NULL`, `UNIQUE`, `CHECK`…) las vemos en detalle en el apartado 5.

4.1 La sintaxis CE 2b

```
CREATE TABLE nombre_tabla (
    columna1  TIPO  [DEFAULT valor]  [restricción de columna],
    columna2  TIPO  ...,
    ...
    [CONSTRAINT nombre] restricción de tabla,
    ...
);
```

Una restricción se puede escribir de dos formas:

Restricción de columna

> Va en la misma línea que la columna y solo afecta a ella: `email VARCHAR2(120) NOT NULL`.

Restricción de tabla

> Va al final, separada por comas, y puede afectar a varias columnas: `CONSTRAINT pk_logro PRIMARY KEY (id_juego, num_logro)`.

Las restricciones de varias columnas (claves compuestas) **solo** se pueden escribir como restricción de tabla. En PixelVault escribimos todas así, salvo `NOT NULL` y `DEFAULT`, porque quedan más ordenadas y todas llevan nombre.

Pon siempre nombre a las restricciones

Si no le das nombre, Oracle inventa uno del tipo `SYS_C008412`. Cuando una inserción falle, el mensaje dirá «restricción SYS_C008412 violada» y no sabrás cuál es. Con nombre, el error se explica solo: `ORA-02290: restricción de control (PIXELVAULT.CK_VIDEOJUEGO_PEGI) violada`. Nuestro convenio: `pk_tabla`, `fk_tabla_referida`, `uk_tabla_columna`, `ck_tabla_regla`.

4.2 La primera tabla CE 2b

Conectados como `PIXELVAULT`, creamos ESTUDIO, que no depende de ninguna otra tabla:

ESTUDIO

```
CREATE TABLE estudio (
    id_estudio  NUMBER(10)    GENERATED BY DEFAULT AS IDENTITY,
    nombre      VARCHAR2(100) NOT NULL,
    pais        VARCHAR2(60),
```

```
    web         VARCHAR2(200),
    CONSTRAINT pk_estudio PRIMARY KEY (id_estudio)
);
```

```
Table ESTUDIO creado.

SQL> DESC estudio
Nombre     ¿Nulo?   Tipo
---------- -------- -------------
ID_ESTUDIO NOT NULL NUMBER(10)
NOMBRE     NOT NULL VARCHAR2(100)
PAIS                VARCHAR2(60)
WEB                 VARCHAR2(200)
```

Fíjate en que `ID_ESTUDIO` aparece como `NOT NULL` aunque no lo hayamos escrito: la clave primaria implica que no puede ser nula.

4.3 La clave primaria CE 2d

La restricción `PRIMARY KEY` implanta la regla de integridad de entidad: la clave no se repite y no es nula. Al declararla, Oracle hace tres cosas:

1. Añade `NOT NULL` a sus columnas.
2. Comprueba que la combinación de valores no se repite.
3. Crea automáticamente un **índice único** con el mismo nombre que la restricción.

Cada tabla tiene **una sola** clave primaria, que puede ser **simple** o **compuesta**:

Simple

```
CONSTRAINT pk_videojuego PRIMARY KEY (id_juego)
```

Compuesta (entidad débil)

```
-- LOGRO es débil por identificación: su clave es la del juego + el discriminador
CONSTRAINT pk_logro PRIMARY KEY (id_juego, num_logro)
```

Compuesta (relación N:M)

```
-- BIBLIOTECA viene de la N:M «posee»
CONSTRAINT pk_biblioteca PRIMARY KEY (id_jugador, id_juego)
```

4.4 Valores autonuméricos CE 2d

Las claves subrogadas (`id_juego` , `id_usuario` …) no las inventa el usuario: las genera la base de datos. Oracle ofrece dos mecanismos.

Columnas IDENTITY (desde 12c)

La columna genera su propio valor, 1, 2, 3...:

Forma	Qué hace
`GENERATED ALWAYS AS IDENTITY`	Oracle genera **siempre** el valor; si intentas darlo tú, error **ORA-32795**
`GENERATED BY DEFAULT AS IDENTITY`	Lo genera si no lo das; si lo das, usa el tuyo
`GENERATED BY DEFAULT ON NULL AS IDENTITY`	Como el anterior, y además lo genera si le pasas `NULL`

En PixelVault usamos `BY DEFAULT`, porque en los datos de prueba queremos dar identificadores conocidos (Hades = 6) para poder escribir las claves ajenas. Pero hay un detalle: cuando das tú el valor, el generador **no se entera**. Tras cargar datos con identificadores explícitos hay que sincronizarlo con el máximo existente:

```
ALTER TABLE videojuego
  MODIFY id_juego GENERATED BY DEFAULT AS IDENTITY (START WITH LIMIT VALUE);
```

Si no lo haces, el primer juego que se inserte sin identificador recibirá el 1, que ya existe, y fallará con **ORA-00001**.

Secuencias

Una **secuencia** es un objeto independiente que genera números. Es el mecanismo clásico (anterior a 12c) y sigue siendo útil cuando varias tablas comparten numeración:

```
CREATE SEQUENCE seq_pedido
    START WITH 5001
    INCREMENT BY 1
    NOCACHE;

-- Uso directo:
INSERT INTO pedido (num_pedido, id_jugador) VALUES (seq_pedido.NEXTVAL, 101);

-- O como valor por defecto de la columna (desde 12c):
num_pedido NUMBER(10) DEFAULT seq_pedido.NEXTVAL
```

`NEXTVAL` da el siguiente número y `CURRVAL` el último obtenido en tu sesión.

> **Los números pueden tener huecos**
>
> Ni las secuencias ni las columnas IDENTITY garantizan números consecutivos: si una inserción falla o se deshace, ese número se pierde. No uses estos identificadores para cosas que exigen numeración sin huecos (por ejemplo, las facturas en algunos países).

4.5 Claves ajenas CE 2b CE 2d

La restricción `FOREIGN KEY` implanta la integridad referencial y es la que materializa la **propagación de clave** del diseño:

```
CONSTRAINT nombre FOREIGN KEY (columnas)
    REFERENCES tabla_referida (columnas)
    [ON DELETE CASCADE | ON DELETE SET NULL]
```

Reglas:

- La tabla referida **tiene que existir antes**.
- Las columnas referidas deben ser su **clave primaria** o una clave `UNIQUE`.
- Los tipos de las columnas deben coincidir.
- Si la relación era obligatoria en el diseño (mínimo 1 en el lado 1), la columna lleva `NOT NULL`.

Los casos de PixelVault

1:N obligatoria

```
-- Todo juego tiene estudio: NOT NULL. Sin ON DELETE: no se puede borrar
-- un estudio que tenga juegos.
id_estudio NUMBER(10) NOT NULL,
CONSTRAINT fk_videojuego_estudio FOREIGN KEY (id_estudio)
    REFERENCES estudio (id_estudio)
```

Entidad débil

```
-- Si se borra el juego, se borran sus logros
CONSTRAINT fk_logro_juego FOREIGN KEY (id_juego)
    REFERENCES videojuego (id_juego) ON DELETE CASCADE
```

Reflexiva 1:1

```
-- La clave apunta a la propia tabla. Si se borra el original,
-- la secuela se queda sin original (NULL). UNIQUE la hace 1:1.
id_juego_original NUMBER(10),
CONSTRAINT fk_videojuego_original FOREIGN KEY (id_juego_original)
    REFERENCES videojuego (id_juego) ON DELETE SET NULL,
CONSTRAINT uk_videojuego_original UNIQUE (id_juego_original)
```

Compuesta

```
-- La clave de LOGRO tiene dos columnas: la FK también
CONSTRAINT fk_desbloquea_logro FOREIGN KEY (id_juego, num_logro)
    REFERENCES logro (id_juego, num_logro) ON DELETE CASCADE
```

Hacia un subtipo

```
-- Solo los juegos de pago se venden: la FK apunta a JUEGO_PAGO,
-- no a VIDEOJUEGO. Así Fortnite no puede aparecer en un pedido.
CONSTRAINT fk_linea_pedido_juego FOREIGN KEY (id_juego)
    REFERENCES juego_pago (id_juego)
```

Acciones al borrar

Cláusula	Al borrar la fila referenciada...	Ejemplo en PixelVault
(ninguna)	Se **impide** el borrado si hay filas que la referencian (**ORA-02292**)	Estudio con juegos, jugador con pedidos
`ON DELETE CASCADE`	Se borran también las filas que la referencian	Logros y DLC de un juego; bibliotecas de un jugador
`ON DELETE SET NULL`	Se pone a `NULL` la clave ajena (la columna debe admitir nulos)	La secuela cuyo original desaparece

> ⚠ **Oracle no tiene ON UPDATE**
>
> Oracle no admite `ON UPDATE CASCADE`. Si se intenta cambiar el valor de una clave primaria que está referenciada, falla con ORA-02292. Es una razón más para usar claves subrogadas, que nunca cambian.

4.6 El orden de creación CE 2b

Como una clave ajena exige que la tabla referida exista, las tablas se crean **por niveles**: primero las que no dependen de nadie, después las que dependen de ellas, y así sucesivamente. Si te equivocas, Oracle responde con **ORA-00942** (la tabla o vista no existe).

Nivel	Tablas	Referencian a...
0	USUARIO, ESTUDIO, GENERO, PLATAFORMA	— (no tienen claves ajenas)
1	JUGADOR, DESARROLLADOR, VIDEOJUEGO	USUARIO, ESTUDIO (y VIDEOJUEGO a sí misma)
2	IDIOMA_JUEGO, JUEGO_PAGO, JUEGO_F2P, JUEGO_GENERO, BIBLIOTECA, JUEGA, AMISTAD, LOGRO, DLC, PEDIDO	Tablas de los niveles 0 y 1
3	DESBLOQUEA, LINEA_PEDIDO, RESENA	LOGRO, JUEGO_PAGO, PEDIDO, BIBLIOTECA (nivel 2)

Una clave ajena **reflexiva** (VIDEOJUEGO → VIDEOJUEGO) no plantea problema: la tabla ya existe en el momento en que se declara la restricción.

Para **borrar**, el orden es el inverso: primero las del último nivel. Si intentas borrar una tabla referenciada, Oracle responde con **ORA-02449** (apartado 6).

¿Y si dos tablas se referencian mutuamente?

A veces el diseño tiene claves ajenas cruzadas (un servicio tiene un jefe que es un médico, y cada médico pertenece a un servicio). No hay orden posible: se crean las dos tablas **sin** una de las claves ajenas y después se añade con `ALTER TABLE ... ADD CONSTRAINT` (apartado 6).

4.7 Crear una tabla a partir de una consulta

`CREATE TABLE ... AS SELECT` crea una tabla con la estructura y las filas que devuelve una consulta. Es útil para copias de seguridad rápidas o para pruebas:

```
CREATE TABLE copia_catalogo AS
SELECT id_juego, titulo, pegi
  FROM videojuego
 WHERE tipo = 'PAGO';
```

Solo se copian los NOT NULL

La tabla nueva hereda los tipos y las restricciones `NOT NULL`, pero **no** la clave primaria, las claves ajenas, los `UNIQUE`, los `CHECK` ni los índices. No es una forma de «clonar» una tabla del diseño.

Oracle 23ai / 26ai: IF NOT EXISTS

Desde 23ai se puede escribir `CREATE TABLE IF NOT EXISTS estudio (...)`: si la tabla ya existe, no hace nada y no da error. Del mismo modo, `DROP TABLE IF EXISTS estudio`. Es muy cómodo para scripts que se ejecutan varias veces, pero en XE 21c da error de sintaxis, así que no lo usamos en los scripts de PixelVault.

Resumen

- `CREATE TABLE` define columnas, tipos y restricciones; las restricciones de varias columnas van a nivel de tabla. **Todas con nombre.**
- `PRIMARY KEY`: no nula, no repetida, con índice único automático. Puede ser simple o compuesta.
- Autonuméricos con `IDENTITY` (`ALWAYS` o `BY DEFAULT`) o con **secuencias**. Tras cargar identificadores a mano, se sincroniza con `START WITH LIMIT VALUE`.
- `FOREIGN KEY ... REFERENCES` implementa la propagación de clave; con `ON DELETE CASCADE` o `SET NULL` se decide qué pasa al borrar. No hay `ON UPDATE`.
- Las tablas se crean **por niveles de dependencia** y se borran en orden inverso.

RA 2 · APARTADO 5

Restricciones

Las **restricciones** (*constraints*) son reglas que el gestor comprueba **cada vez** que se inserta, modifica o borra una fila. Si una operación las incumple, Oracle la rechaza. Son la forma de implantar en la base de datos las reglas del diseño lógico: no dependen de que el programa que accede a los datos «se porte bien».

5.1 Tipos de restricciones CE 2e

Restricción	Qué garantiza	Tipo en el diccionario	Error si se incumple
`PRIMARY KEY`	Identificador no nulo y no repetido	`P`	ORA-00001 / ORA-01400
`FOREIGN KEY`	La referencia existe	`R`	ORA-02291 / ORA-02292
`UNIQUE`	Valor no repetido	`U`	ORA-00001
`NOT NULL`	Valor obligatorio	`C`	ORA-01400 / ORA-01407
`CHECK`	Una condición sobre la fila	`C`	ORA-02290

`DEFAULT` no es una restricción (no rechaza nada), pero se declara junto a ellas y lo vemos también aquí.

5.2 NOT NULL

Hace obligatoria una columna. Se escribe junto a la columna:

```
nombre VARCHAR2(100) NOT NULL
```

Si se inserta una fila sin valor para esa columna, o con `NULL` (o con `''`, que en Oracle es lo mismo), el error es **ORA-01400**. Si se intenta cambiar a `NULL` con un `UPDATE`, **ORA-01407**.

5.3 UNIQUE CE 2e

Implanta las **claves alternativas**: el valor no se puede repetir. Oracle crea un índice único para comprobarlo.

```
CONSTRAINT uk_jugador_nick  UNIQUE (nick),
CONSTRAINT uk_usuario_email UNIQUE (email)
```

Puede ser **compuesta**. En RESENA garantiza «una reseña como máximo por juego y jugador»:

```
CONSTRAINT uk_resena_posesion UNIQUE (id_jugador, id_juego)
```

UNIQUE y los NULL

Un `UNIQUE` **admite varios** `NULL` en Oracle, porque `NULL` no es igual a nada, ni siquiera a otro `NULL`. Por eso `uk_videojuego_original` permite que haya muchos juegos sin original y, a la vez, impide que dos juegos tengan el **mismo** original. Si además el valor fuera obligatorio, habría que añadir `NOT NULL`.

5.4 CHECK CE 2e

Una restricción `CHECK` contiene una condición que **toda fila** debe cumplir. Es la herramienta para implantar las restricciones de dominio y las que comparan columnas de una misma fila. Así quedan las del catálogo de PixelVault:

```
-- R-01 · PEGI europeo
CONSTRAINT ck_videojuego_pegi CHECK (pegi IN (3, 7, 12, 16, 18))

-- R-02 · Nota de 1 a 10
CONSTRAINT ck_resena_nota CHECK (nota BETWEEN 1 AND 10)

-- R-03 · Importes no negativos
CONSTRAINT ck_juego_pago_precio CHECK (precio >= 0)

-- R-04 · Tipo de juego (y en la práctica, el discriminador de la jerarquía)
CONSTRAINT ck_videojuego_tipo CHECK (tipo IN ('PAGO', 'F2P'))

-- R-05 y R-06 · Nadie es amigo de sí mismo y cada amistad se guarda una vez
CONSTRAINT ck_amistad_orden CHECK (id_jugador < id_amigo)

-- R-07 · Un juego no es secuela de sí mismo
CONSTRAINT ck_videojuego_secuela CHECK (id_juego_original <> id_juego)

-- Un formato mínimo de correo: algo@algo.xx
CONSTRAINT ck_usuario_email CHECK (email LIKE '%_@_%.__%')
```

R-06 incluye a R-05

Si exigimos `id_jugador < id_amigo`, ya es imposible que sean iguales. Además, al obligar a guardar siempre el menor primero, la amistad entre 101 y 102 solo puede existir como `(101, 102)`: nunca aparecerá duplicada como `(102, 101)`.

Qué no puede hacer un CHECK

Un `CHECK` solo ve **la fila que se está comprobando**. No puede:

- Consultar otras filas ni otras tablas (no admite subconsultas). «Para reseñar hay que haber jugado 2 horas» no es un `CHECK`: es un disparador.
- Usar funciones que cambian con el tiempo o con la sesión, como `SYSDATE` o `USER`. `CHECK (fecha_nacimiento < SYSDATE)` da **ORA-02436**. Tampoco puede usar secuencias.

Esas reglas son las **restricciones no representables** del diseño y se implementan con disparadores en la programación de bases de datos.

> ⚠ **Un CHECK con NULL se cumple**
>
> La condición de un `CHECK` se considera cumplida si es **verdadera o desconocida**. Con `NULL`, `precio >= 0` es desconocido, así que la fila se acepta. Por eso las columnas obligatorias llevan **además** `NOT NULL`: `precio NUMBER(6,2) NOT NULL + CHECK (precio >= 0)`.

5.5 DEFAULT

Da un valor a la columna cuando el `INSERT` no la menciona:

```
fecha_registro   DATE        DEFAULT SYSDATE NOT NULL,
tiene_micropagos CHAR(1)     DEFAULT 'N'     NOT NULL,
horas_jugadas    NUMBER(7,1) DEFAULT 0       NOT NULL
```

A diferencia del `CHECK`, en un `DEFAULT` **sí** se puede usar `SYSDATE`, porque se evalúa una sola vez, al insertar.

> 📎 **DEFAULT solo actúa si no se indica la columna**
>
> Si el `INSERT` pone la columna explícitamente con `NULL`, el `DEFAULT` no actúa y se guarda `NULL` (o falla, si la columna es `NOT NULL`). Desde la versión 12c existe `DEFAULT ON NULL`, que se aplica también en ese caso: `fecha DATE DEFAULT ON NULL SYSDATE`.

5.6 Estado de una restricción

Las restricciones pueden desactivarse temporalmente, por ejemplo para una carga masiva de datos:

```
ALTER TABLE videojuego DISABLE CONSTRAINT ck_videojuego_pegi;
-- ... carga de datos ...
ALTER TABLE videojuego ENABLE CONSTRAINT ck_videojuego_pegi;
```

Al volver a activarla, Oracle comprueba **todas** las filas; si alguna la incumple, el `ENABLE` falla (ORA-02293 en un `CHECK`, ORA-02298 en una clave ajena) y la restricción sigue desactivada.

Restricciones diferibles

Normalmente una restricción se comprueba al terminar cada sentencia. Si se declara `DEFERRABLE INITIALLY DEFERRED`, se comprueba al hacer `COMMIT`. Sirve para las claves ajenas cruzadas, donde hay que insertar dos filas que se referencian mutuamente:

```
CONSTRAINT fk_servicio_jefe FOREIGN KEY (cod_jefe)
    REFERENCES medico (cod_empleado)
    DEFERRABLE INITIALLY DEFERRED
```

Dentro de la transacción se inserta el servicio (apuntando a un médico que aún no existe) y el médico; al hacer `COMMIT`, ya existen los dos y la comprobación se supera.

5.7 Del diseño a las restricciones de PixelVault CE 2e

Elemento del diseño	Restricción en Oracle	Ejemplo
Clave primaria	`PRIMARY KEY`	`pk_videojuego`
Clave alternativa	`UNIQUE` (+ `NOT NULL` si es obligatoria)	`uk_jugador_nick`
Propagación de clave (1:N)	`FOREIGN KEY` en el lado N	`fk_videojuego_estudio`
Mínimo 1 en el lado 1	`NOT NULL` en la clave ajena	`id_estudio NOT NULL`
Relación 1:1	`FOREIGN KEY` + `UNIQUE`	`uk_videojuego_original`
Entidad débil	`FOREIGN KEY ... ON DELETE CASCADE`	`fk_logro_juego`
Dominio de un atributo	Tipo de dato + `CHECK`	`ck_videojuego_pegi`
Regla entre columnas de la fila	`CHECK`	`ck_amistad_orden`
Discriminador de la jerarquía	`CHECK` sobre la columna `tipo`	`ck_videojuego_tipo`
Regla entre filas o tablas	**No es una restricción**: disparador	R-08 a R-15

5.8 Probar las restricciones

Una restricción que no se ha probado es una restricción que no sabemos si funciona. El script `07_pruebas_restricciones.sql` contiene sentencias que **deben fallar**, cada una con el error esperado. Estos son algunos ejemplos:

```
-- R-01 · PEGI no válido: ORA-02290 (CK_VIDEOJUEGO_PEGI)
INSERT INTO videojuego (titulo, pegi, tipo, id_estudio)
VALUES ('Juego raro', 15, 'PAGO', 10);

-- FK hacia un subtipo: un free-to-play en un pedido: ORA-02291 (FK_LINEA_PEDIDO_JUEGO)
INSERT INTO linea_pedido (num_pedido, id_juego, precio_pagado)
VALUES (5001, 9, 0);

-- Borrar un estudio con juegos: ORA-02292 (FK_VIDEOJUEGO_ESTUDIO)
DELETE FROM estudio WHERE id_estudio = 13;
```

```
Error que empieza en la línea: 2 del comando -
INSERT INTO videojuego (titulo, pegi, tipo, id_estudio)
VALUES ('Juego raro', 15, 'PAGO', 10)
Error en la línea de comandos : 2 Columna : 1
Informe de error -
Error SQL: ORA-02290: restricción de control (PIXELVAULT.CK_VIDEOJUEGO_PEGI) violada
```

Script completo de pruebas (07_pruebas_restricciones.sql)

```
-- ======================================================================
-- PixelVault · 07_pruebas_restricciones.sql
-- Sentencias que DEBEN FALLAR. Sirven para comprobar que cada
-- restricción está bien implantada. Ejecutar como PIXELVAULT, una a una
-- (en SQL Developer: cursor sobre la sentencia y Ctrl+Intro).
-- Al lado de cada una, el error que debe aparecer.
-- ======================================================================

-- R-01 · PEGI no válido ............................. ORA-02290 (ck_videojuego_pegi)
INSERT INTO videojuego (titulo, pegi, tipo, id_estudio) VALUES ('Juego raro', 15, 'PAGO',
10);

-- R-04 · tipo no válido ............................. ORA-02290 (ck_videojuego_tipo)
INSERT INTO videojuego (titulo, pegi, tipo, id_estudio) VALUES ('Juego raro', 12, 'FREE',
10);

-- Tipo de dato · 'GRATIS' no cabe en VARCHAR2(4) ..... ORA-12899 (antes incluso del CHECK)
INSERT INTO videojuego (titulo, pegi, tipo, id_estudio) VALUES ('Juego raro', 12, 'GRATIS',
10);

-- R-07 · un juego secuela de sí mismo ................ ORA-02290 (ck_videojuego_secuela)
UPDATE videojuego SET id_juego_original = 5 WHERE id_juego = 5;

-- 1:1 · dos secuelas del mismo original .............. ORA-00001 (uk_videojuego_original)
UPDATE videojuego SET id_juego_original = 6 WHERE id_juego = 5;

-- R-02 · nota fuera de rango ......................... ORA-02290 (ck_resena_nota)
INSERT INTO resena (nota, recomendado, id_jugador, id_juego) VALUES (11, 'S', 101, 5);

-- R-06 · amistad guardada al revés ................... ORA-02290 (ck_amistad_orden)
INSERT INTO amistad (id_jugador, id_amigo) VALUES (102, 101);

-- NOT NULL · juego sin estudio ....................... ORA-01400
INSERT INTO videojuego (titulo, pegi, tipo) VALUES ('Sin estudio', 12, 'PAGO');

-- UNIQUE · nick repetido ............................. ORA-00001 (uk_jugador_nick)
INSERT INTO usuario (email, nombre) VALUES ('otro@ejemplo.es', 'Otro');
INSERT INTO jugador (id_usuario, nick, fecha_nacimiento)
SELECT id_usuario, 'NoctisRJ', DATE '2000-01-01' FROM usuario WHERE email =
'otro@ejemplo.es';
ROLLBACK;

-- FK · estudio inexistente ........................... ORA-02291 (fk_videojuego_estudio)
INSERT INTO videojuego (titulo, pegi, tipo, id_estudio) VALUES ('Huérfano', 12, 'PAGO', 99);

-- FK · borrar un estudio con juegos (FromSoftware) .... ORA-02292 (fk_videojuego_estudio)
DELETE FROM estudio WHERE id_estudio = 13;

-- FK · pedir un juego free-to-play ................... ORA-02291 (fk_linea_pedido_juego)
INSERT INTO linea_pedido (num_pedido, id_juego, precio_pagado) VALUES (5001, 9, 0);

-- Agregación · reseñar un juego que no se tiene ...... ORA-02291 (fk_resena_biblioteca)
INSERT INTO resena (nota, recomendado, id_jugador, id_juego) VALUES (7, 'N', 102, 4);

-- Vista con CHECK OPTION · juego no PEGI 18 .......... ORA-01402
INSERT INTO v_juegos_pegi18 (titulo, pegi, tipo, id_estudio) VALUES ('Juego infantil', 3,
'PAGO', 17);
```

```
-- Vista de solo lectura ............................. ORA-42399
UPDATE v_catalogo SET pegi = 16 WHERE id_juego = 1;

-- ------------------------------------------------------------------
-- Y estas DEBEN FUNCIONAR (acciones ON DELETE). Se deshacen al final.
-- ------------------------------------------------------------------
-- CASCADE: al borrar Elden Ring desaparecen sus logros, su DLC...
-- ...pero falla por BIBLIOTECA (restricción sin cascada): ORA-02292.
DELETE FROM videojuego WHERE id_juego = 4;

-- SET NULL: al borrar Hades, Hades II se queda sin original.
-- Falla igualmente porque Hades está en bibliotecas y pedidos: ORA-02292.
-- Probemos con un juego nuevo sin ventas:
INSERT INTO videojuego (id_juego, titulo, pegi, tipo, id_estudio) VALUES (900, 'Juego de
prueba', 7, 'PAGO', 15);
INSERT INTO videojuego (id_juego, titulo, pegi, tipo, id_estudio, id_juego_original)
VALUES (901, 'Juego de prueba 2', 7, 'PAGO', 15, 900);
DELETE FROM videojuego WHERE id_juego = 900;
SELECT id_juego, titulo, id_juego_original FROM videojuego WHERE id_juego = 901;  --
original: NULL
ROLLBACK;
```

Resumen

- Las restricciones implantan las reglas del diseño **en la base de datos**, para cualquier programa que acceda a ella.
- `NOT NULL` hace obligatorio un valor; `UNIQUE` implanta las claves alternativas (y admite varios `NULL`).
- `CHECK` comprueba una condición **sobre la propia fila**: no puede consultar otras filas ni usar `SYSDATE`. Con `NULL` se da por cumplido.
- `DEFAULT` rellena valores que no se indican; sí admite `SYSDATE`.
- Las restricciones se pueden desactivar, y las diferibles se comprueban en el `COMMIT`.
- Cada restricción se **prueba** con sentencias que deben fallar.

RA 2 · APARTADO 6

Modificar y borrar tablas

Una base de datos no se crea una vez y se deja quieta: los requisitos cambian y la estructura tiene que acompañarlos. En este apartado vemos el resto de sentencias DDL: `ALTER TABLE`, `DROP`, `TRUNCATE`, `RENAME` y `COMMENT`, y la papelera de reciclaje de Oracle.

6.1 El cliente pide cambios

Un mes después de poner PixelVault en marcha, llegan estos cambios. Los resolveremos a lo largo del apartado:

1. Los jugadores ya no se borran: se hace una **baja lógica** con una columna `activo`.
2. Los *nicks* pueden tener hasta 40 caracteres.
3. Se quiere guardar el **teléfono** de los usuarios (opcional).
4. La columna `web` de ESTUDIO no se ha usado nunca: hay que quitarla.
5. El precio de los DLC no puede superar los 100 €.

6.2 ALTER TABLE: columnas CE 2h

```
-- Añadir una columna (cambios 1 y 3)
ALTER TABLE usuario ADD (
    activo   CHAR(1) DEFAULT 'S' NOT NULL,
    telefono VARCHAR2(20)
);

-- Cambiar el tipo o el tamaño (cambio 2)
ALTER TABLE jugador MODIFY (nick VARCHAR2(40));

-- Renombrar una columna
ALTER TABLE usuario RENAME COLUMN telefono TO movil;

-- Eliminar una columna (cambio 4)
ALTER TABLE estudio DROP COLUMN web;
```

> ⚠ **Añadir una columna NOT NULL a una tabla con filas**
>
> Si la tabla ya tiene filas, una columna nueva `NOT NULL` **necesita un** `DEFAULT`: si no, las filas existentes quedarían con `NULL` y Oracle lo rechaza (**ORA-01758**). Por eso `activo` se añade con `DEFAULT 'S'`: todos los usuarios existentes quedan activos.

Al **modificar** una columna:

- Aumentar el tamaño siempre se puede.
- Reducirlo solo si los datos caben; si no, **ORA-01441**.
- Cambiar a un tipo incompatible (por ejemplo, de `VARCHAR2` a `NUMBER`) solo se puede si la columna está vacía, es decir, si todos sus valores son `NULL`.

Borrar columnas en tablas enormes

`DROP COLUMN` reescribe todas las filas de la tabla, lo que en tablas muy grandes puede tardar. Oracle permite marcarla primero como no usada (`ALTER TABLE t SET UNUSED (columna)`), que es instantáneo, y borrarla físicamente más tarde (`ALTER TABLE t DROP UNUSED COLUMNS`).

6.3 ALTER TABLE: restricciones CE 2h

```sql
-- Añadir una restricción (cambio 5)
ALTER TABLE dlc ADD CONSTRAINT ck_dlc_precio_max CHECK (precio <= 100);

-- Añadir o quitar NOT NULL (se hace con MODIFY)
ALTER TABLE estudio MODIFY (pais NOT NULL);
ALTER TABLE estudio MODIFY (pais NULL);

-- Añadir una clave ajena después de crear la tabla
ALTER TABLE desarrollador ADD CONSTRAINT fk_desarrollador_estudio
    FOREIGN KEY (id_estudio) REFERENCES estudio (id_estudio);

-- Eliminar una restricción
ALTER TABLE dlc DROP CONSTRAINT ck_dlc_precio_max;

-- Desactivarla y activarla
ALTER TABLE videojuego DISABLE CONSTRAINT ck_videojuego_pegi;
ALTER TABLE videojuego ENABLE  CONSTRAINT ck_videojuego_pegi;

-- Renombrar una restricción
ALTER TABLE dlc RENAME CONSTRAINT ck_dlc_precio TO ck_dlc_precio_positivo;
```

Al añadir una restricción a una tabla con datos, Oracle comprueba las filas existentes. Si alguna la incumple, la restricción no se crea.

Añadir claves ajenas al final

Algunos equipos crean primero **todas** las tablas sin claves ajenas y después añaden todas las claves ajenas con `ALTER TABLE ... ADD CONSTRAINT`. Así el orden de creación de las tablas da igual y se resuelven las referencias cruzadas. Es lo que hace, por ejemplo, el DDL que genera Oracle SQL Data Modeler.

6.4 Renombrar y comentar

```sql
-- Renombrar una tabla (dos formas equivalentes)
RENAME copia_catalogo TO catalogo_2026;
ALTER TABLE catalogo_2026 RENAME TO catalogo_backup;

-- Comentarios: documentan el esquema dentro de la propia BD
COMMENT ON TABLE biblioteca IS 'Juegos que posee cada jugador (relación POSEE)';
COMMENT ON COLUMN videojuego.tipo IS 'PAGO o F2P. Indica en qué tabla de subtipo tiene fila';
```

Los comentarios quedan guardados en el diccionario de datos (`USER_TAB_COMMENTS`, `USER_COL_COMMENTS`) y los muestran herramientas como SQL Developer. Son la mejor forma de que la documentación del diseño viaje con la base de datos.

6.5 DROP TABLE y la papelera CE 2h

`DROP TABLE` elimina la tabla: su estructura, sus datos, sus índices y sus restricciones.

```
DROP TABLE copia_catalogo;
```

Si otra tabla tiene una clave ajena que apunta a ella, Oracle no la deja borrar (**ORA-02449**). Hay dos soluciones:

```
-- 1. Borrar antes las tablas que la referencian (orden inverso al de creación)

-- 2. Borrar también las claves ajenas que apuntan a ella (¡no las tablas!)
DROP TABLE estudio CASCADE CONSTRAINTS;
```

La papelera de reciclaje

Por defecto, una tabla borrada no desaparece del todo: va a la **papelera** (*recycle bin*) con un nombre del tipo `BIN$...`, y se puede recuperar:

```
DROP TABLE copia_catalogo;
SHOW RECYCLEBIN;                                -- lista lo que hay en la papelera
FLASHBACK TABLE copia_catalogo TO BEFORE DROP;  -- ¡recuperada!

DROP TABLE copia_catalogo PURGE;                -- borrado definitivo, sin papelera
PURGE RECYCLEBIN;                               -- vacía tu papelera
```

> ⚠ **La papelera ocupa cuota**
>
> Las tablas en la papelera siguen ocupando espacio en el tablespace. Por eso el script `99_borrar.sql` de PixelVault usa `PURGE`: cuando borramos para volver a empezar, no queremos llenar la cuota del usuario de copias viejas.

6.6 DROP, TRUNCATE y DELETE

Las tres «borran», pero hacen cosas muy distintas:

	`DELETE FROM t`	`TRUNCATE TABLE t`	`DROP TABLE t`
Lenguaje	DML	**DDL**	**DDL**
Qué elimina	Las filas (todas o las del `WHERE`)	**Todas** las filas	La tabla entera: filas y estructura
¿Admite `WHERE`?	Sí	No	No
¿Se puede deshacer con `ROLLBACK`?	Sí	**No** (hace `COMMIT` implícito)	No (pero va a la papelera)

	`DELETE FROM t`	`TRUNCATE TABLE t`	`DROP TABLE t`
Velocidad en tablas grandes	Lenta (fila a fila)	Muy rápida	Muy rápida
¿Respeta claves ajenas?	Sí, fila a fila (y ejecuta los `ON DELETE`)	Falla si otra tabla la referencia con una FK activa (ORA-02266)	Falla si otra tabla la referencia (ORA-02449)

6.7 Otros objetos: DROP y ALTER

Lo visto para las tablas se aplica, con pequeños cambios, al resto de objetos:

```
DROP VIEW v_juegos_pegi18;
DROP INDEX ix_videojuego_pegi;
DROP SEQUENCE seq_pedido;
ALTER SEQUENCE seq_pedido INCREMENT BY 10;
```

Oracle 23ai / 26ai: IF EXISTS

En 23ai/26ai se puede escribir `DROP TABLE IF EXISTS copia_catalogo PURGE;` para que no dé error si la tabla no existe. En XE 21c hay que ejecutarlo sin `IF EXISTS` (y aceptar el error ORA-00942 si no existe).

Resumen

- `ALTER TABLE` añade (`ADD`), modifica (`MODIFY`), renombra (`RENAME COLUMN`) y elimina (`DROP COLUMN`) columnas, y añade, elimina, activa o desactiva restricciones.
- Una columna nueva `NOT NULL` en una tabla con filas necesita `DEFAULT`.
- `DROP TABLE` falla si la tabla está referenciada: se borra en orden inverso o con `CASCADE CONSTRAINTS`. Sin `PURGE`, la tabla va a la **papelera** y se recupera con `FLASHBACK TABLE`.
- `TRUNCATE` vacía la tabla muy rápido pero **no se puede deshacer**; `DELETE` sí.
- `COMMENT ON` guarda la documentación en el diccionario de datos.

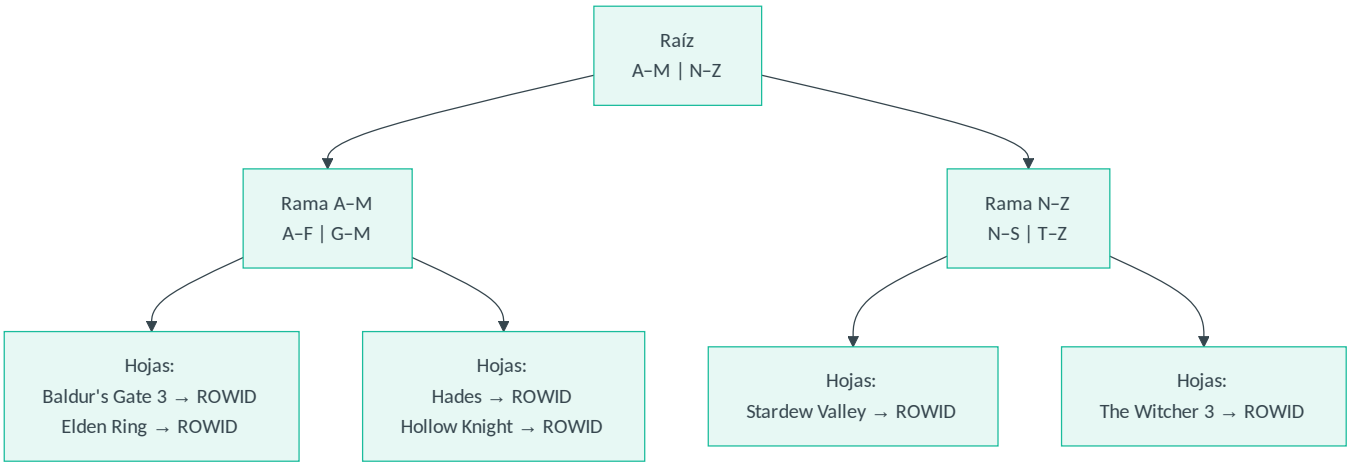
RA 2 · APARTADO 7

Índices

Un **índice** es una estructura adicional que permite encontrar filas sin recorrer toda la tabla, igual que el índice alfabético de un libro permite ir a la página de un tema sin leerlo entero. Los índices no cambian el resultado de ninguna consulta: solo la hacen más rápida (o no). Pero también tienen un coste, y saber cuándo crearlos forma parte del diseño físico.

7.1 Cómo funciona un índice CE 2e

El índice normal de Oracle es un **árbol B** (*B-tree*): guarda los valores de la columna **ordenados**, y junto a cada valor el **ROWID** de la fila donde está (apartado 1.4).



Para buscar `'Hades'`, Oracle baja por el árbol en unos pocos pasos, obtiene el ROWID y va directamente al bloque de la fila. Sin índice, tendría que leer **todos** los bloques de la tabla (un **recorrido completo**, *full table scan*).

Ventajas	Inconvenientes
Consultas mucho más rápidas al buscar, filtrar, ordenar o combinar por esa columna	**Ocupan espacio** en el tablespace
Permiten comprobar rápido `PRIMARY KEY` y `UNIQUE`	**Ralentizan** `INSERT`, `UPDATE` y `DELETE`, porque hay que mantener el índice

7.2 Los índices que Oracle crea solo

Al declarar una restricción `PRIMARY KEY` o `UNIQUE`, Oracle crea automáticamente un **índice único** con el nombre de la restricción. En PixelVault ya existen, sin hacer nada, `PK_VIDEOJUEGO`, `UK_JUGADOR_NICK`, `PK_BIBLIOTECA`…

> ⚠ **Las claves ajenas no tienen índice automático**
>
> Oracle **no** crea índices para las claves ajenas. Y conviene tenerlos, por dos motivos:
>
> 1. Las consultas que combinan tablas (`JOIN`) suelen hacerse por la clave ajena.
> 2. Al borrar o modificar una fila de la tabla referenciada (un estudio), Oracle tiene que buscar las filas hijas (sus juegos). Sin índice, recorre toda la tabla hija y la **bloquea** mientras tanto.
>
> Regla práctica: **indexa cada clave ajena que no sea ya el principio de otro índice**.

7.3 CREATE INDEX

```
CREATE [UNIQUE] INDEX nombre ON tabla (columna1 [, columna2 ...]);
```

Estos son los índices de PixelVault (script `03_indices.sql`):

```
-- ========================================================================
-- PixelVault · 03_indices.sql
-- Índices adicionales. Ejecutar conectado como PIXELVAULT.
--
-- Oracle ya crea automáticamente un índice único para cada PRIMARY KEY
-- y cada UNIQUE. Lo que NO crea es un índice para las claves ajenas:
-- conviene crearlo en las columnas FK que no sean ya el inicio de
-- otro índice, porque se usan en las combinaciones (JOIN) y porque
-- evitan bloqueos al borrar filas de la tabla referenciada.
-- ========================================================================

-- Claves ajenas simples
CREATE INDEX ix_desarrollador_estudio ON desarrollador (id_estudio);
CREATE INDEX ix_videojuego_estudio    ON videojuego (id_estudio);
CREATE INDEX ix_juego_genero_genero   ON juego_genero (id_genero);
CREATE INDEX ix_biblioteca_juego      ON biblioteca (id_juego);
CREATE INDEX ix_juega_juego           ON juega (id_juego);
CREATE INDEX ix_juega_plataforma      ON juega (id_plataforma);
CREATE INDEX ix_amistad_amigo         ON amistad (id_amigo);
CREATE INDEX ix_dlc_juego             ON dlc (id_juego);
CREATE INDEX ix_pedido_jugador        ON pedido (id_jugador);
CREATE INDEX ix_linea_pedido_juego    ON linea_pedido (id_juego);

-- Clave ajena compuesta (hacia LOGRO)
CREATE INDEX ix_desbloquea_logro      ON desbloquea (id_juego, num_logro);

-- Índice basado en función: búsquedas de títulos sin distinguir mayúsculas
-- (sirve para consultas con WHERE UPPER(titulo) = 'HADES')
CREATE INDEX ix_videojuego_titulo_up  ON videojuego (UPPER(titulo));

-- Índice para filtrar el catálogo por clasificación de edad
CREATE INDEX ix_videojuego_pegi       ON videojuego (pegi);
```

¿Por qué no hay índice para `biblioteca.id_jugador` o para `logro.id_juego` ? Porque ya son la **primera columna** de la clave primaria de su tabla, y el índice de la clave primaria sirve también para buscar por ella.

Índices compuestos: el orden importa

Un índice sobre `(id_jugador, id_juego)` está ordenado primero por jugador y, dentro de cada jugador, por juego. Como una guía telefónica ordenada por apellido y después por nombre:

- Sirve para buscar **por jugador** o **por jugador y juego**.
- **No** sirve (o sirve mal) para buscar **solo por juego**. Por eso `ix_biblioteca_juego` es necesario.

Índices basados en función

Un índice normal sobre `titulo` no ayuda a la consulta `WHERE UPPER(titulo) = 'HADES'`, porque lo que se compara es `UPPER(titulo)`, no `titulo`. Para eso se indexa la expresión:

```
CREATE INDEX ix_videojuego_titulo_up ON videojuego (UPPER(titulo));
```

Otros tipos

Oracle tiene más tipos de índice, como los de **mapa de bits** (*bitmap*), pensados para almacenes de datos con columnas de pocos valores distintos, o las tablas **organizadas por índice** (IOT). No los usaremos en este curso.

7.4 ¿Cuándo crear un índice?

Conviene	No conviene
Claves ajenas	Tablas pequeñas (unos cientos de filas): se leen enteras igual de rápido
Columnas muy usadas en `WHERE`, `JOIN` u `ORDER BY`	Columnas que casi nunca se consultan
Columnas con muchos valores distintos (títulos, correos)	Tablas en las que se escribe mucho más de lo que se lee
	Columnas que se modifican continuamente

No sustituyas restricciones por índices

Un `CREATE UNIQUE INDEX` impide duplicados igual que una restricción `UNIQUE`, pero la regla de negocio queda «escondida» en un objeto de rendimiento. Declara siempre la **restricción** (`UNIQUE`) y deja que Oracle cree el índice. Así aparece en `USER_CONSTRAINTS`, tiene un mensaje de error claro y la pueden usar las claves ajenas.

7.5 Consultar y borrar índices

```
-- Índices de mis tablas y sus columnas
SELECT index_name, table_name, uniqueness
  FROM user_indexes
 ORDER BY table_name;
```

```sql
SELECT index_name, column_name, column_position
  FROM user_ind_columns
 WHERE table_name = 'BIBLIOTECA';

-- Borrar un índice
DROP INDEX ix_videojuego_pegi;
```

Un índice creado por una restricción no se puede borrar con `DROP INDEX` (**ORA-02429**): hay que borrar la restricción.

Resumen

- Un **índice** (árbol B) guarda valores ordenados con el ROWID de su fila: acelera búsquedas, pero ocupa espacio y ralentiza las escrituras.
- `PRIMARY KEY` y `UNIQUE` crean su índice automáticamente; las **claves ajenas no**, y conviene indexarlas.
- En un índice compuesto importa el **orden** de las columnas.
- Los índices **basados en función** sirven para consultas sobre expresiones como `UPPER(titulo)`.
- Las reglas de negocio se declaran como **restricciones**, no como índices.

RA 2 · APARTADO 8

Vistas

Una **vista** es una consulta guardada con nombre que se usa como si fuera una tabla. No almacena datos: cada vez que se consulta, Oracle ejecuta la consulta que la define sobre las tablas reales. Son la herramienta para ofrecer a cada usuario **la parte de la base de datos que necesita, con la forma que necesita**.

8.1 Para qué sirven CE 2f

Uso	Ejemplo en PixelVault
Calcular atributos derivados que el diseño decidió no guardar	La edad del jugador y la nota media de cada juego
Simplificar consultas que combinan muchas tablas	Un catálogo con el juego, su estudio y su precio en una sola «tabla»
Seguridad: mostrar solo algunas columnas o filas	El analista ve estadísticas, pero no correos ni fechas de nacimiento
Independencia: si cambia la estructura de las tablas, se ajusta la vista y los programas no notan el cambio	Si mañana el precio pasara a otra tabla, `v_catalogo` seguiría igual para la web

8.2 CREATE VIEW CE 2f

```
CREATE [OR REPLACE] [FORCE] VIEW nombre [(alias_columnas)] AS
    consulta
[WITH CHECK OPTION [CONSTRAINT nombre] | WITH READ ONLY];
```

Cláusula	Qué hace
`OR REPLACE`	Si la vista existe, la sustituye. Evita tener que borrarla antes y **conserva los permisos** concedidos sobre ella
`FORCE`	Crea la vista aunque las tablas aún no existan (quedará inválida hasta que existan)
`WITH CHECK OPTION`	Solo deja insertar o modificar a través de la vista filas que la vista seguiría mostrando
`WITH READ ONLY`	No se puede insertar, modificar ni borrar a través de la vista

Para crear vistas hace falta el privilegio de sistema `CREATE VIEW`, que dimos a `PIXELVAULT` en `01_usuarios.sql`.

Las consultas de las vistas

Las vistas de PixelVault usan `JOIN`, `LEFT JOIN`, `GROUP BY` y funciones como `AVG`. Son consultas del RA 3; aquí nos centramos en cómo se crean y se usan las vistas.

8.3 Las vistas de PixelVault

04_vistas.sql

```
-- ========================================================================
-- PixelVault · 04_vistas.sql
-- Vistas del esquema. Ejecutar conectado como PIXELVAULT.
-- ========================================================================

-- ------------------------------------------------------------------------
-- 1. Atributos derivados del diseño: se calculan, no se guardan
-- ------------------------------------------------------------------------

-- Edad de cada jugador, calculada a partir de su fecha de nacimiento
CREATE OR REPLACE VIEW v_jugador_edad AS
SELECT j.id_usuario,
       j.nick,
       u.email,
       j.fecha_nacimiento,
       TRUNC(MONTHS_BETWEEN(SYSDATE, j.fecha_nacimiento) / 12) AS edad
  FROM jugador j
  JOIN usuario u ON u.id_usuario = j.id_usuario;

-- Nota media y número de reseñas de cada juego (0 reseñas si no tiene)
CREATE OR REPLACE VIEW v_nota_media AS
SELECT v.id_juego,
       v.titulo,
       ROUND(AVG(r.nota), 1) AS nota_media,
       COUNT(r.id_resena)    AS num_resenas
  FROM videojuego v
  LEFT JOIN resena r ON r.id_juego = v.id_juego
 GROUP BY v.id_juego, v.titulo;

-- ------------------------------------------------------------------------
-- 2. Vista de catálogo: reúne varias tablas y es de solo lectura
-- ------------------------------------------------------------------------
CREATE OR REPLACE VIEW v_catalogo AS
SELECT v.id_juego,
       v.titulo,
       e.nombre   AS estudio,
       v.fecha_lanzamiento,
       v.pegi,
       v.tipo,
       p.precio                                  -- NULL si es free-to-play
  FROM videojuego v
  JOIN estudio e         ON e.id_estudio = v.id_estudio
  LEFT JOIN juego_pago p ON p.id_juego   = v.id_juego
  WITH READ ONLY;

-- ------------------------------------------------------------------------
-- 3. Vista actualizable con WITH CHECK OPTION:
--    solo deja insertar o modificar juegos que cumplan el WHERE
-- ------------------------------------------------------------------------
CREATE OR REPLACE VIEW v_juegos_pegi18 AS
SELECT id_juego, titulo, fecha_lanzamiento, pegi, tipo, id_estudio
  FROM videojuego
 WHERE pegi = 18
  WITH CHECK OPTION CONSTRAINT ck_v_juegos_pegi18;
```

Atributos derivados

En el diseño decidimos que `edad` y `nota_media` eran **atributos derivados**: no se guardan porque se calculan. Las vistas son el sitio natural para calcularlos:

```
SELECT nick, edad FROM v_jugador_edad;
```

```
NICK            EDAD
---------- ---------
NoctisRJ          20
LaTarasca         26
Kirbyto           14
Ezio_Lgr          25
```

La edad se calcula en el momento de la consulta, así que siempre es correcta. Dentro de un año, la misma consulta dará otras edades sin tocar ningún dato.

```
SELECT titulo, nota_media, num_resenas
  FROM v_nota_media
 WHERE num_resenas > 0;
```

```
TITULO               NOTA_MEDIA NUM_RESENAS
-------------------- ---------- -----------
Baldur's Gate 3               8           1
Hades                        10           1
Stardew Valley                9           1
```

> **¿Y por qué no una columna calculada?**
>
> Oracle permite **columnas virtuales** (`edad AS (...)`), pero su expresión no puede usar `SYSDATE` ni consultar otras tablas. La edad depende de la fecha actual y la nota media, de la tabla RESENA: las dos tienen que ir en una vista.

Una vista de catálogo de solo lectura

`v_catalogo` reúne VIDEOJUEGO, ESTUDIO y JUEGO_PAGO. Para la web de la tienda es como una tabla más:

```
SELECT titulo, estudio, precio
  FROM v_catalogo
 WHERE tipo = 'PAGO' AND precio < 30
 ORDER BY precio;
```

Como lleva `WITH READ ONLY` , cualquier intento de modificar datos a través de ella falla:

```
UPDATE v_catalogo SET pegi = 16 WHERE id_juego = 1;
-- ORA-42399: no se puede realizar una operación DML en una vista de solo lectura
```

8.4 Vistas actualizables y WITH CHECK OPTION CE 2f

A través de una vista **sencilla** se puede insertar, modificar y borrar, y los cambios van a la tabla. Una vista es actualizable si, entre otras condiciones:

- Se basa en **una sola tabla** (o, con matices, en una combinación donde se pueda identificar la fila de la tabla que se modifica).
- No tiene `GROUP BY`, `DISTINCT`, funciones de agregado (`AVG`, `COUNT`…) ni operadores de conjuntos.
- Las columnas que se modifican no son expresiones calculadas.

`v_nota_media` **no** es actualizable (tiene `AVG` y `GROUP BY`). `v_juegos_pegi18` **sí** lo es: se basa solo en VIDEOJUEGO.

¿Qué pasa si insertamos, a través de `v_juegos_pegi18`, un juego que no es PEGI 18? Sin control, la fila entraría en VIDEOJUEGO pero **desaparecería de la vista** nada más insertarla. `WITH CHECK OPTION` lo impide:

```
INSERT INTO v_juegos_pegi18 (titulo, pegi, tipo, id_estudio)
VALUES ('Juego infantil', 3, 'PAGO', 17);
-- ORA-01402: violación de la cláusula WHERE en la vista WITH CHECK OPTION

INSERT INTO v_juegos_pegi18 (titulo, pegi, tipo, id_estudio)
VALUES ('Juego adulto', 18, 'PAGO', 17);
-- 1 fila insertada: cumple el WHERE de la vista
```

8.5 Vistas y seguridad

Una vista es la forma más sencilla de dar acceso **parcial** a los datos: se concede permiso sobre la vista y **no** sobre las tablas. El usuario puede consultar la vista aunque no tenga ningún permiso sobre las tablas de las que sale.

En PixelVault, el analista solo puede consultar `v_catalogo` y `v_nota_media`: ve juegos, precios y notas, pero ningún dato personal de los jugadores. Lo configuramos en el apartado 9.

8.6 Consultar y borrar vistas

```
-- Mis vistas y la consulta que las define
SELECT view_name, text FROM user_views;

-- Borrar una vista (no borra ningún dato)
DROP VIEW v_juegos_pegi18;
```

Si se borra o se modifica una tabla de la que depende una vista, la vista queda **inválida** y falla al consultarla. Se recompila sola cuando vuelve a ser correcta, o a mano con `ALTER VIEW nombre COMPILE`.

Vistas materializadas

Una **vista materializada** sí guarda físicamente el resultado de su consulta y lo refresca cada cierto tiempo. Es útil para informes pesados (por ejemplo, ventas por mes), pero es un objeto distinto (`CREATE MATERIALIZED VIEW`) que no veremos en este curso.

Resumen

- Una **vista** es una consulta con nombre; no guarda datos y siempre muestra el estado actual de las tablas.
- Sirven para **calcular derivados**, **simplificar**, **proteger** datos y dar **independencia** a los programas.
- `CREATE OR REPLACE VIEW` la crea o sustituye conservando los permisos.
- `WITH READ ONLY` impide modificar a través de la vista; `WITH CHECK OPTION` impide modificaciones que dejarían filas fuera de ella.
- Las vistas con agregados, `GROUP BY` o `DISTINCT` no son actualizables.

RA 2 · APARTADO 9

Usuarios, roles y privilegios

Una base de datos de empresa la usan muchas personas y programas, y no todos deben poder hacer lo mismo. La web de la tienda tiene que registrar pedidos, pero no debería poder borrar tablas; un analista tiene que hacer estadísticas, pero no debería ver los correos de los clientes. En este apartado creamos los usuarios de PixelVault y les damos **exactamente** los permisos que necesitan, con el **lenguaje de control de datos (DCL)**.

9.1 Conceptos CE 2g

Usuario

: Cuenta con la que se conecta una persona o un programa. En Oracle, cada usuario tiene un **esquema** con sus objetos (apartado 1.5).

Privilegio de sistema

: Permiso para hacer una **acción** en la base de datos: conectarse, crear tablas, crear vistas…

Privilegio de objeto

: Permiso para hacer algo sobre **un objeto concreto** de otro usuario: consultar su tabla VIDEOJUEGO, insertar en su tabla PEDIDO…

Rol

: Conjunto de privilegios con nombre. Se dan los permisos al rol y el rol a los usuarios, en lugar de repetir los mismos permisos usuario a usuario.

Principio de mínimo privilegio

Cada usuario debe tener **solo** los permisos imprescindibles para su tarea, y ninguno más. Si una cuenta se ve comprometida, el daño queda limitado a lo que esa cuenta podía hacer. Es también una exigencia del RGPD: el acceso a datos personales debe limitarse a quien lo necesita.

9.2 Crear y gestionar usuarios CE 2g

Los usuarios los crea un administrador (en clase, `SYSTEM`), **conectado a la PDB**:

```
CREATE USER pixelvault IDENTIFIED BY "PixelVault_2026"
    DEFAULT TABLESPACE users        -- dónde irán sus tablas
    TEMPORARY TABLESPACE temp       -- espacio para ordenaciones
    QUOTA 100M ON users;            -- cuánto puede ocupar
```

Cláusula	Para qué sirve
`IDENTIFIED BY "contraseña"`	Contraseña. Entre comillas dobles si tiene símbolos o distingue mayúsculas
`DEFAULT TABLESPACE`	Tablespace donde se crearán sus objetos si no se indica otro
`QUOTA n ON tablespace`	Espacio máximo que puede ocupar (`UNLIMITED` sin límite). Sin cuota no puede guardar datos
`PASSWORD EXPIRE`	Obliga a cambiar la contraseña en la primera conexión
`ACCOUNT LOCK` / `ACCOUNT UNLOCK`	Bloquea o desbloquea la cuenta

Un usuario recién creado **no puede ni conectarse**: le falta el privilegio `CREATE SESSION` (error **ORA-01045**).

```
-- Cambiar la contraseña o la cuota
ALTER USER pv_app IDENTIFIED BY "Nueva_Clave_2026";
ALTER USER pixelvault QUOTA 200M ON users;

-- Bloquear y desbloquear
ALTER USER pv_analista ACCOUNT LOCK;      -- al conectar: ORA-28000
ALTER USER pv_analista ACCOUNT UNLOCK;

-- Borrar un usuario (CASCADE borra también todos sus objetos)
DROP USER pv_analista;
DROP USER pixelvault CASCADE;
```

Un usuario propietario y otros que solo usan

Es una buena práctica que las tablas pertenezcan a un usuario **propietario** (`PIXELVAULT`) con el que no se conecta nadie en el día a día, y que la aplicación y las personas usen **otras cuentas** con permisos limitados. Si la aplicación se conectara como propietaria, un fallo de seguridad le permitiría hacer `DROP TABLE` .

9.3 Privilegios de sistema CE 2g

```
GRANT privilegio [, privilegio ...] TO usuario_o_rol [WITH ADMIN OPTION];
```

Privilegio	Permite
`CREATE SESSION`	Conectarse
`CREATE TABLE`	Crear tablas (y sus índices) **en su propio esquema**
`CREATE VIEW`	Crear vistas en su esquema
`CREATE SEQUENCE`	Crear secuencias en su esquema
`CREATE SYNONYM`	Crear sinónimos en su esquema

Privilegio	Permite
`CREATE ANY TABLE`, `DROP ANY TABLE`...	Lo mismo, **en cualquier esquema**
`UNLIMITED TABLESPACE`	Ocupar espacio sin límite en cualquier tablespace

Así recibe sus privilegios el propietario de PixelVault:

```
GRANT CREATE SESSION, CREATE TABLE, CREATE VIEW, CREATE SEQUENCE
   TO pixelvault;
```

Cuidado con ANY

Los privilegios con `ANY` afectan a **todos los esquemas** de la base de datos, incluidos los de otros proyectos. `DROP ANY TABLE` permite borrar cualquier tabla de cualquiera. Son privilegios de administrador y no se dan a usuarios normales.

`WITH ADMIN OPTION` permite al que recibe el privilegio concederlo a otros. Solo tiene sentido para administradores.

9.4 Privilegios de objeto CE 2g

El **propietario** de un objeto tiene todos los permisos sobre él y puede concederlos a otros:

```
GRANT privilegio [, privilegio ...] ON [esquema.]objeto
   TO usuario_o_rol [WITH GRANT OPTION];
```

Privilegio	Sobre	Permite
`SELECT`	Tablas, vistas, secuencias	Consultar
`INSERT`	Tablas, vistas	Insertar filas
`UPDATE [(columnas)]`	Tablas, vistas	Modificar filas (todas las columnas o solo las indicadas)
`DELETE`	Tablas, vistas	Borrar filas
`REFERENCES`	Tablas	Crear claves ajenas que apunten a la tabla
`ALTER`, `INDEX`	Tablas	Modificar la tabla o crear índices sobre ella
`EXECUTE`	Procedimientos y funciones	Ejecutarlos

```
-- La aplicación puede leer el catálogo...
GRANT SELECT ON videojuego TO rol_pv_aplicacion;
-- ...y registrar pedidos, pero no borrarlos ni modificarlos
GRANT SELECT, INSERT ON pedido TO rol_pv_aplicacion;
-- Un ejemplo de UPDATE limitado a columnas: solo puede cambiar el nick
```

```
GRANT UPDATE (nick) ON jugador TO pv_app;
```

`WITH GRANT OPTION` permite al que recibe el permiso concederlo a su vez. No se puede usar al conceder a un **rol**.

9.5 Roles CE 2g

Con dos usuarios la diferencia es pequeña, pero imagina diez programas y veinte analistas: dar treinta permisos a cada uno sería inmanejable. Con roles:

1. Se crea el rol (lo hace el administrador).
2. Se le dan los privilegios (el propietario de los objetos).
3. Se concede el rol a los usuarios.

Si mañana la aplicación necesita un permiso más, se da **al rol** y todos sus usuarios lo reciben a la vez.

```
CREATE ROLE rol_pv_lectura;                          -- 1 (SYSTEM)
GRANT SELECT ON v_catalogo   TO rol_pv_lectura;      -- 2 (PIXELVAULT)
GRANT SELECT ON v_nota_media TO rol_pv_lectura;
GRANT rol_pv_lectura TO pv_analista;                 -- 3 (SYSTEM)
```

Roles predefinidos

Oracle trae roles ya creados. Conviene conocerlos, sobre todo para **no** usarlos a la ligera:

Rol	Qué contiene	¿Usarlo?
`CONNECT`	Hoy, solo `CREATE SESSION`	Se puede, pero es más claro dar `CREATE SESSION`
`RESOURCE`	Crear tablas, secuencias, procedimientos, disparadores, tipos…	Cómodo para cuentas de desarrollo; desde 12c ya no incluye cuota ilimitada
`DBA`	**Todos** los privilegios de sistema	**Nunca** a usuarios normales

Oracle 23ai / 26ai: DB_DEVELOPER_ROLE

Desde 23ai existe el rol `DB_DEVELOPER_ROLE`, pensado precisamente para cuentas de desarrollo: incluye lo necesario para crear tablas, vistas, secuencias, procedimientos, etc., sin privilegios de administración. En XE 21c no existe.

9.6 Quitar privilegios: REVOKE CE 2g

```
REVOKE privilegio ON objeto FROM usuario_o_rol;   -- privilegio de objeto
REVOKE privilegio FROM usuario_o_rol;             -- privilegio de sistema o rol

REVOKE DELETE ON resena FROM rol_pv_aplicacion;
REVOKE rol_pv_lectura FROM pv_analista;
```

> ⚠ **Revocar en cascada... o no**
>
> - Si se revoca un **privilegio de objeto** que se había dado `WITH GRANT OPTION`, se revoca también a todos los que lo recibieron de ese usuario (en cascada).
> - Si se revoca un **privilegio de sistema** dado `WITH ADMIN OPTION`, los que lo recibieron de ese usuario **lo conservan**.

9.7 Usar las tablas de otro esquema

`PV_APP` no tiene tablas: usa las de `PIXELVAULT`. Por eso tiene que escribir el esquema delante:

```
SELECT titulo, precio FROM pixelvault.v_catalogo;
```

Para no escribirlo en cada sentencia, la sesión puede cambiar su esquema por defecto:

```
ALTER SESSION SET CURRENT_SCHEMA = pixelvault;
SELECT titulo, precio FROM v_catalogo;     -- ya no hace falta el prefijo
```

Esto no da ningún permiso: solo cambia dónde se buscan los nombres. Otra alternativa son los **sinónimos** (`CREATE SYNONYM catalogo FOR pixelvault.v_catalogo`), que crean un nombre alternativo para un objeto.

9.8 PixelVault: usuarios y permisos CE 2g

La configuración completa está repartida en dos scripts, porque cada parte la ejecuta un usuario distinto:

01_usuarios.sql (SYSTEM)

```
-- ========================================================================
-- PixelVault · 01_usuarios.sql
-- Crea el propietario del esquema, los roles y los usuarios de la BD.
--
-- Ejecutar como SYSTEM, conectado a la PDB:
--   · Oracle Database XE 21c ............... servicio XEPDB1
--   · Oracle Database 23ai / 26ai Free ..... servicio FREEPDB1
-- NUNCA en la CDB raíz (CDB$ROOT): daría ORA-65096.
-- ========================================================================

-- Comprobación: debe mostrar XEPDB1 o FREEPDB1, no CDB$ROOT
SHOW CON_NAME

-- ------------------------------------------------------------------------
-- 1. Propietario del esquema: crea y posee todas las tablas y vistas
-- ------------------------------------------------------------------------
CREATE USER pixelvault IDENTIFIED BY "PixelVault_2026"
    DEFAULT TABLESPACE users
    TEMPORARY TABLESPACE temp
    QUOTA 100M ON users;

-- Privilegios de sistema: los justos para crear sus objetos
GRANT CREATE SESSION,
      CREATE TABLE,
```

```
      CREATE VIEW,
      CREATE SEQUENCE
   TO pixelvault;

-- ----------------------------------------------------------------------
-- 2. Roles: agrupan los privilegios sobre los objetos
--    (el propietario les dará los permisos en 05_privilegios.sql)
-- ----------------------------------------------------------------------
CREATE ROLE rol_pv_aplicacion;   -- lectura y escritura de datos
CREATE ROLE rol_pv_lectura;      -- solo consulta de vistas

-- ----------------------------------------------------------------------
-- 3. Usuarios que usan la BD pero no poseen objetos
--    (sin cuota: no pueden crear nada)
-- ----------------------------------------------------------------------
CREATE USER pv_app      IDENTIFIED BY "App_PixelVault_2026";
CREATE USER pv_analista IDENTIFIED BY "Analista_PV_2026";

GRANT CREATE SESSION TO pv_app, pv_analista;

GRANT rol_pv_aplicacion TO pv_app;
GRANT rol_pv_lectura    TO pv_analista;
```

05_privilegios.sql (PIXELVAULT)

```
-- ======================================================================
-- PixelVault · 05_privilegios.sql
-- Privilegios sobre objetos. Ejecutar conectado como PIXELVAULT:
-- el propietario de un objeto puede conceder permisos sobre él.
-- Los roles se crearon en 01_usuarios.sql (como SYSTEM).
-- ======================================================================

-- ----------------------------------------------------------------------
-- ROL_PV_APLICACION: la web de la tienda
--   · Lee el catálogo, pero no lo modifica
--   · Lee y escribe los datos de los jugadores y sus compras
-- ----------------------------------------------------------------------
GRANT SELECT ON estudio       TO rol_pv_aplicacion;
GRANT SELECT ON videojuego    TO rol_pv_aplicacion;
GRANT SELECT ON idioma_juego  TO rol_pv_aplicacion;
GRANT SELECT ON juego_pago    TO rol_pv_aplicacion;
GRANT SELECT ON juego_f2p     TO rol_pv_aplicacion;
GRANT SELECT ON genero        TO rol_pv_aplicacion;
GRANT SELECT ON juego_genero  TO rol_pv_aplicacion;
GRANT SELECT ON plataforma    TO rol_pv_aplicacion;
GRANT SELECT ON logro         TO rol_pv_aplicacion;
GRANT SELECT ON dlc           TO rol_pv_aplicacion;

GRANT SELECT, INSERT, UPDATE, DELETE ON usuario      TO rol_pv_aplicacion;
GRANT SELECT, INSERT, UPDATE, DELETE ON jugador      TO rol_pv_aplicacion;
GRANT SELECT, INSERT, UPDATE, DELETE ON biblioteca   TO rol_pv_aplicacion;
GRANT SELECT, INSERT, UPDATE, DELETE ON juega        TO rol_pv_aplicacion;
GRANT SELECT, INSERT, DELETE         ON amistad      TO rol_pv_aplicacion;
GRANT SELECT, INSERT                 ON desbloquea   TO rol_pv_aplicacion;
GRANT SELECT, INSERT, UPDATE, DELETE ON resena       TO rol_pv_aplicacion;

-- Los pedidos no se borran ni se modifican (obligación contable)
GRANT SELECT, INSERT ON pedido       TO rol_pv_aplicacion;
GRANT SELECT, INSERT ON linea_pedido TO rol_pv_aplicacion;
```

```
GRANT SELECT ON v_catalogo     TO rol_pv_aplicacion;
GRANT SELECT ON v_nota_media   TO rol_pv_aplicacion;
GRANT SELECT ON v_jugador_edad TO rol_pv_aplicacion;

-- ---------------------------------------------------------------------
-- ROL_PV_LECTURA: analistas de negocio
--   Solo vistas sin datos personales (principio de mínimo privilegio
--   y minimización de datos del RGPD)
-- ---------------------------------------------------------------------
GRANT SELECT ON v_catalogo   TO rol_pv_lectura;
GRANT SELECT ON v_nota_media TO rol_pv_lectura;
```

Comprobar que funciona

Conectados como `PV_ANALISTA`:

```
SELECT titulo, nota_media FROM pixelvault.v_nota_media;      -- funciona

SELECT email FROM pixelvault.usuario;
-- ORA-00942: la tabla o vista no existe

DELETE FROM pixelvault.v_catalogo;
-- ORA-01031: privilegios insuficientes
```

¿Por qué dice que la tabla no existe si sí existe?

Cuando un usuario **no tiene ningún permiso** sobre un objeto, Oracle responde **ORA-00942** («no existe») en lugar de «no tienes permiso». Es deliberado: así no revela a un usuario sin permisos qué objetos hay en la base de datos. En cambio, si puede ver el objeto pero no hacer esa operación (el analista ve `v_catalogo` pero no puede borrar), el error es **ORA-01031** (privilegios insuficientes).

Conectados como `PV_APP`:

```
INSERT INTO pixelvault.pedido (id_jugador) VALUES (102);    -- funciona
DELETE FROM pixelvault.pedido WHERE num_pedido = 5001;      -- ORA-01031
DROP TABLE pixelvault.pedido;                               -- ORA-01031
```

9.9 Consultar los privilegios

Vista del diccionario	Muestra
`SESSION_PRIVS`	Los privilegios de sistema **activos** en mi sesión ahora
`USER_SYS_PRIVS`	Privilegios de sistema concedidos directamente a mí
`USER_ROLE_PRIVS`	Roles que tengo
`USER_TAB_PRIVS`	Privilegios de objeto que he dado o recibido

Vista del diccionario	Muestra
`ROLE_TAB_PRIVS`	Privilegios de objeto de los roles que tengo

```
-- Como PV_APP: ¿qué puedo hacer y sobre qué?
SELECT * FROM user_role_privs;
SELECT table_name, privilege FROM role_tab_privs ORDER BY table_name;
```

Oracle 23ai / 26ai: privilegios de esquema

En 23ai se pueden conceder privilegios sobre **todo un esquema** de una vez, incluidas las tablas que se creen en el futuro:

```
GRANT SELECT ANY TABLE ON SCHEMA pixelvault TO rol_pv_lectura;
```

Es cómodo, pero da acceso a **todas** las tablas y vistas del esquema. Para el analista de PixelVault no nos sirve: debe ver solo dos vistas.

Resumen

- El **usuario** se crea con `CREATE USER`, con tablespace y **cuota**; sin `CREATE SESSION` no puede conectarse.
- **Privilegios de sistema** (acciones: `CREATE TABLE` …) y **de objeto** (sobre un objeto: `SELECT ON videojuego` …). Se conceden con `GRANT` y se quitan con `REVOKE`.
- Los **roles** agrupan privilegios; se conceden a los usuarios. `DBA` nunca a usuarios normales.
- **Mínimo privilegio**: un propietario que crea los objetos y otras cuentas que solo los usan.
- Sin permisos sobre un objeto, Oracle dice **ORA-00942** (no existe); con permisos insuficientes, **ORA-01031**.

RA 2 · APARTADO 10

El diccionario de datos

¿Cómo sabe SQL Developer qué tablas tienes, qué columnas tienen y qué restricciones? Lo consulta en el **diccionario de datos**: un conjunto de tablas y vistas internas donde Oracle describe todo lo que existe en la base de datos. Aprender a consultarlo te permite **comprobar** que tu script ha creado exactamente lo que querías, sin depender de ninguna herramienta gráfica.

10.1 Tres familias de vistas CE 2h

Casi todas las vistas del diccionario existen en tres versiones, según el prefijo:

Prefijo	Muestra	Ejemplo
`USER_`	Los objetos **de tu esquema**	`USER_TABLES` : mis tablas
`ALL_`	Los objetos a los que **tienes acceso** (tuyos y de otros con permiso)	`ALL_TABLES` : las tablas que puedo usar
`DBA_`	**Todos** los objetos de la base de datos (solo administradores)	`DBA_TABLES` : todas las tablas

Las vistas `ALL_` y `DBA_` tienen una columna `OWNER` con el propietario; las `USER_` no la necesitan.

Para saber qué vistas existen, consulta la propia lista del diccionario:

```
SELECT table_name, comments
  FROM dictionary
 WHERE table_name LIKE 'USER_%'
 ORDER BY table_name;
```

Los nombres se guardan en mayúsculas

El diccionario guarda los nombres de los objetos en **mayúsculas**. `WHERE table_name = 'videojuego'` no encuentra nada; hay que escribir `'VIDEOJUEGO'` .

10.2 Las vistas más útiles CE 2h

Vista	Qué describe
`USER_OBJECTS`	Todos mis objetos (tablas, vistas, índices, secuencias…) y si son válidos
`USER_TABLES`	Mis tablas
`USER_TAB_COLUMNS`	Columnas de mis tablas: tipo, tamaño, si admiten nulos
`USER_CONSTRAINTS`	Restricciones: tipo (`P` , `R` , `U` , `C`), condición del `CHECK` , regla de borrado

Vista	Qué describe
`USER_CONS_COLUMNS`	Columnas que forman cada restricción
`USER_INDEXES` / `USER_IND_COLUMNS`	Índices y sus columnas
`USER_VIEWS`	Vistas y su consulta
`USER_SEQUENCES`	Secuencias
`USER_TAB_IDENTITY_COLS`	Columnas `IDENTITY`
`USER_TAB_COMMENTS` / `USER_COL_COMMENTS`	Comentarios de tablas y columnas
`USER_RECYCLEBIN`	Lo que hay en la papelera
`USER_USERS` , `USER_TS_QUOTAS`	Datos de mi usuario y mis cuotas

10.3 Comprobar el esquema de PixelVault

Después de ejecutar los scripts, estas consultas permiten verificar el resultado.

¿Están todas las tablas?

```
SELECT COUNT(*) AS num_tablas FROM user_tables;          -- debe dar 20

SELECT object_type, COUNT(*) AS cuantos
  FROM user_objects
 GROUP BY object_type
 ORDER BY object_type;
```

¿Qué columnas tiene una tabla?

```
SELECT column_name, data_type, data_length, data_precision, data_scale, nullable
  FROM user_tab_columns
 WHERE table_name = 'VIDEOJUEGO'
 ORDER BY column_id;
```

Es lo mismo que muestra `DESC videojuego` , con más detalle.

¿Qué restricciones tiene?

```
SELECT constraint_name, constraint_type, search_condition, delete_rule, status
  FROM user_constraints
 WHERE table_name = 'VIDEOJUEGO'
 ORDER BY constraint_type, constraint_name;
```

```
CONSTRAINT_NAME         C SEARCH_CONDITION                  DELETE_RULE STATUS
----------------------- - --------------------------------- ----------- -------
CK_VIDEOJUEGO_PEGI      C pegi IN (3, 7, 12, 16, 18)                    ENABLED
CK_VIDEOJUEGO_SECUELA   C id_juego_original <> id_juego                 ENABLED
CK_VIDEOJUEGO_TIPO      C tipo IN ('PAGO', 'F2P')                       ENABLED
SYS_C008751             C "ID_ESTUDIO" IS NOT NULL                      ENABLED
...
PK_VIDEOJUEGO           P                                               ENABLED
FK_VIDEOJUEGO_ESTUDIO   R                                   NO ACTION   ENABLED
FK_VIDEOJUEGO_ORIGINAL  R                                   SET NULL    ENABLED
UK_VIDEOJUEGO_ORIGINAL  U                                               ENABLED
```

Fíjate en dos cosas:

- Los `NOT NULL` son restricciones de tipo `C` con nombre automático (`SYS_C...`), porque no les pusimos nombre.
- La columna `DELETE_RULE` muestra la acción de cada clave ajena: `NO ACTION` (restringir), `CASCADE` o `SET NULL`.

¿A qué tabla apunta cada clave ajena?

Una clave ajena apunta a una restricción (la PK o UK de la otra tabla), así que hay que combinar `USER_CONSTRAINTS` consigo misma:

```sql
SELECT hija.table_name       AS tabla,
       hija.constraint_name  AS clave_ajena,
       madre.table_name      AS referencia_a,
       hija.delete_rule      AS al_borrar
  FROM user_constraints hija
  JOIN user_constraints madre ON madre.constraint_name = hija.r_constraint_name
 WHERE hija.constraint_type = 'R'
 ORDER BY hija.table_name;
```

¿Qué índices hay y sobre qué columnas?

```sql
SELECT i.table_name, i.index_name, i.uniqueness, c.column_name, c.column_position
  FROM user_indexes i
  JOIN user_ind_columns c ON c.index_name = i.index_name
 ORDER BY i.table_name, i.index_name, c.column_position;
```

¿Hay objetos inválidos?

```sql
SELECT object_name, object_type
  FROM user_objects
 WHERE status = 'INVALID';
```

Si una vista aparece como inválida, es que alguna tabla de la que depende ha cambiado o no existe.

10.4 Y en SQL Developer

Todo lo anterior es lo que SQL Developer consulta por debajo cuando abres una tabla y miras las pestañas **Columnas**, **Restricciones** o **Índices**. La pestaña **SQL** va más allá: reconstruye el `CREATE TABLE` completo a partir del diccionario. Se puede obtener lo mismo con una función del paquete `DBMS_METADATA`:

```
SELECT DBMS_METADATA.GET_DDL('TABLE', 'VIDEOJUEGO') FROM dual;
```

Resumen

- El **diccionario de datos** describe todos los objetos de la base de datos y se consulta con SQL.
- Prefijos: `USER_` (lo mío), `ALL_` (a lo que tengo acceso) y `DBA_` (todo).
- Vistas clave: `USER_TABLES`, `USER_TAB_COLUMNS`, `USER_CONSTRAINTS`, `USER_CONS_COLUMNS`, `USER_INDEXES`, `USER_VIEWS`.
- Los nombres están en **mayúsculas**.
- Se usa para **verificar** que un script ha creado exactamente lo previsto.

RA 2 · APARTADO 11

El script completo de PixelVault

Aquí están, juntos y en orden, todos los scripts que hemos ido construyendo. Con ellos se crea desde cero la base de datos completa de PixelVault en Oracle: usuarios, 20 tablas con todas sus restricciones, índices, vistas, permisos y unos datos de prueba.

Descargar todos los scripts de Oracle (zip)

11.1 Orden de ejecución CE 2b CE 2h

Paso	Script	Conectado como	Descarga
1	Crea el propietario, los roles y los usuarios	`SYSTEM` en la PDB	01_usuarios.sql
2	Crea las 20 tablas con sus restricciones	`PIXELVAULT`	02_tablas.sql
3	Crea los índices adicionales	`PIXELVAULT`	03_indices.sql
4	Crea las vistas	`PIXELVAULT`	04_vistas.sql
5	Concede permisos a los roles	`PIXELVAULT`	05_privilegios.sql
6	Carga datos de prueba	`PIXELVAULT`	06_datos_prueba.sql
7	Comprueba que las restricciones funcionan (deben fallar)	`PIXELVAULT`	07_pruebas_restricciones.sql
—	Lo borra todo para empezar de cero	`PIXELVAULT` y `SYSTEM`	99_borrar.sql

Con SQL Developer

1. Abre la conexión de **SYSTEM** a la PDB (`XEPDB1` o `FREEPDB1`), abre `01_usuarios.sql` (**Archivo → Abrir**) y ejecútalo como script con F5.
2. Crea una conexión nueva para **PIXELVAULT** (contraseña `PixelVault_2026`) y conéctate.
3. Con esa conexión, abre y ejecuta con F5, uno detrás de otro, los scripts 02 a 06.
4. Ejecuta las sentencias de `07_pruebas_restricciones.sql` **una a una** con ^ Ctrl + Enter ⏎ y comprueba que cada una da el error que indica su comentario.

Con SQL*Plus o SQLcl

Desde la carpeta donde están los scripts:

```
$ sqlplus system@//localhost:1521/XEPDB1
SQL> @01_usuarios.sql
SQL> CONNECT pixelvault@//localhost:1521/XEPDB1
SQL> @02_tablas.sql
SQL> @03_indices.sql
SQL> @04_vistas.sql
SQL> @05_privilegios.sql
SQL> @06_datos_prueba.sql
```

(En Oracle 23ai/26ai Free, cambia `XEPDB1` por `FREEPDB1`).

Empezar de cero

Si algo sale mal a mitad, lo más limpio es borrarlo todo y repetir. Conectado como `SYSTEM`, basta con la parte B de `99_borrar.sql`: `DROP USER pixelvault CASCADE` elimina el usuario **y todos sus objetos**; después se borran los otros dos usuarios y los roles, y se vuelve a empezar por el paso 1.

11.2 Los scripts

01 · Usuarios

01_usuarios.sql

```
-- ========================================================================
-- PixelVault · 01_usuarios.sql
-- Crea el propietario del esquema, los roles y los usuarios de la BD.
--
-- Ejecutar como SYSTEM, conectado a la PDB:
--   · Oracle Database XE 21c ............... servicio XEPDB1
--   · Oracle Database 23ai / 26ai Free ..... servicio FREEPDB1
-- NUNCA en la CDB raíz (CDB$ROOT): daría ORA-65096.
-- ========================================================================

-- Comprobación: debe mostrar XEPDB1 o FREEPDB1, no CDB$ROOT
SHOW CON_NAME

-- ------------------------------------------------------------------------
-- 1. Propietario del esquema: crea y posee todas las tablas y vistas
-- ------------------------------------------------------------------------
CREATE USER pixelvault IDENTIFIED BY "PixelVault_2026"
    DEFAULT TABLESPACE users
    TEMPORARY TABLESPACE temp
    QUOTA 100M ON users;

-- Privilegios de sistema: los justos para crear sus objetos
GRANT CREATE SESSION,
      CREATE TABLE,
      CREATE VIEW,
      CREATE SEQUENCE
   TO pixelvault;

-- ------------------------------------------------------------------------
-- 2. Roles: agrupan los privilegios sobre los objetos
--    (el propietario les dará los permisos en 05_privilegios.sql)
```

```
-- ------------------------------------------------------------------------
CREATE ROLE rol_pv_aplicacion;   -- lectura y escritura de datos
CREATE ROLE rol_pv_lectura;      -- solo consulta de vistas

-- ------------------------------------------------------------------------
-- 3. Usuarios que usan la BD pero no poseen objetos
--    (sin cuota: no pueden crear nada)
-- ------------------------------------------------------------------------
CREATE USER pv_app      IDENTIFIED BY "App_PixelVault_2026";
CREATE USER pv_analista IDENTIFIED BY "Analista_PV_2026";

GRANT CREATE SESSION TO pv_app, pv_analista;

GRANT rol_pv_aplicacion TO pv_app;
GRANT rol_pv_lectura    TO pv_analista;
```

02 · Tablas

02_tablas.sql

```
-- ========================================================================
-- PixelVault · 02_tablas.sql
-- Crea las 20 tablas del esquema relacional con todas sus restricciones.
--
-- Ejecutar conectado como PIXELVAULT (en la PDB).
-- Compatible con Oracle XE 21c y Oracle 23ai / 26ai.
--
-- Convenio de nombres de las restricciones:
--   pk_tabla            clave primaria
--   fk_tabla_referida   clave ajena
--   uk_tabla_columna    clave alternativa (UNIQUE)
--   ck_tabla_regla      restricción CHECK
-- Las tablas se crean en orden: primero las referenciadas.
-- ========================================================================

-- ------------------------------------------------------------------------
-- Tablas sin claves ajenas
-- ------------------------------------------------------------------------
CREATE TABLE usuario (
    id_usuario      NUMBER(10)    GENERATED BY DEFAULT AS IDENTITY,
    email           VARCHAR2(120) NOT NULL,
    nombre          VARCHAR2(50)  NOT NULL,
    apellidos       VARCHAR2(80),
    fecha_registro  DATE          DEFAULT SYSDATE NOT NULL,
    CONSTRAINT pk_usuario       PRIMARY KEY (id_usuario),
    CONSTRAINT uk_usuario_email UNIQUE (email),
    CONSTRAINT ck_usuario_email CHECK (email LIKE '%_@_%.__%')
);

CREATE TABLE estudio (
    id_estudio  NUMBER(10)    GENERATED BY DEFAULT AS IDENTITY,
    nombre      VARCHAR2(100) NOT NULL,
    pais        VARCHAR2(60),
    web         VARCHAR2(200),
    CONSTRAINT pk_estudio PRIMARY KEY (id_estudio)
);

CREATE TABLE genero (
    id_genero  NUMBER(10)   GENERATED BY DEFAULT AS IDENTITY,
```

```
    nombre     VARCHAR2(40) NOT NULL,
    CONSTRAINT pk_genero        PRIMARY KEY (id_genero),
    CONSTRAINT uk_genero_nombre UNIQUE (nombre)
);

CREATE TABLE plataforma (
    id_plataforma  NUMBER(10)   GENERATED BY DEFAULT AS IDENTITY,
    nombre         VARCHAR2(60) NOT NULL,
    fabricante     VARCHAR2(60),
    CONSTRAINT pk_plataforma        PRIMARY KEY (id_plataforma),
    CONSTRAINT uk_plataforma_nombre UNIQUE (nombre)
);

-- ---------------------------------------------------------------------
-- Jerarquía USUARIO → JUGADOR, DESARROLLADOR (una tabla por entidad)
-- ---------------------------------------------------------------------
CREATE TABLE jugador (
    id_usuario        NUMBER(10)    NOT NULL,
    nick              VARCHAR2(30)  NOT NULL,
    fecha_nacimiento  DATE          NOT NULL,
    calle             VARCHAR2(100),
    numero            VARCHAR2(10),
    cp                VARCHAR2(10),
    ciudad            VARCHAR2(60),
    pais              VARCHAR2(60),
    CONSTRAINT pk_jugador         PRIMARY KEY (id_usuario),
    CONSTRAINT fk_jugador_usuario FOREIGN KEY (id_usuario)
        REFERENCES usuario (id_usuario) ON DELETE CASCADE,
    CONSTRAINT uk_jugador_nick    UNIQUE (nick)
);

CREATE TABLE desarrollador (
    id_usuario  NUMBER(10)   NOT NULL,
    cargo       VARCHAR2(60),
    id_estudio  NUMBER(10)   NOT NULL,
    CONSTRAINT pk_desarrollador         PRIMARY KEY (id_usuario),
    CONSTRAINT fk_desarrollador_usuario FOREIGN KEY (id_usuario)
        REFERENCES usuario (id_usuario) ON DELETE CASCADE,
    CONSTRAINT fk_desarrollador_estudio FOREIGN KEY (id_estudio)
        REFERENCES estudio (id_estudio)
);

-- ---------------------------------------------------------------------
-- VIDEOJUEGO y sus subtipos (R-01, R-04, R-07)
-- ---------------------------------------------------------------------
CREATE TABLE videojuego (
    id_juego           NUMBER(10)    GENERATED BY DEFAULT AS IDENTITY,
    titulo             VARCHAR2(150) NOT NULL,
    fecha_lanzamiento  DATE,
    pegi               NUMBER(2)     NOT NULL,
    tipo               VARCHAR2(4)   NOT NULL,
    id_estudio         NUMBER(10)    NOT NULL,
    id_juego_original  NUMBER(10),
    CONSTRAINT pk_videojuego          PRIMARY KEY (id_juego),
    CONSTRAINT fk_videojuego_estudio  FOREIGN KEY (id_estudio)
        REFERENCES estudio (id_estudio),
    CONSTRAINT fk_videojuego_original FOREIGN KEY (id_juego_original)
        REFERENCES videojuego (id_juego) ON DELETE SET NULL,
    CONSTRAINT uk_videojuego_original UNIQUE (id_juego_original),       -- 1:1
    CONSTRAINT ck_videojuego_pegi     CHECK (pegi IN (3, 7, 12, 16, 18)),           -- R-01
```

```
    CONSTRAINT ck_videojuego_tipo     CHECK (tipo IN ('PAGO', 'F2P')),              -- R-04
    CONSTRAINT ck_videojuego_secuela  CHECK (id_juego_original <> id_juego)         -- R-07
);

CREATE TABLE idioma_juego (
    id_juego  NUMBER(10)   NOT NULL,
    idioma    VARCHAR2(40) NOT NULL,
    CONSTRAINT pk_idioma_juego PRIMARY KEY (id_juego, idioma),
    CONSTRAINT fk_idioma_juego FOREIGN KEY (id_juego)
        REFERENCES videojuego (id_juego) ON DELETE CASCADE
);

CREATE TABLE juego_pago (
    id_juego  NUMBER(10)   NOT NULL,
    precio    NUMBER(6,2)  NOT NULL,
    CONSTRAINT pk_juego_pago        PRIMARY KEY (id_juego),
    CONSTRAINT fk_juego_pago_juego  FOREIGN KEY (id_juego)
        REFERENCES videojuego (id_juego) ON DELETE CASCADE,
    CONSTRAINT ck_juego_pago_precio CHECK (precio >= 0)                         -- R-03
);

CREATE TABLE juego_f2p (
    id_juego          NUMBER(10) NOT NULL,
    tiene_micropagos  CHAR(1)    DEFAULT 'N' NOT NULL,
    CONSTRAINT pk_juego_f2p            PRIMARY KEY (id_juego),
    CONSTRAINT fk_juego_f2p_juego      FOREIGN KEY (id_juego)
        REFERENCES videojuego (id_juego) ON DELETE CASCADE,
    CONSTRAINT ck_juego_f2p_micropagos CHECK (tiene_micropagos IN ('S', 'N'))
);

CREATE TABLE juego_genero (
    id_juego   NUMBER(10) NOT NULL,
    id_genero  NUMBER(10) NOT NULL,
    CONSTRAINT pk_juego_genero        PRIMARY KEY (id_juego, id_genero),
    CONSTRAINT fk_juego_genero_juego  FOREIGN KEY (id_juego)
        REFERENCES videojuego (id_juego) ON DELETE CASCADE,
    CONSTRAINT fk_juego_genero_genero FOREIGN KEY (id_genero)
        REFERENCES genero (id_genero)
);

-- ---------------------------------------------------------------------
-- Relaciones del jugador con los juegos
-- ---------------------------------------------------------------------
CREATE TABLE biblioteca (
    id_jugador         NUMBER(10) NOT NULL,
    id_juego           NUMBER(10) NOT NULL,
    fecha_adquisicion  DATE       DEFAULT SYSDATE NOT NULL,
    CONSTRAINT pk_biblioteca         PRIMARY KEY (id_jugador, id_juego),
    CONSTRAINT fk_biblioteca_jugador FOREIGN KEY (id_jugador)
        REFERENCES jugador (id_usuario) ON DELETE CASCADE,
    CONSTRAINT fk_biblioteca_juego   FOREIGN KEY (id_juego)
        REFERENCES videojuego (id_juego)
);

CREATE TABLE juega (
    id_jugador     NUMBER(10)  NOT NULL,
    id_juego       NUMBER(10)  NOT NULL,
    id_plataforma  NUMBER(10)  NOT NULL,
    horas_jugadas  NUMBER(7,1) DEFAULT 0 NOT NULL,
    CONSTRAINT pk_juega            PRIMARY KEY (id_jugador, id_juego, id_plataforma),
```

```
    CONSTRAINT fk_juega_jugador    FOREIGN KEY (id_jugador)
        REFERENCES jugador (id_usuario) ON DELETE CASCADE,
    CONSTRAINT fk_juega_juego      FOREIGN KEY (id_juego)
        REFERENCES videojuego (id_juego),
    CONSTRAINT fk_juega_plataforma FOREIGN KEY (id_plataforma)
        REFERENCES plataforma (id_plataforma),
    CONSTRAINT ck_juega_horas      CHECK (horas_jugadas >= 0)                  -- R-03
);

CREATE TABLE amistad (
    id_jugador  NUMBER(10) NOT NULL,
    id_amigo    NUMBER(10) NOT NULL,
    CONSTRAINT pk_amistad         PRIMARY KEY (id_jugador, id_amigo),
    CONSTRAINT fk_amistad_jugador FOREIGN KEY (id_jugador)
        REFERENCES jugador (id_usuario) ON DELETE CASCADE,
    CONSTRAINT fk_amistad_amigo   FOREIGN KEY (id_amigo)
        REFERENCES jugador (id_usuario) ON DELETE CASCADE,
    CONSTRAINT ck_amistad_orden   CHECK (id_jugador < id_amigo)               -- R-05 y R-06
);

-- ---------------------------------------------------------------------
-- Entidades débiles: LOGRO (identificación) y DLC (existencia)
-- ---------------------------------------------------------------------
CREATE TABLE logro (
    id_juego     NUMBER(10)    NOT NULL,
    num_logro    NUMBER(4)     NOT NULL,
    nombre       VARCHAR2(100) NOT NULL,
    descripcion  VARCHAR2(400),
    puntos       NUMBER(4)     NOT NULL,
    CONSTRAINT pk_logro        PRIMARY KEY (id_juego, num_logro),
    CONSTRAINT fk_logro_juego  FOREIGN KEY (id_juego)
        REFERENCES videojuego (id_juego) ON DELETE CASCADE,
    CONSTRAINT ck_logro_puntos CHECK (puntos > 0)                             -- R-03
);

CREATE TABLE desbloquea (
    id_jugador  NUMBER(10) NOT NULL,
    id_juego    NUMBER(10) NOT NULL,
    num_logro   NUMBER(4)  NOT NULL,
    CONSTRAINT pk_desbloquea         PRIMARY KEY (id_jugador, id_juego, num_logro),
    CONSTRAINT fk_desbloquea_jugador FOREIGN KEY (id_jugador)
        REFERENCES jugador (id_usuario) ON DELETE CASCADE,
    CONSTRAINT fk_desbloquea_logro   FOREIGN KEY (id_juego, num_logro)   -- FK compuesta
        REFERENCES logro (id_juego, num_logro) ON DELETE CASCADE
);

CREATE TABLE dlc (
    id_dlc             NUMBER(10)    GENERATED BY DEFAULT AS IDENTITY,
    titulo             VARCHAR2(150) NOT NULL,
    precio             NUMBER(6,2)   NOT NULL,
    fecha_lanzamiento  DATE,
    id_juego           NUMBER(10)    NOT NULL,
    CONSTRAINT pk_dlc        PRIMARY KEY (id_dlc),
    CONSTRAINT fk_dlc_juego  FOREIGN KEY (id_juego)
        REFERENCES videojuego (id_juego) ON DELETE CASCADE,
    CONSTRAINT ck_dlc_precio CHECK (precio >= 0)                              -- R-03
);

-- ---------------------------------------------------------------------
-- Pedidos
```

```sql
-- ------------------------------------------------------------------------
CREATE TABLE pedido (
    num_pedido  NUMBER(10) GENERATED BY DEFAULT AS IDENTITY,
    fecha       DATE       DEFAULT SYSDATE NOT NULL,
    id_jugador  NUMBER(10) NOT NULL,
    CONSTRAINT pk_pedido         PRIMARY KEY (num_pedido),
    CONSTRAINT fk_pedido_jugador FOREIGN KEY (id_jugador)
        REFERENCES jugador (id_usuario)
);

CREATE TABLE linea_pedido (
    num_pedido     NUMBER(10)  NOT NULL,
    id_juego       NUMBER(10)  NOT NULL,
    precio_pagado  NUMBER(6,2) NOT NULL,
    CONSTRAINT pk_linea_pedido        PRIMARY KEY (num_pedido, id_juego),
    CONSTRAINT fk_linea_pedido_pedido FOREIGN KEY (num_pedido)
        REFERENCES pedido (num_pedido) ON DELETE CASCADE,
    CONSTRAINT fk_linea_pedido_juego  FOREIGN KEY (id_juego)
        REFERENCES juego_pago (id_juego),
    CONSTRAINT ck_linea_pedido_precio CHECK (precio_pagado >= 0)               -- R-03
);

-- ------------------------------------------------------------------------
-- Reseñas (cuelgan de la agregación BIBLIOTECA)
-- ------------------------------------------------------------------------
CREATE TABLE resena (
    id_resena    NUMBER(10)     GENERATED BY DEFAULT AS IDENTITY,
    nota         NUMBER(2)      NOT NULL,
    texto        VARCHAR2(2000),
    fecha        DATE           DEFAULT SYSDATE NOT NULL,
    recomendado  CHAR(1)        NOT NULL,
    id_jugador   NUMBER(10)     NOT NULL,
    id_juego     NUMBER(10)     NOT NULL,
    CONSTRAINT pk_resena             PRIMARY KEY (id_resena),
    CONSTRAINT fk_resena_biblioteca  FOREIGN KEY (id_jugador, id_juego)
        REFERENCES biblioteca (id_jugador, id_juego) ON DELETE CASCADE,
    CONSTRAINT uk_resena_posesion    UNIQUE (id_jugador, id_juego),         -- 1 reseña por
posesión
    CONSTRAINT ck_resena_nota        CHECK (nota BETWEEN 1 AND 10),            -- R-02
    CONSTRAINT ck_resena_recomendado CHECK (recomendado IN ('S', 'N'))
);

-- ------------------------------------------------------------------------
-- Comentarios: quedan guardados en el diccionario de datos
-- ------------------------------------------------------------------------
COMMENT ON TABLE videojuego IS 'Catálogo de juegos. Subtipos: JUEGO_PAGO y JUEGO_F2P';
COMMENT ON COLUMN videojuego.tipo IS 'PAGO o F2P. Indica en qué tabla de subtipo tiene fila';
COMMENT ON COLUMN videojuego.id_juego_original IS 'Juego del que este es secuela directa (1:1)';
COMMENT ON TABLE biblioteca IS 'Juegos que posee cada jugador (relación POSEE)';
COMMENT ON TABLE resena IS 'Una reseña como máximo por juego poseído';
```

03 · Índices

03_indices.sql

```sql
-- ========================================================================
-- PixelVault · 03_indices.sql
-- Índices adicionales. Ejecutar conectado como PIXELVAULT.
--
```

```sql
-- Oracle ya crea automáticamente un índice único para cada PRIMARY KEY
-- y cada UNIQUE. Lo que NO crea es un índice para las claves ajenas:
-- conviene crearlo en las columnas FK que no sean ya el inicio de
-- otro índice, porque se usan en las combinaciones (JOIN) y porque
-- evitan bloqueos al borrar filas de la tabla referenciada.
-- ========================================================================

-- Claves ajenas simples
CREATE INDEX ix_desarrollador_estudio ON desarrollador (id_estudio);
CREATE INDEX ix_videojuego_estudio    ON videojuego (id_estudio);
CREATE INDEX ix_juego_genero_genero   ON juego_genero (id_genero);
CREATE INDEX ix_biblioteca_juego      ON biblioteca (id_juego);
CREATE INDEX ix_juega_juego           ON juega (id_juego);
CREATE INDEX ix_juega_plataforma      ON juega (id_plataforma);
CREATE INDEX ix_amistad_amigo         ON amistad (id_amigo);
CREATE INDEX ix_dlc_juego             ON dlc (id_juego);
CREATE INDEX ix_pedido_jugador        ON pedido (id_jugador);
CREATE INDEX ix_linea_pedido_juego    ON linea_pedido (id_juego);

-- Clave ajena compuesta (hacia LOGRO)
CREATE INDEX ix_desbloquea_logro      ON desbloquea (id_juego, num_logro);

-- Índice basado en función: búsquedas de títulos sin distinguir mayúsculas
-- (sirve para consultas con WHERE UPPER(titulo) = 'HADES')
CREATE INDEX ix_videojuego_titulo_up  ON videojuego (UPPER(titulo));

-- Índice para filtrar el catálogo por clasificación de edad
CREATE INDEX ix_videojuego_pegi       ON videojuego (pegi);
```

04 · Vistas

04_vistas.sql

```sql
-- ========================================================================
-- PixelVault · 04_vistas.sql
-- Vistas del esquema. Ejecutar conectado como PIXELVAULT.
-- ========================================================================

-- ------------------------------------------------------------------------
-- 1. Atributos derivados del diseño: se calculan, no se guardan
-- ------------------------------------------------------------------------

-- Edad de cada jugador, calculada a partir de su fecha de nacimiento
CREATE OR REPLACE VIEW v_jugador_edad AS
SELECT j.id_usuario,
       j.nick,
       u.email,
       j.fecha_nacimiento,
       TRUNC(MONTHS_BETWEEN(SYSDATE, j.fecha_nacimiento) / 12) AS edad
  FROM jugador j
  JOIN usuario u ON u.id_usuario = j.id_usuario;

-- Nota media y número de reseñas de cada juego (0 reseñas si no tiene)
CREATE OR REPLACE VIEW v_nota_media AS
SELECT v.id_juego,
       v.titulo,
       ROUND(AVG(r.nota), 1) AS nota_media,
       COUNT(r.id_resena)    AS num_resenas
  FROM videojuego v
```

```
  LEFT JOIN resena r ON r.id_juego = v.id_juego
 GROUP BY v.id_juego, v.titulo;

-- ---------------------------------------------------------------------
-- 2. Vista de catálogo: reúne varias tablas y es de solo lectura
-- ---------------------------------------------------------------------
CREATE OR REPLACE VIEW v_catalogo AS
SELECT v.id_juego,
       v.titulo,
       e.nombre   AS estudio,
       v.fecha_lanzamiento,
       v.pegi,
       v.tipo,
       p.precio                                  -- NULL si es free-to-play
  FROM videojuego v
  JOIN estudio e         ON e.id_estudio = v.id_estudio
  LEFT JOIN juego_pago p ON p.id_juego   = v.id_juego
  WITH READ ONLY;

-- ---------------------------------------------------------------------
-- 3. Vista actualizable con WITH CHECK OPTION:
--    solo deja insertar o modificar juegos que cumplan el WHERE
-- ---------------------------------------------------------------------
CREATE OR REPLACE VIEW v_juegos_pegi18 AS
SELECT id_juego, titulo, fecha_lanzamiento, pegi, tipo, id_estudio
  FROM videojuego
 WHERE pegi = 18
  WITH CHECK OPTION CONSTRAINT ck_v_juegos_pegi18;
```

05 · Privilegios

05_privilegios.sql

```
-- ======================================================================
-- PixelVault · 05_privilegios.sql
-- Privilegios sobre objetos. Ejecutar conectado como PIXELVAULT:
-- el propietario de un objeto puede conceder permisos sobre él.
-- Los roles se crearon en 01_usuarios.sql (como SYSTEM).
-- ======================================================================

-- ---------------------------------------------------------------------
-- ROL_PV_APLICACION: la web de la tienda
--   · Lee el catálogo, pero no lo modifica
--   · Lee y escribe los datos de los jugadores y sus compras
-- ---------------------------------------------------------------------
GRANT SELECT ON estudio       TO rol_pv_aplicacion;
GRANT SELECT ON videojuego    TO rol_pv_aplicacion;
GRANT SELECT ON idioma_juego  TO rol_pv_aplicacion;
GRANT SELECT ON juego_pago    TO rol_pv_aplicacion;
GRANT SELECT ON juego_f2p     TO rol_pv_aplicacion;
GRANT SELECT ON genero        TO rol_pv_aplicacion;
GRANT SELECT ON juego_genero  TO rol_pv_aplicacion;
GRANT SELECT ON plataforma    TO rol_pv_aplicacion;
GRANT SELECT ON logro         TO rol_pv_aplicacion;
GRANT SELECT ON dlc           TO rol_pv_aplicacion;

GRANT SELECT, INSERT, UPDATE, DELETE ON usuario      TO rol_pv_aplicacion;
GRANT SELECT, INSERT, UPDATE, DELETE ON jugador      TO rol_pv_aplicacion;
GRANT SELECT, INSERT, UPDATE, DELETE ON biblioteca   TO rol_pv_aplicacion;
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON juega        TO rol_pv_aplicacion;
GRANT SELECT, INSERT, DELETE         ON amistad      TO rol_pv_aplicacion;
GRANT SELECT, INSERT                 ON desbloquea   TO rol_pv_aplicacion;
GRANT SELECT, INSERT, UPDATE, DELETE ON resena       TO rol_pv_aplicacion;

-- Los pedidos no se borran ni se modifican (obligación contable)
GRANT SELECT, INSERT ON pedido       TO rol_pv_aplicacion;
GRANT SELECT, INSERT ON linea_pedido TO rol_pv_aplicacion;

GRANT SELECT ON v_catalogo     TO rol_pv_aplicacion;
GRANT SELECT ON v_nota_media   TO rol_pv_aplicacion;
GRANT SELECT ON v_jugador_edad TO rol_pv_aplicacion;

-- ---------------------------------------------------------------------
-- ROL_PV_LECTURA: analistas de negocio
--   Solo vistas sin datos personales (principio de mínimo privilegio
--   y minimización de datos del RGPD)
-- ---------------------------------------------------------------------
GRANT SELECT ON v_catalogo   TO rol_pv_lectura;
GRANT SELECT ON v_nota_media TO rol_pv_lectura;
```

06 · Datos de prueba

06_datos_prueba.sql

```
-- =====================================================================
-- PixelVault · 06_datos_prueba.sql
-- Carga mínima de datos para probar restricciones y vistas.
-- Ejecutar conectado como PIXELVAULT.
--
-- La sentencia INSERT se estudia a fondo en el RA 4; aquí solo se usa
-- para tener filas con las que comprobar que el esquema funciona.
-- Títulos, estudios, fechas y PEGI son reales; jugadores, precios,
-- pedidos y logros son inventados.
-- =====================================================================

-- ---------------------------------------------------------------------
-- Catálogo
-- ---------------------------------------------------------------------
INSERT INTO estudio (id_estudio, nombre, pais) VALUES (10, 'Rockstar North', 'Reino Unido');
INSERT INTO estudio (id_estudio, nombre, pais) VALUES (11, 'Larian Studios', 'Bélgica');
INSERT INTO estudio (id_estudio, nombre, pais) VALUES (12, 'Nintendo EPD', 'Japón');
INSERT INTO estudio (id_estudio, nombre, pais) VALUES (13, 'FromSoftware', 'Japón');
INSERT INTO estudio (id_estudio, nombre, pais) VALUES (14, 'CD Projekt Red', 'Polonia');
INSERT INTO estudio (id_estudio, nombre, pais) VALUES (15, 'Supergiant Games', 'EE. UU.');
INSERT INTO estudio (id_estudio, nombre, pais) VALUES (16, 'Team Cherry', 'Australia');
INSERT INTO estudio (id_estudio, nombre, pais) VALUES (17, 'ConcernedApe', 'EE. UU.');
INSERT INTO estudio (id_estudio, nombre, pais) VALUES (18, 'Epic Games', 'EE. UU.');
INSERT INTO estudio (id_estudio, nombre, pais) VALUES (19, 'Valve', 'EE. UU.');

INSERT INTO videojuego (id_juego, titulo, fecha_lanzamiento, pegi, tipo, id_estudio)
VALUES (1, 'Grand Theft Auto V', DATE '2013-09-17', 18, 'PAGO', 10);
INSERT INTO videojuego (id_juego, titulo, fecha_lanzamiento, pegi, tipo, id_estudio)
VALUES (2, 'Baldur''s Gate 3', DATE '2023-08-03', 18, 'PAGO', 11);
INSERT INTO videojuego (id_juego, titulo, fecha_lanzamiento, pegi, tipo, id_estudio)
VALUES (3, 'The Legend of Zelda: Tears of the Kingdom', DATE '2023-05-12', 12, 'PAGO', 12);
INSERT INTO videojuego (id_juego, titulo, fecha_lanzamiento, pegi, tipo, id_estudio)
VALUES (4, 'Elden Ring', DATE '2022-02-25', 16, 'PAGO', 13);
INSERT INTO videojuego (id_juego, titulo, fecha_lanzamiento, pegi, tipo, id_estudio)
```

```
VALUES (5, 'Hollow Knight', DATE '2017-02-24', 7, 'PAGO', 16);
INSERT INTO videojuego (id_juego, titulo, fecha_lanzamiento, pegi, tipo, id_estudio)
VALUES (6, 'Hades', DATE '2020-09-17', 12, 'PAGO', 15);
INSERT INTO videojuego (id_juego, titulo, fecha_lanzamiento, pegi, tipo, id_estudio)
VALUES (7, 'The Witcher 3: Wild Hunt', DATE '2015-05-19', 18, 'PAGO', 14);
INSERT INTO videojuego (id_juego, titulo, fecha_lanzamiento, pegi, tipo, id_estudio)
VALUES (8, 'Stardew Valley', DATE '2016-02-26', 7, 'PAGO', 17);
INSERT INTO videojuego (id_juego, titulo, fecha_lanzamiento, pegi, tipo, id_estudio)
VALUES (9, 'Fortnite', DATE '2017-07-25', 12, 'F2P', 18);
INSERT INTO videojuego (id_juego, titulo, fecha_lanzamiento, pegi, tipo, id_estudio)
VALUES (10, 'Counter-Strike 2', DATE '2023-09-27', 18, 'F2P', 19);
-- Secuela: Hades II apunta a su original, Hades (id 6)
INSERT INTO videojuego (id_juego, titulo, fecha_lanzamiento, pegi, tipo, id_estudio,
id_juego_original)
VALUES (11, 'Hades II', DATE '2025-09-25', 12, 'PAGO', 15, 6);

INSERT INTO juego_pago (id_juego, precio) VALUES (1, 29.99);
INSERT INTO juego_pago (id_juego, precio) VALUES (2, 59.99);
INSERT INTO juego_pago (id_juego, precio) VALUES (3, 69.99);
INSERT INTO juego_pago (id_juego, precio) VALUES (4, 59.99);
INSERT INTO juego_pago (id_juego, precio) VALUES (5, 14.79);
INSERT INTO juego_pago (id_juego, precio) VALUES (6, 24.50);
INSERT INTO juego_pago (id_juego, precio) VALUES (7, 29.99);
INSERT INTO juego_pago (id_juego, precio) VALUES (8, 13.99);
INSERT INTO juego_pago (id_juego, precio) VALUES (11, 29.50);
INSERT INTO juego_f2p (id_juego, tiene_micropagos) VALUES (9, 'S');
INSERT INTO juego_f2p (id_juego, tiene_micropagos) VALUES (10, 'S');

INSERT INTO idioma_juego (id_juego, idioma) VALUES (2, 'español');
INSERT INTO idioma_juego (id_juego, idioma) VALUES (2, 'inglés');
INSERT INTO idioma_juego (id_juego, idioma) VALUES (2, 'francés');
INSERT INTO idioma_juego (id_juego, idioma) VALUES (6, 'español');
INSERT INTO idioma_juego (id_juego, idioma) VALUES (6, 'inglés');

INSERT INTO genero (id_genero, nombre) VALUES (1, 'Acción');
INSERT INTO genero (id_genero, nombre) VALUES (2, 'Rol');
INSERT INTO genero (id_genero, nombre) VALUES (3, 'Aventura');
INSERT INTO genero (id_genero, nombre) VALUES (4, 'Roguelike');
INSERT INTO genero (id_genero, nombre) VALUES (5, 'Metroidvania');
INSERT INTO genero (id_genero, nombre) VALUES (6, 'Simulación');
INSERT INTO genero (id_genero, nombre) VALUES (7, 'Shooter');

INSERT INTO juego_genero (id_juego, id_genero) VALUES (1, 1);
INSERT INTO juego_genero (id_juego, id_genero) VALUES (2, 2);
INSERT INTO juego_genero (id_juego, id_genero) VALUES (3, 3);
INSERT INTO juego_genero (id_juego, id_genero) VALUES (4, 1);
INSERT INTO juego_genero (id_juego, id_genero) VALUES (4, 2);
INSERT INTO juego_genero (id_juego, id_genero) VALUES (5, 5);
INSERT INTO juego_genero (id_juego, id_genero) VALUES (6, 4);
INSERT INTO juego_genero (id_juego, id_genero) VALUES (7, 2);
INSERT INTO juego_genero (id_juego, id_genero) VALUES (8, 6);
INSERT INTO juego_genero (id_juego, id_genero) VALUES (9, 7);
INSERT INTO juego_genero (id_juego, id_genero) VALUES (10, 7);
INSERT INTO juego_genero (id_juego, id_genero) VALUES (11, 4);

INSERT INTO plataforma (id_plataforma, nombre, fabricante) VALUES (1, 'PC', NULL);
INSERT INTO plataforma (id_plataforma, nombre, fabricante) VALUES (2, 'PlayStation 5', 'Sony');
INSERT INTO plataforma (id_plataforma, nombre, fabricante) VALUES (3, 'Xbox Series X|S',
'Microsoft');
```

```sql
INSERT INTO plataforma (id_plataforma, nombre, fabricante) VALUES (4, 'Nintendo Switch 2',
'Nintendo');

INSERT INTO logro (id_juego, num_logro, nombre, puntos) VALUES (4, 1, 'Primera victoria', 10);
INSERT INTO logro (id_juego, num_logro, nombre, puntos) VALUES (4, 2, 'Explorador de las Tierras
Intermedias', 30);
INSERT INTO logro (id_juego, num_logro, nombre, puntos) VALUES (6, 1, 'Primera victoria', 10);
INSERT INTO logro (id_juego, num_logro, nombre, puntos) VALUES (6, 2, 'Escapar del Inframundo',
50);

INSERT INTO dlc (id_dlc, titulo, precio, fecha_lanzamiento, id_juego)
VALUES (1, 'Shadow of the Erdtree', 39.99, DATE '2024-06-21', 4);
INSERT INTO dlc (id_dlc, titulo, precio, fecha_lanzamiento, id_juego)
VALUES (2, 'Hearts of Stone', 9.99, DATE '2015-10-13', 7);
INSERT INTO dlc (id_dlc, titulo, precio, fecha_lanzamiento, id_juego)
VALUES (3, 'Blood and Wine', 19.99, DATE '2016-05-31', 7);

-- ---------------------------------------------------------------------
-- Usuarios, jugadores y desarrolladores
-- ---------------------------------------------------------------------
INSERT INTO usuario (id_usuario, email, nombre, apellidos) VALUES (101, 'noctis@ejemplo.es',
'Noa', 'Sáenz Ruiz');
INSERT INTO usuario (id_usuario, email, nombre, apellidos) VALUES (102, 'tarasca@ejemplo.es',
'Lucía', 'Garrido Pérez');
INSERT INTO usuario (id_usuario, email, nombre, apellidos) VALUES (103, 'kirbyto@ejemplo.es',
'Hugo', 'Martínez Sanz');
INSERT INTO usuario (id_usuario, email, nombre, apellidos) VALUES (104, 'ezio@ejemplo.es',
'Iker', 'López Ochoa');
INSERT INTO usuario (id_usuario, email, nombre, apellidos) VALUES (201, 'marta.dev@ejemplo.es',
'Marta', 'Ibáñez Gil');
INSERT INTO usuario (id_usuario, email, nombre, apellidos) VALUES (301, 'soporte@ejemplo.es',
'Soporte', 'PixelVault');

INSERT INTO jugador (id_usuario, nick, fecha_nacimiento, ciudad, pais) VALUES (101, 'NoctisRJ',
DATE '2006-03-14', 'Logroño',   'España');
INSERT INTO jugador (id_usuario, nick, fecha_nacimiento, ciudad, pais) VALUES (102, 'LaTarasca',
DATE '1999-11-02', 'Haro',      'España');
INSERT INTO jugador (id_usuario, nick, fecha_nacimiento, ciudad, pais) VALUES (103, 'Kirbyto',
DATE '2012-06-30', 'Calahorra', 'España');
INSERT INTO jugador (id_usuario, nick, fecha_nacimiento, ciudad, pais) VALUES (104, 'Ezio_Lgr',
DATE '2001-01-21', 'Logroño',   'España');

-- Jerarquía solapada: 104 es jugador y además desarrollador
INSERT INTO desarrollador (id_usuario, cargo, id_estudio) VALUES (104, 'QA tester', 15);
INSERT INTO desarrollador (id_usuario, cargo, id_estudio) VALUES (201, 'Diseñadora de niveles',
11);
-- El usuario 301 (soporte) no es de ningún subtipo: jerarquía parcial

-- ---------------------------------------------------------------------
-- Actividad de los jugadores
-- ---------------------------------------------------------------------
INSERT INTO biblioteca (id_jugador, id_juego, fecha_adquisicion) VALUES (101, 6, DATE '2026-09-
01');
INSERT INTO biblioteca (id_jugador, id_juego, fecha_adquisicion) VALUES (101, 5, DATE '2026-09-
01');
INSERT INTO biblioteca (id_jugador, id_juego, fecha_adquisicion) VALUES (101, 4, DATE '2026-09-
02');
INSERT INTO biblioteca (id_jugador, id_juego, fecha_adquisicion) VALUES (101, 9, DATE '2025-12-
24');
```

```
INSERT INTO biblioteca (id_jugador, id_juego, fecha_adquisicion) VALUES (102, 6, DATE '2026-09-
02');
INSERT INTO biblioteca (id_jugador, id_juego, fecha_adquisicion) VALUES (102, 8, DATE '2026-09-
02');
INSERT INTO biblioteca (id_jugador, id_juego, fecha_adquisicion) VALUES (103, 5, DATE '2026-03-
10');
INSERT INTO biblioteca (id_jugador, id_juego, fecha_adquisicion) VALUES (103, 9, DATE '2026-01-
05');
INSERT INTO biblioteca (id_jugador, id_juego, fecha_adquisicion) VALUES (104, 2, DATE '2026-09-
10');
INSERT INTO biblioteca (id_jugador, id_juego, fecha_adquisicion) VALUES (104, 7, DATE '2026-09-
10');
INSERT INTO biblioteca (id_jugador, id_juego, fecha_adquisicion) VALUES (104, 10, DATE '2025-06-
01');

INSERT INTO juega (id_jugador, id_juego, id_plataforma, horas_jugadas) VALUES (101, 6, 1, 40);
INSERT INTO juega (id_jugador, id_juego, id_plataforma, horas_jugadas) VALUES (101, 6, 4, 12);
INSERT INTO juega (id_jugador, id_juego, id_plataforma, horas_jugadas) VALUES (101, 4, 1, 1.5);
INSERT INTO juega (id_jugador, id_juego, id_plataforma, horas_jugadas) VALUES (101, 4, 2, 0.4);
INSERT INTO juega (id_jugador, id_juego, id_plataforma, horas_jugadas) VALUES (102, 6, 1, 25);
INSERT INTO juega (id_jugador, id_juego, id_plataforma, horas_jugadas) VALUES (102, 8, 4, 80);
INSERT INTO juega (id_jugador, id_juego, id_plataforma, horas_jugadas) VALUES (103, 5, 4, 10);
INSERT INTO juega (id_jugador, id_juego, id_plataforma, horas_jugadas) VALUES (103, 9, 4, 30);
INSERT INTO juega (id_jugador, id_juego, id_plataforma, horas_jugadas) VALUES (104, 2, 1, 120);
INSERT INTO juega (id_jugador, id_juego, id_plataforma, horas_jugadas) VALUES (104, 10, 1, 300);

-- La amistad se guarda una sola vez, con el identificador menor primero
INSERT INTO amistad (id_jugador, id_amigo) VALUES (101, 102);
INSERT INTO amistad (id_jugador, id_amigo) VALUES (101, 104);
INSERT INTO amistad (id_jugador, id_amigo) VALUES (102, 103);

INSERT INTO desbloquea (id_jugador, id_juego, num_logro) VALUES (101, 6, 1);
INSERT INTO desbloquea (id_jugador, id_juego, num_logro) VALUES (101, 6, 2);
INSERT INTO desbloquea (id_jugador, id_juego, num_logro) VALUES (101, 4, 1);
INSERT INTO desbloquea (id_jugador, id_juego, num_logro) VALUES (102, 6, 1);

INSERT INTO pedido (num_pedido, fecha, id_jugador) VALUES (5001, DATE '2026-09-01', 101);
INSERT INTO pedido (num_pedido, fecha, id_jugador) VALUES (5002, DATE '2026-09-02', 102);
INSERT INTO pedido (num_pedido, fecha, id_jugador) VALUES (5003, DATE '2026-09-02', 101);
INSERT INTO pedido (num_pedido, fecha, id_jugador) VALUES (5004, DATE '2026-09-10', 104);

INSERT INTO linea_pedido (num_pedido, id_juego, precio_pagado) VALUES (5001, 6, 24.50);
INSERT INTO linea_pedido (num_pedido, id_juego, precio_pagado) VALUES (5001, 5, 14.79);
INSERT INTO linea_pedido (num_pedido, id_juego, precio_pagado) VALUES (5002, 6, 19.99);
INSERT INTO linea_pedido (num_pedido, id_juego, precio_pagado) VALUES (5002, 8, 13.99);
INSERT INTO linea_pedido (num_pedido, id_juego, precio_pagado) VALUES (5003, 4, 59.99);
INSERT INTO linea_pedido (num_pedido, id_juego, precio_pagado) VALUES (5004, 2, 49.99);
INSERT INTO linea_pedido (num_pedido, id_juego, precio_pagado) VALUES (5004, 7, 9.99);

INSERT INTO resena (id_resena, nota, texto, recomendado, id_jugador, id_juego)
VALUES (1, 10, 'Cada partida es distinta. Imprescindible.', 'S', 101, 6);
INSERT INTO resena (id_resena, nota, texto, recomendado, id_jugador, id_juego)
VALUES (2, 9, 'Relajante y adictivo.', 'S', 102, 8);
INSERT INTO resena (id_resena, nota, texto, recomendado, id_jugador, id_juego)
VALUES (3, 8, 'Enorme, aunque el tercer acto se hace largo.', 'S', 104, 2);

COMMIT;

-- ------------------------------------------------------------------------
-- Hemos dado valores explícitos a columnas IDENTITY. Para que el
```

```
-- próximo valor automático no choque con ellos, se sincroniza el
-- generador con el máximo existente (START WITH LIMIT VALUE).
-- ------------------------------------------------------------------
ALTER TABLE usuario    MODIFY id_usuario    GENERATED BY DEFAULT AS IDENTITY (START WITH LIMIT
VALUE);
ALTER TABLE estudio    MODIFY id_estudio    GENERATED BY DEFAULT AS IDENTITY (START WITH LIMIT
VALUE);
ALTER TABLE genero     MODIFY id_genero     GENERATED BY DEFAULT AS IDENTITY (START WITH LIMIT
VALUE);
ALTER TABLE plataforma MODIFY id_plataforma GENERATED BY DEFAULT AS IDENTITY (START WITH LIMIT
VALUE);
ALTER TABLE videojuego MODIFY id_juego      GENERATED BY DEFAULT AS IDENTITY (START WITH LIMIT
VALUE);
ALTER TABLE dlc        MODIFY id_dlc        GENERATED BY DEFAULT AS IDENTITY (START WITH LIMIT
VALUE);
ALTER TABLE pedido     MODIFY num_pedido    GENERATED BY DEFAULT AS IDENTITY (START WITH LIMIT
VALUE);
ALTER TABLE resena     MODIFY id_resena     GENERATED BY DEFAULT AS IDENTITY (START WITH LIMIT
VALUE);
```

07 · Pruebas

07_pruebas_restricciones.sql

```
-- ======================================================================
-- PixelVault · 07_pruebas_restricciones.sql
-- Sentencias que DEBEN FALLAR. Sirven para comprobar que cada
-- restricción está bien implantada. Ejecutar como PIXELVAULT, una a una
-- (en SQL Developer: cursor sobre la sentencia y Ctrl+Intro).
-- Al lado de cada una, el error que debe aparecer.
-- ======================================================================

-- R-01 · PEGI no válido ............................. ORA-02290 (ck_videojuego_pegi)
INSERT INTO videojuego (titulo, pegi, tipo, id_estudio) VALUES ('Juego raro', 15, 'PAGO', 10);

-- R-04 · tipo no válido ............................. ORA-02290 (ck_videojuego_tipo)
INSERT INTO videojuego (titulo, pegi, tipo, id_estudio) VALUES ('Juego raro', 12, 'FREE', 10);

-- Tipo de dato · 'GRATIS' no cabe en VARCHAR2(4) ..... ORA-12899 (antes incluso del CHECK)
INSERT INTO videojuego (titulo, pegi, tipo, id_estudio) VALUES ('Juego raro', 12, 'GRATIS', 10);

-- R-07 · un juego secuela de sí mismo ................ ORA-02290 (ck_videojuego_secuela)
UPDATE videojuego SET id_juego_original = 5 WHERE id_juego = 5;

-- 1:1 · dos secuelas del mismo original .............. ORA-00001 (uk_videojuego_original)
UPDATE videojuego SET id_juego_original = 6 WHERE id_juego = 5;

-- R-02 · nota fuera de rango ......................... ORA-02290 (ck_resena_nota)
INSERT INTO resena (nota, recomendado, id_jugador, id_juego) VALUES (11, 'S', 101, 5);

-- R-06 · amistad guardada al revés ................... ORA-02290 (ck_amistad_orden)
INSERT INTO amistad (id_jugador, id_amigo) VALUES (102, 101);

-- NOT NULL · juego sin estudio ....................... ORA-01400
INSERT INTO videojuego (titulo, pegi, tipo) VALUES ('Sin estudio', 12, 'PAGO');

-- UNIQUE · nick repetido ............................. ORA-00001 (uk_jugador_nick)
INSERT INTO usuario (email, nombre) VALUES ('otro@ejemplo.es', 'Otro');
INSERT INTO jugador (id_usuario, nick, fecha_nacimiento)
```

```sql
SELECT id_usuario, 'NoctisRJ', DATE '2000-01-01' FROM usuario WHERE email = 'otro@ejemplo.es';
ROLLBACK;

-- FK · estudio inexistente ........................... ORA-02291 (fk_videojuego_estudio)
INSERT INTO videojuego (titulo, pegi, tipo, id_estudio) VALUES ('Huérfano', 12, 'PAGO', 99);

-- FK · borrar un estudio con juegos (FromSoftware) .... ORA-02292 (fk_videojuego_estudio)
DELETE FROM estudio WHERE id_estudio = 13;

-- FK · pedir un juego free-to-play ................... ORA-02291 (fk_linea_pedido_juego)
INSERT INTO linea_pedido (num_pedido, id_juego, precio_pagado) VALUES (5001, 9, 0);

-- Agregación · reseñar un juego que no se tiene ...... ORA-02291 (fk_resena_biblioteca)
INSERT INTO resena (nota, recomendado, id_jugador, id_juego) VALUES (7, 'N', 102, 4);

-- Vista con CHECK OPTION · juego no PEGI 18 .......... ORA-01402
INSERT INTO v_juegos_pegi18 (titulo, pegi, tipo, id_estudio) VALUES ('Juego infantil', 3, 'PAGO',
17);

-- Vista de solo lectura .............................. ORA-42399
UPDATE v_catalogo SET pegi = 16 WHERE id_juego = 1;

-- ---------------------------------------------------------------------
-- Y estas DEBEN FUNCIONAR (acciones ON DELETE). Se deshacen al final.
-- ---------------------------------------------------------------------
-- CASCADE: al borrar Elden Ring desaparecen sus logros, su DLC...
-- ...pero falla por BIBLIOTECA (restricción sin cascada): ORA-02292.
DELETE FROM videojuego WHERE id_juego = 4;

-- SET NULL: al borrar Hades, Hades II se queda sin original.
-- Falla igualmente porque Hades está en bibliotecas y pedidos: ORA-02292.
-- Probemos con un juego nuevo sin ventas:
INSERT INTO videojuego (id_juego, titulo, pegi, tipo, id_estudio) VALUES (900, 'Juego de prueba',
7, 'PAGO', 15);
INSERT INTO videojuego (id_juego, titulo, pegi, tipo, id_estudio, id_juego_original)
VALUES (901, 'Juego de prueba 2', 7, 'PAGO', 15, 900);
DELETE FROM videojuego WHERE id_juego = 900;
SELECT id_juego, titulo, id_juego_original FROM videojuego WHERE id_juego = 901;  -- original:
NULL
ROLLBACK;
```

99 · Borrar

99_borrar.sql

```sql
-- ======================================================================
-- PixelVault · 99_borrar.sql
-- Elimina todo para poder empezar de cero.
-- ======================================================================

-- ---------------------------------------------------------------------
-- PARTE A · Conectado como PIXELVAULT: vistas y tablas
-- Se borran en orden inverso al de creación (primero las que
-- referencian). PURGE evita que vayan a la papelera de reciclaje.
-- ---------------------------------------------------------------------
DROP VIEW v_juegos_pegi18;
DROP VIEW v_catalogo;
DROP VIEW v_nota_media;
DROP VIEW v_jugador_edad;
```

```
DROP TABLE resena        PURGE;
DROP TABLE linea_pedido  PURGE;
DROP TABLE pedido        PURGE;
DROP TABLE dlc           PURGE;
DROP TABLE desbloquea    PURGE;
DROP TABLE logro         PURGE;
DROP TABLE amistad       PURGE;
DROP TABLE juega         PURGE;
DROP TABLE biblioteca    PURGE;
DROP TABLE juego_genero  PURGE;
DROP TABLE juego_f2p     PURGE;
DROP TABLE juego_pago    PURGE;
DROP TABLE idioma_juego  PURGE;
DROP TABLE videojuego    PURGE;
DROP TABLE desarrollador PURGE;
DROP TABLE jugador       PURGE;
DROP TABLE plataforma    PURGE;
DROP TABLE genero        PURGE;
DROP TABLE estudio       PURGE;
DROP TABLE usuario       PURGE;

-- ----------------------------------------------------------------------
-- PARTE B · Conectado como SYSTEM (en la PDB): usuarios y roles
-- DROP USER ... CASCADE borraría también todos sus objetos, así que
-- esta parte sola bastaría para dejar la base de datos limpia.
-- ----------------------------------------------------------------------
-- DROP USER pv_analista;
-- DROP USER pv_app;
-- DROP USER pixelvault CASCADE;
-- DROP ROLE rol_pv_lectura;
-- DROP ROLE rol_pv_aplicacion;
```

11.3 Lista de comprobación

Después de ejecutar los scripts, comprueba con el diccionario de datos que todo está en su sitio:

- ◯ `SELECT COUNT(*) FROM user_tables` da **20**.
- ◯ `SELECT COUNT(*) FROM user_views` da **4**.
- ◯ Todas las restricciones tienen nombre propio salvo los `NOT NULL` (en `USER_CONSTRAINTS`, las únicas `SYS_C...` son de tipo `C` con condición `IS NOT NULL`).
- ◯ Las claves ajenas de LOGRO, DLC, JUGADOR, BIBLIOTECA, RESENA… tienen `DELETE_RULE = CASCADE`, y la de la secuela, `SET NULL`.
- ◯ No hay objetos inválidos (`status = 'INVALID'` en `USER_OBJECTS`).
- ◯ Como `PV_ANALISTA`, `SELECT * FROM pixelvault.v_catalogo` funciona y `SELECT * FROM pixelvault.usuario` da ORA-00942.
- ◯ Todas las sentencias de `07_pruebas_restricciones.sql` fallan con el error indicado.

11.4 Lo que queda para más adelante

El script implanta todo lo que el modelo relacional permite expresar con restricciones. Quedan fuera, a propósito, las reglas del catálogo que necesitan consultar otras filas o tablas (R-08 a R-15: la secuela posterior al original, dos horas de juego para reseñar, edad mínima según PEGI…). Se implementarán con **disparadores** y **procedimientos** cuando se estudie la programación de bases de datos.

RA 2 · APARTADO 12

El mismo esquema en SQL Server y MariaDB

SQL es un estándar, pero cada gestor tiene su «dialecto»: tipos de datos con otros nombres, formas distintas de generar autonuméricos o de gestionar usuarios. En la empresa te encontrarás con varios gestores, así que en este apartado comparamos Oracle con dos muy extendidos, **Microsoft SQL Server** y **MariaDB** (el sucesor libre de MySQL), y creamos en ellos la misma base de datos de PixelVault.

12.1 Comparativa CE 2h

Organización y seguridad

Concepto	Oracle	SQL Server	MariaDB
Qué se crea para empezar	Un **usuario** (su esquema es la BD del proyecto)	Una **base de datos** (`CREATE DATABASE`) con uno o varios esquemas	Una **base de datos** (`CREATE DATABASE`); base de datos y esquema son lo mismo
Cuentas	`CREATE USER`	Dos niveles: `CREATE LOGIN` (servidor) y `CREATE USER` (base de datos)	`CREATE USER 'nombre'@'equipo'`
Nombre completo de una tabla	`pixelvault.videojuego`	`PixelVault.pv.videojuego` (bd.esquema.tabla)	`pixelvault.videojuego`
Roles	`CREATE ROLE` + `GRANT rol TO usuario`	`CREATE ROLE` + `ALTER ROLE rol ADD MEMBER usuario`	`CREATE ROLE` + `GRANT rol TO usuario` + **`SET DEFAULT ROLE`** (si no, el rol no se activa al conectar)
Separador de lotes	`;` (y `/` para bloques PL/SQL)	`;` y **`GO`** (`CREATE VIEW` y `CREATE SCHEMA` deben ir solos en su lote)	`;` (`DELIMITER` para procedimientos)
Diccionario de datos	`USER_TABLES` , `USER_CONSTRAINTS` …	`INFORMATION_SCHEMA` y vistas `sys.*`	`INFORMATION_SCHEMA` y `SHOW TABLES` , `SHOW CREATE TABLE`
Herramienta gráfica	SQL Developer	SQL Server Management Studio (SSMS)	Cliente `mariadb` , HeidiSQL, DBeaver

Tipos de datos

Dato	Oracle	SQL Server	MariaDB
Texto variable	`VARCHAR2(n)`	`NVARCHAR(n)` (Unicode) / `VARCHAR(n)`	`VARCHAR(n)` (con `utf8mb4`)

Dato	Oracle	SQL Server	MariaDB
Texto largo	`CLOB`	`NVARCHAR(MAX)`	`TEXT`, `LONGTEXT`
Entero	`NUMBER(10)`	`INT`, `SMALLINT`, `TINYINT`, `BIGINT`	`INT`, `SMALLINT`, `TINYINT`, `BIGINT`
Decimal exacto	`NUMBER(6,2)`	`DECIMAL(6,2)`	`DECIMAL(6,2)`
Fecha	`DATE` (**con hora**)	`DATE` (solo fecha)	`DATE` (solo fecha)
Fecha y hora	`TIMESTAMP`	`DATETIME2`	`DATETIME`, `TIMESTAMP`
Booleano	`CHAR(1)` + `CHECK` (`BOOLEAN` en 23ai)	`BIT`	`BOOLEAN` (= `TINYINT(1)`)
¿`''` es `NULL`?	**Sí**	No	No

Claves y restricciones

Concepto	Oracle	SQL Server	MariaDB
Autonumérico	`GENERATED ... AS IDENTITY` o secuencia	`IDENTITY(1,1)` o secuencia	`AUTO_INCREMENT`
Dar un valor explícito al autonumérico	Con `BY DEFAULT`; luego resincronizar	`SET IDENTITY_INSERT tabla ON`	Directamente; el contador se ajusta solo
`ON DELETE`	`CASCADE`, `SET NULL`	`CASCADE`, `SET NULL`, `SET DEFAULT`, `NO ACTION`	`CASCADE`, `SET NULL`, `RESTRICT`, `NO ACTION`
`ON UPDATE`	**No existe**	Sí	Sí
Varias rutas de cascada hacia una tabla	Permitido	**No permitido** (error 1785)	Permitido
`UNIQUE` con varios `NULL`	Permitido	**Solo un** `NULL` (se usa un índice único filtrado)	Permitido
`CHECK`	Sí	Sí	Sí (desde 10.2), pero no sobre columnas de una clave ajena con `SET NULL`
Índice automático en claves ajenas	No	No	**Sí** (InnoDB)

Concepto	Oracle	SQL Server	MariaDB
Vistas `WITH READ ONLY`	Sí	No (se controla con permisos)	No (se controla con permisos)
`IF [NOT] EXISTS` en `CREATE` / `DROP`	Solo 23ai / 26ai	`DROP ... IF EXISTS` (2016+)	Sí

Errores equivalentes

Situación	Oracle	SQL Server	MariaDB
Clave primaria o única repetida	ORA-00001	2627 / 2601	1062
Valor nulo en columna obligatoria	ORA-01400	515	1048
`CHECK` incumplido	ORA-02290	547	4025
Clave ajena sin referencia	ORA-02291	547	1452
Borrar una fila referenciada	ORA-02292	547	1451
Valor demasiado grande para la columna	ORA-12899	2628	1406

12.2 La misma tabla en los tres gestores

Así queda VIDEOJUEGO, la tabla con más casuística (autonumérico, clave ajena reflexiva, `UNIQUE` que admite nulos y tres `CHECK`):

Oracle

```
CREATE TABLE videojuego (
    id_juego           NUMBER(10)    GENERATED BY DEFAULT AS IDENTITY,
    titulo             VARCHAR2(150) NOT NULL,
    fecha_lanzamiento  DATE,
    pegi               NUMBER(2)     NOT NULL,
    tipo               VARCHAR2(4)   NOT NULL,
    id_estudio         NUMBER(10)    NOT NULL,
    id_juego_original  NUMBER(10),
    CONSTRAINT pk_videojuego          PRIMARY KEY (id_juego),
    CONSTRAINT fk_videojuego_estudio  FOREIGN KEY (id_estudio)
        REFERENCES estudio (id_estudio),
    CONSTRAINT fk_videojuego_original FOREIGN KEY (id_juego_original)
        REFERENCES videojuego (id_juego) ON DELETE SET NULL,
    CONSTRAINT uk_videojuego_original UNIQUE (id_juego_original),
    CONSTRAINT ck_videojuego_pegi     CHECK (pegi IN (3, 7, 12, 16, 18)),
    CONSTRAINT ck_videojuego_tipo     CHECK (tipo IN ('PAGO', 'F2P')),
    CONSTRAINT ck_videojuego_secuela  CHECK (id_juego_original <> id_juego)
```

```
);
```

SQL Server

```
CREATE TABLE pv.videojuego (
    id_juego           INT           IDENTITY(1,1) NOT NULL,
    titulo             NVARCHAR(150) NOT NULL,
    fecha_lanzamiento  DATE          NULL,
    pegi               TINYINT       NOT NULL,
    tipo               VARCHAR(4)    NOT NULL,
    id_estudio         INT           NOT NULL,
    id_juego_original  INT           NULL,
    CONSTRAINT pk_videojuego          PRIMARY KEY (id_juego),
    CONSTRAINT fk_videojuego_estudio  FOREIGN KEY (id_estudio)
        REFERENCES pv.estudio (id_estudio),
    -- Reflexiva: SET NULL no está permitido (error 1785)
    CONSTRAINT fk_videojuego_original FOREIGN KEY (id_juego_original)
        REFERENCES pv.videojuego (id_juego) ON DELETE NO ACTION,
    CONSTRAINT ck_videojuego_pegi     CHECK (pegi IN (3, 7, 12, 16, 18)),
    CONSTRAINT ck_videojuego_tipo     CHECK (tipo IN ('PAGO', 'F2P')),
    CONSTRAINT ck_videojuego_secuela  CHECK (id_juego_original <> id_juego)
);
-- UNIQUE que admite varios NULL: índice único filtrado
CREATE UNIQUE INDEX uk_videojuego_original
    ON pv.videojuego (id_juego_original)
    WHERE id_juego_original IS NOT NULL;
```

MariaDB

```
CREATE TABLE videojuego (
    id_juego           INT          NOT NULL AUTO_INCREMENT,
    titulo             VARCHAR(150) NOT NULL,
    fecha_lanzamiento  DATE,
    pegi               TINYINT      NOT NULL,
    tipo               VARCHAR(4)   NOT NULL,
    id_estudio         INT          NOT NULL,
    id_juego_original  INT,
    CONSTRAINT pk_videojuego          PRIMARY KEY (id_juego),
    CONSTRAINT fk_videojuego_estudio  FOREIGN KEY (id_estudio)
        REFERENCES estudio (id_estudio),
    CONSTRAINT fk_videojuego_original FOREIGN KEY (id_juego_original)
        REFERENCES videojuego (id_juego) ON DELETE SET NULL,
    CONSTRAINT uk_videojuego_original UNIQUE (id_juego_original),
    CONSTRAINT ck_videojuego_pegi     CHECK (pegi IN (3, 7, 12, 16, 18)),
    CONSTRAINT ck_videojuego_tipo     CHECK (tipo IN ('PAGO', 'F2P'))
    -- R-07 no se puede declarar: CHECK sobre una FK con SET NULL (error 1901)
) ENGINE = InnoDB;
```

Las diferencias que obligan a cambiar el diseño

Casi todo es traducción de nombres, pero hay tres casos en los que el gestor **no admite** exactamente lo que decidimos en el diseño. Es importante detectarlos y documentarlos:

Gestor	Qué no admite	Solución en el script	Consecuencia
SQL Server	`ON DELETE SET NULL` en la clave ajena reflexiva de la secuela	`NO ACTION`	Para borrar un juego original hay que quitar antes la referencia de su secuela (o hacerlo con un disparador)
SQL Server	Dos rutas de borrado en cascada desde JUGADOR hacia AMISTAD	`NO ACTION` en `fk_amistad_amigo`	Para borrar un jugador que aparece como `id_amigo` hay que borrar antes esas amistades
MariaDB	`CHECK` sobre `id_juego_original`, que es clave ajena con `SET NULL`	Se omite R-07	La regla se controlaría con un disparador

12.3 Script completo para SQL Server

Preparado para SQL Server 2019 o posterior (también la edición gratuita Express). Se ejecuta completo en SQL Server Management Studio (botón **Ejecutar**) o con `sqlcmd`.

Descargar pixelvault_sqlserver.sql

Ver el script completo de SQL Server

pixelvault_sqlserver.sql

```
-- ========================================================================
-- PixelVault · pixelvault_sqlserver.sql
-- Script completo para Microsoft SQL Server 2019 o posterior
-- (válido también en la edición gratuita Express).
-- Ejecutar como administrador (sa o un inicio de sesión sysadmin)
-- desde SQL Server Management Studio o con sqlcmd:
--     sqlcmd -S localhost -U sa -i pixelvault_sqlserver.sql
--
-- Diferencias principales con Oracle:
--   · Un servidor tiene varias bases de datos (CREATE DATABASE) y cada
--     una varios esquemas; aquí las tablas van en el esquema pv.
--   · Seguridad en dos niveles: LOGIN (servidor) y USER (base de datos).
--   · Autonumérico con IDENTITY(inicio, incremento).
--   · Tipos: INT, NVARCHAR, DECIMAL, DATE; cadenas Unicode con N'...'.
--   · GO separa lotes: CREATE VIEW y CREATE SCHEMA deben ir solos.
--   · No admite varias rutas de borrado en cascada hacia una misma tabla
--     ni cascadas en claves ajenas reflexivas (error 1785): en AMISTAD
--     y en la secuela se usa NO ACTION.
--   · No hay vistas WITH READ ONLY: se controla con los permisos.
-- ========================================================================

-- Opciones de sesión necesarias para índices filtrados (sqlcmd las
-- desactiva por defecto; en SSMS ya vienen activadas)
SET ANSI_NULLS ON;
SET QUOTED_IDENTIFIER ON;
GO

USE master;
GO
IF DB_ID(N'PixelVault') IS NOT NULL
BEGIN
    ALTER DATABASE PixelVault SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE PixelVault;
END;
GO
CREATE DATABASE PixelVault;
GO
USE PixelVault;
GO

-- ------------------------------------------------------------------------
-- Inicio de sesión y usuario propietario del esquema pv
-- ------------------------------------------------------------------------
IF SUSER_ID(N'pixelvault') IS NULL
    CREATE LOGIN pixelvault WITH PASSWORD = N'PixelVault_2026';
GO
CREATE USER pixelvault FOR LOGIN pixelvault;
GO
CREATE SCHEMA pv AUTHORIZATION pixelvault;
GO
ALTER USER pixelvault WITH DEFAULT_SCHEMA = pv;
GRANT CREATE TABLE, CREATE VIEW TO pixelvault;
GO

-- ------------------------------------------------------------------------
```

```sql
-- Tablas sin claves ajenas
-- ------------------------------------------------------------------------
CREATE TABLE pv.usuario (
    id_usuario      INT           IDENTITY(1,1) NOT NULL,
    email           NVARCHAR(120) NOT NULL,
    nombre          NVARCHAR(50)  NOT NULL,
    apellidos       NVARCHAR(80)  NULL,
    fecha_registro  DATE          NOT NULL
        CONSTRAINT df_usuario_fecha DEFAULT (CAST(GETDATE() AS DATE)),
    CONSTRAINT pk_usuario       PRIMARY KEY (id_usuario),
    CONSTRAINT uk_usuario_email UNIQUE (email),
    CONSTRAINT ck_usuario_email CHECK (email LIKE N'%_@_%.__%')
);

CREATE TABLE pv.estudio (
    id_estudio  INT           IDENTITY(1,1) NOT NULL,
    nombre      NVARCHAR(100) NOT NULL,
    pais        NVARCHAR(60)  NULL,
    web         NVARCHAR(200) NULL,
    CONSTRAINT pk_estudio PRIMARY KEY (id_estudio)
);

CREATE TABLE pv.genero (
    id_genero  INT          IDENTITY(1,1) NOT NULL,
    nombre     NVARCHAR(40) NOT NULL,
    CONSTRAINT pk_genero        PRIMARY KEY (id_genero),
    CONSTRAINT uk_genero_nombre UNIQUE (nombre)
);

CREATE TABLE pv.plataforma (
    id_plataforma  INT          IDENTITY(1,1) NOT NULL,
    nombre         NVARCHAR(60) NOT NULL,
    fabricante     NVARCHAR(60) NULL,
    CONSTRAINT pk_plataforma        PRIMARY KEY (id_plataforma),
    CONSTRAINT uk_plataforma_nombre UNIQUE (nombre)
);

-- ------------------------------------------------------------------------
-- Jerarquía USUARIO → JUGADOR, DESARROLLADOR
-- ------------------------------------------------------------------------
CREATE TABLE pv.jugador (
    id_usuario        INT           NOT NULL,
    nick              NVARCHAR(30)  NOT NULL,
    fecha_nacimiento  DATE          NOT NULL,
    calle             NVARCHAR(100) NULL,
    numero            NVARCHAR(10)  NULL,
    cp                NVARCHAR(10)  NULL,
    ciudad            NVARCHAR(60)  NULL,
    pais              NVARCHAR(60)  NULL,
    CONSTRAINT pk_jugador         PRIMARY KEY (id_usuario),
    CONSTRAINT fk_jugador_usuario FOREIGN KEY (id_usuario)
        REFERENCES pv.usuario (id_usuario) ON DELETE CASCADE,
    CONSTRAINT uk_jugador_nick    UNIQUE (nick)
);

CREATE TABLE pv.desarrollador (
    id_usuario  INT          NOT NULL,
    cargo       NVARCHAR(60) NULL,
    id_estudio  INT          NOT NULL,
    CONSTRAINT pk_desarrollador         PRIMARY KEY (id_usuario),
```

```
    CONSTRAINT fk_desarrollador_usuario FOREIGN KEY (id_usuario)
        REFERENCES pv.usuario (id_usuario) ON DELETE CASCADE,
    CONSTRAINT fk_desarrollador_estudio FOREIGN KEY (id_estudio)
        REFERENCES pv.estudio (id_estudio)
);

-- ---------------------------------------------------------------------
-- VIDEOJUEGO y subtipos
-- ---------------------------------------------------------------------
CREATE TABLE pv.videojuego (
    id_juego           INT           IDENTITY(1,1) NOT NULL,
    titulo             NVARCHAR(150) NOT NULL,
    fecha_lanzamiento  DATE          NULL,
    pegi               TINYINT       NOT NULL,
    tipo               VARCHAR(4)    NOT NULL,
    id_estudio         INT           NOT NULL,
    id_juego_original  INT           NULL,
    CONSTRAINT pk_videojuego          PRIMARY KEY (id_juego),
    CONSTRAINT fk_videojuego_estudio  FOREIGN KEY (id_estudio)
        REFERENCES pv.estudio (id_estudio),
    -- Reflexiva: SQL Server no permite SET NULL aquí (error 1785);
    -- borrar un original con secuelas queda restringido.
    CONSTRAINT fk_videojuego_original FOREIGN KEY (id_juego_original)
        REFERENCES pv.videojuego (id_juego) ON DELETE NO ACTION,
    CONSTRAINT ck_videojuego_pegi     CHECK (pegi IN (3, 7, 12, 16, 18)),
    CONSTRAINT ck_videojuego_tipo     CHECK (tipo IN ('PAGO', 'F2P')),
    CONSTRAINT ck_videojuego_secuela  CHECK (id_juego_original <> id_juego)
);
-- UNIQUE que admite varios NULL: en SQL Server un UNIQUE normal solo
-- deja un NULL, así que se usa un índice único filtrado.
CREATE UNIQUE INDEX uk_videojuego_original
    ON pv.videojuego (id_juego_original)
    WHERE id_juego_original IS NOT NULL;

CREATE TABLE pv.idioma_juego (
    id_juego  INT          NOT NULL,
    idioma    NVARCHAR(40) NOT NULL,
    CONSTRAINT pk_idioma_juego PRIMARY KEY (id_juego, idioma),
    CONSTRAINT fk_idioma_juego FOREIGN KEY (id_juego)
        REFERENCES pv.videojuego (id_juego) ON DELETE CASCADE
);

CREATE TABLE pv.juego_pago (
    id_juego  INT          NOT NULL,
    precio    DECIMAL(6,2) NOT NULL,
    CONSTRAINT pk_juego_pago        PRIMARY KEY (id_juego),
    CONSTRAINT fk_juego_pago_juego  FOREIGN KEY (id_juego)
        REFERENCES pv.videojuego (id_juego) ON DELETE CASCADE,
    CONSTRAINT ck_juego_pago_precio CHECK (precio >= 0)
);

CREATE TABLE pv.juego_f2p (
    id_juego          INT     NOT NULL,
    tiene_micropagos  CHAR(1) NOT NULL
        CONSTRAINT df_juego_f2p_micropagos DEFAULT ('N'),
    CONSTRAINT pk_juego_f2p            PRIMARY KEY (id_juego),
    CONSTRAINT fk_juego_f2p_juego      FOREIGN KEY (id_juego)
        REFERENCES pv.videojuego (id_juego) ON DELETE CASCADE,
    CONSTRAINT ck_juego_f2p_micropagos CHECK (tiene_micropagos IN ('S', 'N'))
);
```

```sql
CREATE TABLE pv.juego_genero (
    id_juego   INT NOT NULL,
    id_genero  INT NOT NULL,
    CONSTRAINT pk_juego_genero        PRIMARY KEY (id_juego, id_genero),
    CONSTRAINT fk_juego_genero_juego  FOREIGN KEY (id_juego)
        REFERENCES pv.videojuego (id_juego) ON DELETE CASCADE,
    CONSTRAINT fk_juego_genero_genero FOREIGN KEY (id_genero)
        REFERENCES pv.genero (id_genero)
);

-- ---------------------------------------------------------------------
-- Relaciones del jugador con los juegos
-- ---------------------------------------------------------------------
CREATE TABLE pv.biblioteca (
    id_jugador         INT  NOT NULL,
    id_juego           INT  NOT NULL,
    fecha_adquisicion  DATE NOT NULL
        CONSTRAINT df_biblioteca_fecha DEFAULT (CAST(GETDATE() AS DATE)),
    CONSTRAINT pk_biblioteca         PRIMARY KEY (id_jugador, id_juego),
    CONSTRAINT fk_biblioteca_jugador FOREIGN KEY (id_jugador)
        REFERENCES pv.jugador (id_usuario) ON DELETE CASCADE,
    CONSTRAINT fk_biblioteca_juego   FOREIGN KEY (id_juego)
        REFERENCES pv.videojuego (id_juego)
);

CREATE TABLE pv.juega (
    id_jugador     INT          NOT NULL,
    id_juego       INT          NOT NULL,
    id_plataforma  INT          NOT NULL,
    horas_jugadas  DECIMAL(7,1) NOT NULL
        CONSTRAINT df_juega_horas DEFAULT (0),
    CONSTRAINT pk_juega            PRIMARY KEY (id_jugador, id_juego, id_plataforma),
    CONSTRAINT fk_juega_jugador    FOREIGN KEY (id_jugador)
        REFERENCES pv.jugador (id_usuario) ON DELETE CASCADE,
    CONSTRAINT fk_juega_juego      FOREIGN KEY (id_juego)
        REFERENCES pv.videojuego (id_juego),
    CONSTRAINT fk_juega_plataforma FOREIGN KEY (id_plataforma)
        REFERENCES pv.plataforma (id_plataforma),
    CONSTRAINT ck_juega_horas      CHECK (horas_jugadas >= 0)
);

CREATE TABLE pv.amistad (
    id_jugador  INT NOT NULL,
    id_amigo    INT NOT NULL,
    CONSTRAINT pk_amistad         PRIMARY KEY (id_jugador, id_amigo),
    CONSTRAINT fk_amistad_jugador FOREIGN KEY (id_jugador)
        REFERENCES pv.jugador (id_usuario) ON DELETE CASCADE,
    -- Segunda ruta de cascada desde JUGADOR: no permitida (error 1785)
    CONSTRAINT fk_amistad_amigo   FOREIGN KEY (id_amigo)
        REFERENCES pv.jugador (id_usuario) ON DELETE NO ACTION,
    CONSTRAINT ck_amistad_orden   CHECK (id_jugador < id_amigo)
);

-- ---------------------------------------------------------------------
-- Entidades débiles
-- ---------------------------------------------------------------------
CREATE TABLE pv.logro (
    id_juego     INT          NOT NULL,
    num_logro    SMALLINT     NOT NULL,
```

```
    nombre       NVARCHAR(100) NOT NULL,
    descripcion  NVARCHAR(400) NULL,
    puntos       SMALLINT      NOT NULL,
    CONSTRAINT pk_logro        PRIMARY KEY (id_juego, num_logro),
    CONSTRAINT fk_logro_juego  FOREIGN KEY (id_juego)
        REFERENCES pv.videojuego (id_juego) ON DELETE CASCADE,
    CONSTRAINT ck_logro_puntos CHECK (puntos > 0)
);

CREATE TABLE pv.desbloquea (
    id_jugador  INT      NOT NULL,
    id_juego    INT      NOT NULL,
    num_logro   SMALLINT NOT NULL,
    CONSTRAINT pk_desbloquea         PRIMARY KEY (id_jugador, id_juego, num_logro),
    CONSTRAINT fk_desbloquea_jugador FOREIGN KEY (id_jugador)
        REFERENCES pv.jugador (id_usuario) ON DELETE CASCADE,
    CONSTRAINT fk_desbloquea_logro   FOREIGN KEY (id_juego, num_logro)
        REFERENCES pv.logro (id_juego, num_logro) ON DELETE CASCADE
);

CREATE TABLE pv.dlc (
    id_dlc             INT           IDENTITY(1,1) NOT NULL,
    titulo             NVARCHAR(150) NOT NULL,
    precio             DECIMAL(6,2)  NOT NULL,
    fecha_lanzamiento  DATE          NULL,
    id_juego           INT           NOT NULL,
    CONSTRAINT pk_dlc        PRIMARY KEY (id_dlc),
    CONSTRAINT fk_dlc_juego  FOREIGN KEY (id_juego)
        REFERENCES pv.videojuego (id_juego) ON DELETE CASCADE,
    CONSTRAINT ck_dlc_precio CHECK (precio >= 0)
);

-- ---------------------------------------------------------------------
-- Pedidos y reseñas
-- ---------------------------------------------------------------------
CREATE TABLE pv.pedido (
    num_pedido  INT  IDENTITY(1,1) NOT NULL,
    fecha       DATE NOT NULL
        CONSTRAINT df_pedido_fecha DEFAULT (CAST(GETDATE() AS DATE)),
    id_jugador  INT  NOT NULL,
    CONSTRAINT pk_pedido         PRIMARY KEY (num_pedido),
    CONSTRAINT fk_pedido_jugador FOREIGN KEY (id_jugador)
        REFERENCES pv.jugador (id_usuario)
);

CREATE TABLE pv.linea_pedido (
    num_pedido     INT          NOT NULL,
    id_juego       INT          NOT NULL,
    precio_pagado  DECIMAL(6,2) NOT NULL,
    CONSTRAINT pk_linea_pedido        PRIMARY KEY (num_pedido, id_juego),
    CONSTRAINT fk_linea_pedido_pedido FOREIGN KEY (num_pedido)
        REFERENCES pv.pedido (num_pedido) ON DELETE CASCADE,
    CONSTRAINT fk_linea_pedido_juego  FOREIGN KEY (id_juego)
        REFERENCES pv.juego_pago (id_juego),
    CONSTRAINT ck_linea_pedido_precio CHECK (precio_pagado >= 0)
);

CREATE TABLE pv.resena (
    id_resena    INT           IDENTITY(1,1) NOT NULL,
    nota         TINYINT       NOT NULL,
```

```
    texto        NVARCHAR(2000) NULL,
    fecha        DATE           NOT NULL
        CONSTRAINT df_resena_fecha DEFAULT (CAST(GETDATE() AS DATE)),
    recomendado  CHAR(1)        NOT NULL,
    id_jugador   INT            NOT NULL,
    id_juego     INT            NOT NULL,
    CONSTRAINT pk_resena             PRIMARY KEY (id_resena),
    CONSTRAINT fk_resena_biblioteca  FOREIGN KEY (id_jugador, id_juego)
        REFERENCES pv.biblioteca (id_jugador, id_juego) ON DELETE CASCADE,
    CONSTRAINT uk_resena_posesion    UNIQUE (id_jugador, id_juego),
    CONSTRAINT ck_resena_nota        CHECK (nota BETWEEN 1 AND 10),
    CONSTRAINT ck_resena_recomendado CHECK (recomendado IN ('S', 'N'))
);
GO

-- ------------------------------------------------------------------------
-- Índices sobre claves ajenas (SQL Server tampoco los crea solo)
-- ------------------------------------------------------------------------
CREATE INDEX ix_desarrollador_estudio ON pv.desarrollador (id_estudio);
CREATE INDEX ix_videojuego_estudio    ON pv.videojuego (id_estudio);
CREATE INDEX ix_juego_genero_genero   ON pv.juego_genero (id_genero);
CREATE INDEX ix_biblioteca_juego      ON pv.biblioteca (id_juego);
CREATE INDEX ix_juega_juego           ON pv.juega (id_juego);
CREATE INDEX ix_juega_plataforma      ON pv.juega (id_plataforma);
CREATE INDEX ix_amistad_amigo         ON pv.amistad (id_amigo);
CREATE INDEX ix_dlc_juego             ON pv.dlc (id_juego);
CREATE INDEX ix_pedido_jugador        ON pv.pedido (id_jugador);
CREATE INDEX ix_linea_pedido_juego    ON pv.linea_pedido (id_juego);
CREATE INDEX ix_desbloquea_logro      ON pv.desbloquea (id_juego, num_logro);
CREATE INDEX ix_videojuego_pegi       ON pv.videojuego (pegi);
GO

-- ------------------------------------------------------------------------
-- Vistas (cada CREATE VIEW en su propio lote)
-- ------------------------------------------------------------------------
CREATE VIEW pv.v_jugador_edad AS
SELECT j.id_usuario,
       j.nick,
       u.email,
       j.fecha_nacimiento,
       DATEDIFF(YEAR, j.fecha_nacimiento, GETDATE())
         - CASE WHEN DATEADD(YEAR, DATEDIFF(YEAR, j.fecha_nacimiento, GETDATE()),
                             j.fecha_nacimiento) > CAST(GETDATE() AS DATE)
                THEN 1 ELSE 0 END AS edad
  FROM pv.jugador j
  JOIN pv.usuario u ON u.id_usuario = j.id_usuario;
GO

CREATE VIEW pv.v_nota_media AS
SELECT v.id_juego,
       v.titulo,
       ROUND(AVG(CAST(r.nota AS DECIMAL(4,1))), 1) AS nota_media,
       COUNT(r.id_resena)                          AS num_resenas
  FROM pv.videojuego v
  LEFT JOIN pv.resena r ON r.id_juego = v.id_juego
 GROUP BY v.id_juego, v.titulo;
GO

CREATE VIEW pv.v_catalogo AS
SELECT v.id_juego,
```

```
       v.titulo,
       e.nombre AS estudio,
       v.fecha_lanzamiento,
       v.pegi,
       v.tipo,
       p.precio
  FROM pv.videojuego v
  JOIN pv.estudio e         ON e.id_estudio = v.id_estudio
  LEFT JOIN pv.juego_pago p ON p.id_juego   = v.id_juego;
GO

CREATE VIEW pv.v_juegos_pegi18 AS
SELECT id_juego, titulo, fecha_lanzamiento, pegi, tipo, id_estudio
  FROM pv.videojuego
 WHERE pegi = 18
  WITH CHECK OPTION;
GO

-- ------------------------------------------------------------------------
-- Roles, usuarios y privilegios
-- ------------------------------------------------------------------------
CREATE ROLE rol_pv_aplicacion;
CREATE ROLE rol_pv_lectura;

GRANT SELECT ON pv.estudio      TO rol_pv_aplicacion;
GRANT SELECT ON pv.videojuego   TO rol_pv_aplicacion;
GRANT SELECT ON pv.idioma_juego TO rol_pv_aplicacion;
GRANT SELECT ON pv.juego_pago   TO rol_pv_aplicacion;
GRANT SELECT ON pv.juego_f2p    TO rol_pv_aplicacion;
GRANT SELECT ON pv.genero       TO rol_pv_aplicacion;
GRANT SELECT ON pv.juego_genero TO rol_pv_aplicacion;
GRANT SELECT ON pv.plataforma   TO rol_pv_aplicacion;
GRANT SELECT ON pv.logro        TO rol_pv_aplicacion;
GRANT SELECT ON pv.dlc          TO rol_pv_aplicacion;
GRANT SELECT, INSERT, UPDATE, DELETE ON pv.usuario    TO rol_pv_aplicacion;
GRANT SELECT, INSERT, UPDATE, DELETE ON pv.jugador    TO rol_pv_aplicacion;
GRANT SELECT, INSERT, UPDATE, DELETE ON pv.biblioteca TO rol_pv_aplicacion;
GRANT SELECT, INSERT, UPDATE, DELETE ON pv.juega      TO rol_pv_aplicacion;
GRANT SELECT, INSERT, DELETE         ON pv.amistad    TO rol_pv_aplicacion;
GRANT SELECT, INSERT                 ON pv.desbloquea TO rol_pv_aplicacion;
GRANT SELECT, INSERT, UPDATE, DELETE ON pv.resena     TO rol_pv_aplicacion;
GRANT SELECT, INSERT ON pv.pedido       TO rol_pv_aplicacion;
GRANT SELECT, INSERT ON pv.linea_pedido TO rol_pv_aplicacion;
GRANT SELECT ON pv.v_catalogo     TO rol_pv_aplicacion;
GRANT SELECT ON pv.v_nota_media   TO rol_pv_aplicacion;
GRANT SELECT ON pv.v_jugador_edad TO rol_pv_aplicacion;

GRANT SELECT ON pv.v_catalogo   TO rol_pv_lectura;
GRANT SELECT ON pv.v_nota_media TO rol_pv_lectura;
GO

IF SUSER_ID(N'pv_app') IS NULL
    CREATE LOGIN pv_app WITH PASSWORD = N'App_PixelVault_2026';
IF SUSER_ID(N'pv_analista') IS NULL
    CREATE LOGIN pv_analista WITH PASSWORD = N'Analista_PV_2026';
GO
CREATE USER pv_app      FOR LOGIN pv_app      WITH DEFAULT_SCHEMA = pv;
CREATE USER pv_analista FOR LOGIN pv_analista WITH DEFAULT_SCHEMA = pv;
ALTER ROLE rol_pv_aplicacion ADD MEMBER pv_app;
ALTER ROLE rol_pv_lectura    ADD MEMBER pv_analista;
```

```
GO

-- ---------------------------------------------------------------------
-- Datos de prueba (mismos que en Oracle).
-- Para dar valor a una columna IDENTITY hay que activar
-- IDENTITY_INSERT, tabla a tabla.
-- ---------------------------------------------------------------------
SET IDENTITY_INSERT pv.estudio ON;
INSERT INTO pv.estudio (id_estudio, nombre, pais) VALUES
    (10, N'Rockstar North', N'Reino Unido'), (11, N'Larian Studios', N'Bélgica'),
    (12, N'Nintendo EPD', N'Japón'),         (13, N'FromSoftware', N'Japón'),
    (14, N'CD Projekt Red', N'Polonia'),     (15, N'Supergiant Games', N'EE. UU.'),
    (16, N'Team Cherry', N'Australia'),      (17, N'ConcernedApe', N'EE. UU.'),
    (18, N'Epic Games', N'EE. UU.'),         (19, N'Valve', N'EE. UU.');
SET IDENTITY_INSERT pv.estudio OFF;

SET IDENTITY_INSERT pv.videojuego ON;
INSERT INTO pv.videojuego (id_juego, titulo, fecha_lanzamiento, pegi, tipo, id_estudio,
id_juego_original) VALUES
    (1,  N'Grand Theft Auto V', '2013-09-17', 18, 'PAGO', 10, NULL),
    (2,  N'Baldur''s Gate 3', '2023-08-03', 18, 'PAGO', 11, NULL),
    (3,  N'The Legend of Zelda: Tears of the Kingdom', '2023-05-12', 12, 'PAGO', 12, NULL),
    (4,  N'Elden Ring', '2022-02-25', 16, 'PAGO', 13, NULL),
    (5,  N'Hollow Knight', '2017-02-24', 7, 'PAGO', 16, NULL),
    (6,  N'Hades', '2020-09-17', 12, 'PAGO', 15, NULL),
    (7,  N'The Witcher 3: Wild Hunt', '2015-05-19', 18, 'PAGO', 14, NULL),
    (8,  N'Stardew Valley', '2016-02-26', 7, 'PAGO', 17, NULL),
    (9,  N'Fortnite', '2017-07-25', 12, 'F2P', 18, NULL),
    (10, N'Counter-Strike 2', '2023-09-27', 18, 'F2P', 19, NULL),
    (11, N'Hades II', '2025-09-25', 12, 'PAGO', 15, 6);
SET IDENTITY_INSERT pv.videojuego OFF;

INSERT INTO pv.juego_pago (id_juego, precio) VALUES
    (1, 29.99), (2, 59.99), (3, 69.99), (4, 59.99), (5, 14.79),
    (6, 24.50), (7, 29.99), (8, 13.99), (11, 29.50);
INSERT INTO pv.juego_f2p (id_juego, tiene_micropagos) VALUES (9, 'S'), (10, 'S');

INSERT INTO pv.idioma_juego (id_juego, idioma) VALUES
    (2, N'español'), (2, N'inglés'), (2, N'francés'), (6, N'español'), (6, N'inglés');

SET IDENTITY_INSERT pv.genero ON;
INSERT INTO pv.genero (id_genero, nombre) VALUES
    (1, N'Acción'), (2, N'Rol'), (3, N'Aventura'), (4, N'Roguelike'),
    (5, N'Metroidvania'), (6, N'Simulación'), (7, N'Shooter');
SET IDENTITY_INSERT pv.genero OFF;

INSERT INTO pv.juego_genero (id_juego, id_genero) VALUES
    (1, 1), (2, 2), (3, 3), (4, 1), (4, 2), (5, 5), (6, 4),
    (7, 2), (8, 6), (9, 7), (10, 7), (11, 4);

SET IDENTITY_INSERT pv.plataforma ON;
INSERT INTO pv.plataforma (id_plataforma, nombre, fabricante) VALUES
    (1, N'PC', NULL), (2, N'PlayStation 5', N'Sony'),
    (3, N'Xbox Series X|S', N'Microsoft'), (4, N'Nintendo Switch 2', N'Nintendo');
SET IDENTITY_INSERT pv.plataforma OFF;

INSERT INTO pv.logro (id_juego, num_logro, nombre, puntos) VALUES
    (4, 1, N'Primera victoria', 10), (4, 2, N'Explorador de las Tierras Intermedias', 30),
    (6, 1, N'Primera victoria', 10), (6, 2, N'Escapar del Inframundo', 50);
```

```sql
SET IDENTITY_INSERT pv.dlc ON;
INSERT INTO pv.dlc (id_dlc, titulo, precio, fecha_lanzamiento, id_juego) VALUES
    (1, N'Shadow of the Erdtree', 39.99, '2024-06-21', 4),
    (2, N'Hearts of Stone', 9.99, '2015-10-13', 7),
    (3, N'Blood and Wine', 19.99, '2016-05-31', 7);
SET IDENTITY_INSERT pv.dlc OFF;

SET IDENTITY_INSERT pv.usuario ON;
INSERT INTO pv.usuario (id_usuario, email, nombre, apellidos) VALUES
    (101, N'noctis@ejemplo.es', N'Noa', N'Sáenz Ruiz'),
    (102, N'tarasca@ejemplo.es', N'Lucía', N'Garrido Pérez'),
    (103, N'kirbyto@ejemplo.es', N'Hugo', N'Martínez Sanz'),
    (104, N'ezio@ejemplo.es', N'Iker', N'López Ochoa'),
    (201, N'marta.dev@ejemplo.es', N'Marta', N'Ibáñez Gil'),
    (301, N'soporte@ejemplo.es', N'Soporte', N'PixelVault');
SET IDENTITY_INSERT pv.usuario OFF;

INSERT INTO pv.jugador (id_usuario, nick, fecha_nacimiento, ciudad, pais) VALUES
    (101, N'NoctisRJ',  '2006-03-14', N'Logroño',   N'España'),
    (102, N'LaTarasca', '1999-11-02', N'Haro',      N'España'),
    (103, N'Kirbyto',   '2012-06-30', N'Calahorra', N'España'),
    (104, N'Ezio_Lgr',  '2001-01-21', N'Logroño',   N'España');

INSERT INTO pv.desarrollador (id_usuario, cargo, id_estudio) VALUES
    (104, N'QA tester', 15), (201, N'Diseñadora de niveles', 11);

INSERT INTO pv.biblioteca (id_jugador, id_juego, fecha_adquisicion) VALUES
    (101, 6, '2026-09-01'), (101, 5, '2026-09-01'), (101, 4, '2026-09-02'),
    (101, 9, '2025-12-24'), (102, 6, '2026-09-02'), (102, 8, '2026-09-02'),
    (103, 5, '2026-03-10'), (103, 9, '2026-01-05'), (104, 2, '2026-09-10'),
    (104, 7, '2026-09-10'), (104, 10, '2025-06-01');

INSERT INTO pv.juega (id_jugador, id_juego, id_plataforma, horas_jugadas) VALUES
    (101, 6, 1, 40), (101, 6, 4, 12), (101, 4, 1, 1.5), (101, 4, 2, 0.4),
    (102, 6, 1, 25), (102, 8, 4, 80), (103, 5, 4, 10), (103, 9, 4, 30),
    (104, 2, 1, 120), (104, 10, 1, 300);

INSERT INTO pv.amistad (id_jugador, id_amigo) VALUES (101, 102), (101, 104), (102, 103);

INSERT INTO pv.desbloquea (id_jugador, id_juego, num_logro) VALUES
    (101, 6, 1), (101, 6, 2), (101, 4, 1), (102, 6, 1);

SET IDENTITY_INSERT pv.pedido ON;
INSERT INTO pv.pedido (num_pedido, fecha, id_jugador) VALUES
    (5001, '2026-09-01', 101), (5002, '2026-09-02', 102),
    (5003, '2026-09-02', 101), (5004, '2026-09-10', 104);
SET IDENTITY_INSERT pv.pedido OFF;

INSERT INTO pv.linea_pedido (num_pedido, id_juego, precio_pagado) VALUES
    (5001, 6, 24.50), (5001, 5, 14.79), (5002, 6, 19.99), (5002, 8, 13.99),
    (5003, 4, 59.99), (5004, 2, 49.99), (5004, 7, 9.99);

SET IDENTITY_INSERT pv.resena ON;
INSERT INTO pv.resena (id_resena, nota, texto, recomendado, id_jugador, id_juego) VALUES
    (1, 10, N'Cada partida es distinta. Imprescindible.', 'S', 101, 6),
    (2, 9, N'Relajante y adictivo.', 'S', 102, 8),
    (3, 8, N'Enorme, aunque el tercer acto se hace largo.', 'S', 104, 2);
SET IDENTITY_INSERT pv.resena OFF;
GO
```

12.4 Script completo para MariaDB

Para MariaDB 10.6 o posterior. Se ejecuta con el cliente de línea de comandos (`mariadb -u root -p < pixelvault_mariadb.sql`) o desde HeidiSQL/DBeaver.

Descargar pixelvault_mariadb.sql

Ver el script completo de MariaDB

pixelvault_mariadb.sql

```
-- ========================================================================
-- PixelVault · pixelvault_mariadb.sql
-- Script completo para MariaDB 10.6 o posterior (motor InnoDB).
-- Ejecutar con un usuario administrador (root), por ejemplo:
--     mariadb -u root -p < pixelvault_mariadb.sql
--
-- Equivale a los scripts 01 a 06 de Oracle. Diferencias principales:
--   · La base de datos es un objeto propio (CREATE DATABASE); el
--     usuario propietario se crea aparte y recibe permisos sobre ella.
--   · Autonumérico con AUTO_INCREMENT (se resincroniza solo).
--   · Tipos: INT, VARCHAR, DECIMAL, DATE; CHECK se comprueba desde 10.2.
--   · No hay vistas WITH READ ONLY: se controla con los permisos.
--   · Usuarios con la forma 'nombre'@'equipo'.
-- ========================================================================

DROP DATABASE IF EXISTS pixelvault;
CREATE DATABASE pixelvault
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
USE pixelvault;

-- ------------------------------------------------------------------------
-- Tablas sin claves ajenas
-- ------------------------------------------------------------------------
CREATE TABLE usuario (
    id_usuario      INT          NOT NULL AUTO_INCREMENT,
    email           VARCHAR(120) NOT NULL,
    nombre          VARCHAR(50)  NOT NULL,
    apellidos       VARCHAR(80),
    fecha_registro  DATE         NOT NULL DEFAULT (CURRENT_DATE),
    CONSTRAINT pk_usuario       PRIMARY KEY (id_usuario),
    CONSTRAINT uk_usuario_email UNIQUE (email),
    CONSTRAINT ck_usuario_email CHECK (email LIKE '%_@_%.__%')
) ENGINE = InnoDB;

CREATE TABLE estudio (
    id_estudio  INT          NOT NULL AUTO_INCREMENT,
    nombre      VARCHAR(100) NOT NULL,
    pais        VARCHAR(60),
    web         VARCHAR(200),
    CONSTRAINT pk_estudio PRIMARY KEY (id_estudio)
) ENGINE = InnoDB;

CREATE TABLE genero (
    id_genero  INT         NOT NULL AUTO_INCREMENT,
    nombre     VARCHAR(40) NOT NULL,
    CONSTRAINT pk_genero        PRIMARY KEY (id_genero),
    CONSTRAINT uk_genero_nombre UNIQUE (nombre)
) ENGINE = InnoDB;

CREATE TABLE plataforma (
    id_plataforma  INT         NOT NULL AUTO_INCREMENT,
    nombre         VARCHAR(60) NOT NULL,
    fabricante     VARCHAR(60),
    CONSTRAINT pk_plataforma        PRIMARY KEY (id_plataforma),
```

```
    CONSTRAINT uk_plataforma_nombre UNIQUE (nombre)
) ENGINE = InnoDB;

-- ---------------------------------------------------------------------
-- Jerarquía USUARIO → JUGADOR, DESARROLLADOR
-- ---------------------------------------------------------------------
CREATE TABLE jugador (
    id_usuario        INT          NOT NULL,
    nick              VARCHAR(30)  NOT NULL,
    fecha_nacimiento  DATE         NOT NULL,
    calle             VARCHAR(100),
    numero            VARCHAR(10),
    cp                VARCHAR(10),
    ciudad            VARCHAR(60),
    pais              VARCHAR(60),
    CONSTRAINT pk_jugador         PRIMARY KEY (id_usuario),
    CONSTRAINT fk_jugador_usuario FOREIGN KEY (id_usuario)
        REFERENCES usuario (id_usuario) ON DELETE CASCADE,
    CONSTRAINT uk_jugador_nick    UNIQUE (nick)
) ENGINE = InnoDB;

CREATE TABLE desarrollador (
    id_usuario  INT         NOT NULL,
    cargo       VARCHAR(60),
    id_estudio  INT         NOT NULL,
    CONSTRAINT pk_desarrollador         PRIMARY KEY (id_usuario),
    CONSTRAINT fk_desarrollador_usuario FOREIGN KEY (id_usuario)
        REFERENCES usuario (id_usuario) ON DELETE CASCADE,
    CONSTRAINT fk_desarrollador_estudio FOREIGN KEY (id_estudio)
        REFERENCES estudio (id_estudio)
) ENGINE = InnoDB;

-- ---------------------------------------------------------------------
-- VIDEOJUEGO y subtipos
-- ---------------------------------------------------------------------
CREATE TABLE videojuego (
    id_juego           INT          NOT NULL AUTO_INCREMENT,
    titulo             VARCHAR(150) NOT NULL,
    fecha_lanzamiento  DATE,
    pegi               TINYINT      NOT NULL,
    tipo               VARCHAR(4)   NOT NULL,
    id_estudio         INT          NOT NULL,
    id_juego_original  INT,
    CONSTRAINT pk_videojuego          PRIMARY KEY (id_juego),
    CONSTRAINT fk_videojuego_estudio  FOREIGN KEY (id_estudio)
        REFERENCES estudio (id_estudio),
    CONSTRAINT fk_videojuego_original FOREIGN KEY (id_juego_original)
        REFERENCES videojuego (id_juego) ON DELETE SET NULL,
    CONSTRAINT uk_videojuego_original UNIQUE (id_juego_original),
    CONSTRAINT ck_videojuego_pegi     CHECK (pegi IN (3, 7, 12, 16, 18)),
    CONSTRAINT ck_videojuego_tipo     CHECK (tipo IN ('PAGO', 'F2P'))
    -- R-07 (id_juego_original <> id_juego) NO se puede declarar aquí:
    -- MariaDB no admite CHECK sobre columnas de una clave ajena con
    -- ON DELETE SET NULL / CASCADE (error 1901). Se controlaría con un
    -- disparador.
) ENGINE = InnoDB;

CREATE TABLE idioma_juego (
    id_juego  INT         NOT NULL,
    idioma    VARCHAR(40) NOT NULL,
```

```
    CONSTRAINT pk_idioma_juego PRIMARY KEY (id_juego, idioma),
    CONSTRAINT fk_idioma_juego FOREIGN KEY (id_juego)
        REFERENCES videojuego (id_juego) ON DELETE CASCADE
) ENGINE = InnoDB;

CREATE TABLE juego_pago (
    id_juego  INT          NOT NULL,
    precio    DECIMAL(6,2) NOT NULL,
    CONSTRAINT pk_juego_pago        PRIMARY KEY (id_juego),
    CONSTRAINT fk_juego_pago_juego  FOREIGN KEY (id_juego)
        REFERENCES videojuego (id_juego) ON DELETE CASCADE,
    CONSTRAINT ck_juego_pago_precio CHECK (precio >= 0)
) ENGINE = InnoDB;

CREATE TABLE juego_f2p (
    id_juego          INT     NOT NULL,
    tiene_micropagos  CHAR(1) NOT NULL DEFAULT 'N',
    CONSTRAINT pk_juego_f2p           PRIMARY KEY (id_juego),
    CONSTRAINT fk_juego_f2p_juego     FOREIGN KEY (id_juego)
        REFERENCES videojuego (id_juego) ON DELETE CASCADE,
    CONSTRAINT ck_juego_f2p_micropagos CHECK (tiene_micropagos IN ('S', 'N'))
) ENGINE = InnoDB;

CREATE TABLE juego_genero (
    id_juego   INT NOT NULL,
    id_genero  INT NOT NULL,
    CONSTRAINT pk_juego_genero        PRIMARY KEY (id_juego, id_genero),
    CONSTRAINT fk_juego_genero_juego  FOREIGN KEY (id_juego)
        REFERENCES videojuego (id_juego) ON DELETE CASCADE,
    CONSTRAINT fk_juego_genero_genero FOREIGN KEY (id_genero)
        REFERENCES genero (id_genero)
) ENGINE = InnoDB;

-- ---------------------------------------------------------------------
-- Relaciones del jugador con los juegos
-- ---------------------------------------------------------------------
CREATE TABLE biblioteca (
    id_jugador         INT  NOT NULL,
    id_juego           INT  NOT NULL,
    fecha_adquisicion  DATE NOT NULL DEFAULT (CURRENT_DATE),
    CONSTRAINT pk_biblioteca         PRIMARY KEY (id_jugador, id_juego),
    CONSTRAINT fk_biblioteca_jugador FOREIGN KEY (id_jugador)
        REFERENCES jugador (id_usuario) ON DELETE CASCADE,
    CONSTRAINT fk_biblioteca_juego   FOREIGN KEY (id_juego)
        REFERENCES videojuego (id_juego)
) ENGINE = InnoDB;

CREATE TABLE juega (
    id_jugador     INT          NOT NULL,
    id_juego       INT          NOT NULL,
    id_plataforma  INT          NOT NULL,
    horas_jugadas  DECIMAL(7,1) NOT NULL DEFAULT 0,
    CONSTRAINT pk_juega            PRIMARY KEY (id_jugador, id_juego, id_plataforma),
    CONSTRAINT fk_juega_jugador    FOREIGN KEY (id_jugador)
        REFERENCES jugador (id_usuario) ON DELETE CASCADE,
    CONSTRAINT fk_juega_juego      FOREIGN KEY (id_juego)
        REFERENCES videojuego (id_juego),
    CONSTRAINT fk_juega_plataforma FOREIGN KEY (id_plataforma)
        REFERENCES plataforma (id_plataforma),
    CONSTRAINT ck_juega_horas      CHECK (horas_jugadas >= 0)
```

```
) ENGINE = InnoDB;

CREATE TABLE amistad (
    id_jugador  INT NOT NULL,
    id_amigo    INT NOT NULL,
    CONSTRAINT pk_amistad         PRIMARY KEY (id_jugador, id_amigo),
    CONSTRAINT fk_amistad_jugador FOREIGN KEY (id_jugador)
        REFERENCES jugador (id_usuario) ON DELETE CASCADE,
    CONSTRAINT fk_amistad_amigo   FOREIGN KEY (id_amigo)
        REFERENCES jugador (id_usuario) ON DELETE CASCADE,
    CONSTRAINT ck_amistad_orden   CHECK (id_jugador < id_amigo)
) ENGINE = InnoDB;

-- ---------------------------------------------------------------------
-- Entidades débiles
-- ---------------------------------------------------------------------
CREATE TABLE logro (
    id_juego     INT          NOT NULL,
    num_logro    SMALLINT     NOT NULL,
    nombre       VARCHAR(100) NOT NULL,
    descripcion  VARCHAR(400),
    puntos       SMALLINT     NOT NULL,
    CONSTRAINT pk_logro        PRIMARY KEY (id_juego, num_logro),
    CONSTRAINT fk_logro_juego  FOREIGN KEY (id_juego)
        REFERENCES videojuego (id_juego) ON DELETE CASCADE,
    CONSTRAINT ck_logro_puntos CHECK (puntos > 0)
) ENGINE = InnoDB;

CREATE TABLE desbloquea (
    id_jugador  INT      NOT NULL,
    id_juego    INT      NOT NULL,
    num_logro   SMALLINT NOT NULL,
    CONSTRAINT pk_desbloquea         PRIMARY KEY (id_jugador, id_juego, num_logro),
    CONSTRAINT fk_desbloquea_jugador FOREIGN KEY (id_jugador)
        REFERENCES jugador (id_usuario) ON DELETE CASCADE,
    CONSTRAINT fk_desbloquea_logro   FOREIGN KEY (id_juego, num_logro)
        REFERENCES logro (id_juego, num_logro) ON DELETE CASCADE
) ENGINE = InnoDB;

CREATE TABLE dlc (
    id_dlc             INT          NOT NULL AUTO_INCREMENT,
    titulo             VARCHAR(150) NOT NULL,
    precio             DECIMAL(6,2) NOT NULL,
    fecha_lanzamiento  DATE,
    id_juego           INT          NOT NULL,
    CONSTRAINT pk_dlc        PRIMARY KEY (id_dlc),
    CONSTRAINT fk_dlc_juego  FOREIGN KEY (id_juego)
        REFERENCES videojuego (id_juego) ON DELETE CASCADE,
    CONSTRAINT ck_dlc_precio CHECK (precio >= 0)
) ENGINE = InnoDB;

-- ---------------------------------------------------------------------
-- Pedidos y reseñas
-- ---------------------------------------------------------------------
CREATE TABLE pedido (
    num_pedido  INT  NOT NULL AUTO_INCREMENT,
    fecha       DATE NOT NULL DEFAULT (CURRENT_DATE),
    id_jugador  INT  NOT NULL,
    CONSTRAINT pk_pedido         PRIMARY KEY (num_pedido),
    CONSTRAINT fk_pedido_jugador FOREIGN KEY (id_jugador)
```

```sql
        REFERENCES jugador (id_usuario)
) ENGINE = InnoDB;

CREATE TABLE linea_pedido (
    num_pedido     INT          NOT NULL,
    id_juego       INT          NOT NULL,
    precio_pagado  DECIMAL(6,2) NOT NULL,
    CONSTRAINT pk_linea_pedido        PRIMARY KEY (num_pedido, id_juego),
    CONSTRAINT fk_linea_pedido_pedido FOREIGN KEY (num_pedido)
        REFERENCES pedido (num_pedido) ON DELETE CASCADE,
    CONSTRAINT fk_linea_pedido_juego  FOREIGN KEY (id_juego)
        REFERENCES juego_pago (id_juego),
    CONSTRAINT ck_linea_pedido_precio CHECK (precio_pagado >= 0)
) ENGINE = InnoDB;

CREATE TABLE resena (
    id_resena    INT           NOT NULL AUTO_INCREMENT,
    nota         TINYINT       NOT NULL,
    texto        VARCHAR(2000),
    fecha        DATE          NOT NULL DEFAULT (CURRENT_DATE),
    recomendado  CHAR(1)       NOT NULL,
    id_jugador   INT           NOT NULL,
    id_juego     INT           NOT NULL,
    CONSTRAINT pk_resena             PRIMARY KEY (id_resena),
    CONSTRAINT fk_resena_biblioteca  FOREIGN KEY (id_jugador, id_juego)
        REFERENCES biblioteca (id_jugador, id_juego) ON DELETE CASCADE,
    CONSTRAINT uk_resena_posesion    UNIQUE (id_jugador, id_juego),
    CONSTRAINT ck_resena_nota        CHECK (nota BETWEEN 1 AND 10),
    CONSTRAINT ck_resena_recomendado CHECK (recomendado IN ('S', 'N'))
) ENGINE = InnoDB COMMENT = 'Una reseña como máximo por juego poseído';

-- ---------------------------------------------------------------------
-- Índices adicionales
-- InnoDB crea solo un índice para cada clave ajena que no tenga ya uno.
-- MariaDB no admite índices sobre expresiones como UPPER(titulo); con la
-- intercalación _ci las comparaciones ya ignoran mayúsculas.
-- ---------------------------------------------------------------------
CREATE INDEX ix_videojuego_pegi ON videojuego (pegi);
CREATE INDEX ix_videojuego_titulo ON videojuego (titulo);

-- ---------------------------------------------------------------------
-- Vistas
-- ---------------------------------------------------------------------
CREATE OR REPLACE VIEW v_jugador_edad AS
SELECT j.id_usuario,
       j.nick,
       u.email,
       j.fecha_nacimiento,
       TIMESTAMPDIFF(YEAR, j.fecha_nacimiento, CURRENT_DATE) AS edad
  FROM jugador j
  JOIN usuario u ON u.id_usuario = j.id_usuario;

CREATE OR REPLACE VIEW v_nota_media AS
SELECT v.id_juego,
       v.titulo,
       ROUND(AVG(r.nota), 1) AS nota_media,
       COUNT(r.id_resena)    AS num_resenas
  FROM videojuego v
  LEFT JOIN resena r ON r.id_juego = v.id_juego
 GROUP BY v.id_juego, v.titulo;
```

```sql
CREATE OR REPLACE VIEW v_catalogo AS
SELECT v.id_juego,
       v.titulo,
       e.nombre AS estudio,
       v.fecha_lanzamiento,
       v.pegi,
       v.tipo,
       p.precio
  FROM videojuego v
  JOIN estudio e         ON e.id_estudio = v.id_estudio
  LEFT JOIN juego_pago p ON p.id_juego   = v.id_juego;

CREATE OR REPLACE VIEW v_juegos_pegi18 AS
SELECT id_juego, titulo, fecha_lanzamiento, pegi, tipo, id_estudio
  FROM videojuego
 WHERE pegi = 18
  WITH CHECK OPTION;

-- ---------------------------------------------------------------------
-- Usuarios, roles y privilegios
-- (en un servidor compartido, cambia 'localhost' por el equipo o '%')
-- ---------------------------------------------------------------------
DROP USER IF EXISTS 'pixelvault'@'localhost', 'pv_app'@'localhost',
'pv_analista'@'localhost';
DROP ROLE IF EXISTS rol_pv_aplicacion, rol_pv_lectura;

-- Propietario: todos los permisos, pero solo sobre su base de datos
CREATE USER 'pixelvault'@'localhost' IDENTIFIED BY 'PixelVault_2026';
GRANT ALL PRIVILEGES ON pixelvault.* TO 'pixelvault'@'localhost';

CREATE ROLE rol_pv_aplicacion;
CREATE ROLE rol_pv_lectura;

GRANT SELECT ON pixelvault.estudio      TO rol_pv_aplicacion;
GRANT SELECT ON pixelvault.videojuego   TO rol_pv_aplicacion;
GRANT SELECT ON pixelvault.idioma_juego TO rol_pv_aplicacion;
GRANT SELECT ON pixelvault.juego_pago   TO rol_pv_aplicacion;
GRANT SELECT ON pixelvault.juego_f2p    TO rol_pv_aplicacion;
GRANT SELECT ON pixelvault.genero       TO rol_pv_aplicacion;
GRANT SELECT ON pixelvault.juego_genero TO rol_pv_aplicacion;
GRANT SELECT ON pixelvault.plataforma   TO rol_pv_aplicacion;
GRANT SELECT ON pixelvault.logro        TO rol_pv_aplicacion;
GRANT SELECT ON pixelvault.dlc          TO rol_pv_aplicacion;
GRANT SELECT, INSERT, UPDATE, DELETE ON pixelvault.usuario    TO rol_pv_aplicacion;
GRANT SELECT, INSERT, UPDATE, DELETE ON pixelvault.jugador    TO rol_pv_aplicacion;
GRANT SELECT, INSERT, UPDATE, DELETE ON pixelvault.biblioteca TO rol_pv_aplicacion;
GRANT SELECT, INSERT, UPDATE, DELETE ON pixelvault.juega      TO rol_pv_aplicacion;
GRANT SELECT, INSERT, DELETE         ON pixelvault.amistad    TO rol_pv_aplicacion;
GRANT SELECT, INSERT                 ON pixelvault.desbloquea TO rol_pv_aplicacion;
GRANT SELECT, INSERT, UPDATE, DELETE ON pixelvault.resena     TO rol_pv_aplicacion;
GRANT SELECT, INSERT ON pixelvault.pedido       TO rol_pv_aplicacion;
GRANT SELECT, INSERT ON pixelvault.linea_pedido TO rol_pv_aplicacion;
GRANT SELECT ON pixelvault.v_catalogo     TO rol_pv_aplicacion;
GRANT SELECT ON pixelvault.v_nota_media   TO rol_pv_aplicacion;
GRANT SELECT ON pixelvault.v_jugador_edad TO rol_pv_aplicacion;

GRANT SELECT ON pixelvault.v_catalogo   TO rol_pv_lectura;
GRANT SELECT ON pixelvault.v_nota_media TO rol_pv_lectura;
```

```sql
CREATE USER 'pv_app'@'localhost'      IDENTIFIED BY 'App_PixelVault_2026';
CREATE USER 'pv_analista'@'localhost' IDENTIFIED BY 'Analista_PV_2026';
GRANT rol_pv_aplicacion TO 'pv_app'@'localhost';
GRANT rol_pv_lectura    TO 'pv_analista'@'localhost';
-- En MariaDB los roles no se activan solos al conectar:
SET DEFAULT ROLE rol_pv_aplicacion FOR 'pv_app'@'localhost';
SET DEFAULT ROLE rol_pv_lectura    FOR 'pv_analista'@'localhost';

-- ------------------------------------------------------------------------
-- Datos de prueba (mismos que en Oracle)
-- ------------------------------------------------------------------------
INSERT INTO estudio (id_estudio, nombre, pais) VALUES
    (10, 'Rockstar North', 'Reino Unido'), (11, 'Larian Studios', 'Bélgica'),
    (12, 'Nintendo EPD', 'Japón'),         (13, 'FromSoftware', 'Japón'),
    (14, 'CD Projekt Red', 'Polonia'),     (15, 'Supergiant Games', 'EE. UU.'),
    (16, 'Team Cherry', 'Australia'),      (17, 'ConcernedApe', 'EE. UU.'),
    (18, 'Epic Games', 'EE. UU.'),         (19, 'Valve', 'EE. UU.');

INSERT INTO videojuego (id_juego, titulo, fecha_lanzamiento, pegi, tipo, id_estudio,
id_juego_original) VALUES
    (1,  'Grand Theft Auto V', '2013-09-17', 18, 'PAGO', 10, NULL),
    (2,  'Baldur''s Gate 3', '2023-08-03', 18, 'PAGO', 11, NULL),
    (3,  'The Legend of Zelda: Tears of the Kingdom', '2023-05-12', 12, 'PAGO', 12, NULL),
    (4,  'Elden Ring', '2022-02-25', 16, 'PAGO', 13, NULL),
    (5,  'Hollow Knight', '2017-02-24', 7, 'PAGO', 16, NULL),
    (6,  'Hades', '2020-09-17', 12, 'PAGO', 15, NULL),
    (7,  'The Witcher 3: Wild Hunt', '2015-05-19', 18, 'PAGO', 14, NULL),
    (8,  'Stardew Valley', '2016-02-26', 7, 'PAGO', 17, NULL),
    (9,  'Fortnite', '2017-07-25', 12, 'F2P', 18, NULL),
    (10, 'Counter-Strike 2', '2023-09-27', 18, 'F2P', 19, NULL),
    (11, 'Hades II', '2025-09-25', 12, 'PAGO', 15, 6);

INSERT INTO juego_pago (id_juego, precio) VALUES
    (1, 29.99), (2, 59.99), (3, 69.99), (4, 59.99), (5, 14.79),
    (6, 24.50), (7, 29.99), (8, 13.99), (11, 29.50);
INSERT INTO juego_f2p (id_juego, tiene_micropagos) VALUES (9, 'S'), (10, 'S');

INSERT INTO idioma_juego (id_juego, idioma) VALUES
    (2, 'español'), (2, 'inglés'), (2, 'francés'), (6, 'español'), (6, 'inglés');

INSERT INTO genero (id_genero, nombre) VALUES
    (1, 'Acción'), (2, 'Rol'), (3, 'Aventura'), (4, 'Roguelike'),
    (5, 'Metroidvania'), (6, 'Simulación'), (7, 'Shooter');
INSERT INTO juego_genero (id_juego, id_genero) VALUES
    (1, 1), (2, 2), (3, 3), (4, 1), (4, 2), (5, 5), (6, 4),
    (7, 2), (8, 6), (9, 7), (10, 7), (11, 4);

INSERT INTO plataforma (id_plataforma, nombre, fabricante) VALUES
    (1, 'PC', NULL), (2, 'PlayStation 5', 'Sony'),
    (3, 'Xbox Series X|S', 'Microsoft'), (4, 'Nintendo Switch 2', 'Nintendo');

INSERT INTO logro (id_juego, num_logro, nombre, puntos) VALUES
    (4, 1, 'Primera victoria', 10), (4, 2, 'Explorador de las Tierras Intermedias', 30),
    (6, 1, 'Primera victoria', 10), (6, 2, 'Escapar del Inframundo', 50);

INSERT INTO dlc (id_dlc, titulo, precio, fecha_lanzamiento, id_juego) VALUES
    (1, 'Shadow of the Erdtree', 39.99, '2024-06-21', 4),
    (2, 'Hearts of Stone', 9.99, '2015-10-13', 7),
    (3, 'Blood and Wine', 19.99, '2016-05-31', 7);
```

```sql
INSERT INTO usuario (id_usuario, email, nombre, apellidos) VALUES
    (101, 'noctis@ejemplo.es', 'Noa', 'Sáenz Ruiz'),
    (102, 'tarasca@ejemplo.es', 'Lucía', 'Garrido Pérez'),
    (103, 'kirbyto@ejemplo.es', 'Hugo', 'Martínez Sanz'),
    (104, 'ezio@ejemplo.es', 'Iker', 'López Ochoa'),
    (201, 'marta.dev@ejemplo.es', 'Marta', 'Ibáñez Gil'),
    (301, 'soporte@ejemplo.es', 'Soporte', 'PixelVault');

INSERT INTO jugador (id_usuario, nick, fecha_nacimiento, ciudad, pais) VALUES
    (101, 'NoctisRJ',  '2006-03-14', 'Logroño',   'España'),
    (102, 'LaTarasca', '1999-11-02', 'Haro',      'España'),
    (103, 'Kirbyto',   '2012-06-30', 'Calahorra', 'España'),
    (104, 'Ezio_Lgr',  '2001-01-21', 'Logroño',   'España');

INSERT INTO desarrollador (id_usuario, cargo, id_estudio) VALUES
    (104, 'QA tester', 15), (201, 'Diseñadora de niveles', 11);

INSERT INTO biblioteca (id_jugador, id_juego, fecha_adquisicion) VALUES
    (101, 6, '2026-09-01'), (101, 5, '2026-09-01'), (101, 4, '2026-09-02'),
    (101, 9, '2025-12-24'), (102, 6, '2026-09-02'), (102, 8, '2026-09-02'),
    (103, 5, '2026-03-10'), (103, 9, '2026-01-05'), (104, 2, '2026-09-10'),
    (104, 7, '2026-09-10'), (104, 10, '2025-06-01');

INSERT INTO juega (id_jugador, id_juego, id_plataforma, horas_jugadas) VALUES
    (101, 6, 1, 40), (101, 6, 4, 12), (101, 4, 1, 1.5), (101, 4, 2, 0.4),
    (102, 6, 1, 25), (102, 8, 4, 80), (103, 5, 4, 10), (103, 9, 4, 30),
    (104, 2, 1, 120), (104, 10, 1, 300);

INSERT INTO amistad (id_jugador, id_amigo) VALUES (101, 102), (101, 104), (102, 103);

INSERT INTO desbloquea (id_jugador, id_juego, num_logro) VALUES
    (101, 6, 1), (101, 6, 2), (101, 4, 1), (102, 6, 1);

INSERT INTO pedido (num_pedido, fecha, id_jugador) VALUES
    (5001, '2026-09-01', 101), (5002, '2026-09-02', 102),
    (5003, '2026-09-02', 101), (5004, '2026-09-10', 104);

INSERT INTO linea_pedido (num_pedido, id_juego, precio_pagado) VALUES
    (5001, 6, 24.50), (5001, 5, 14.79), (5002, 6, 19.99), (5002, 8, 13.99),
    (5003, 4, 59.99), (5004, 2, 49.99), (5004, 7, 9.99);

INSERT INTO resena (id_resena, nota, texto, recomendado, id_jugador, id_juego) VALUES
    (1, 10, 'Cada partida es distinta. Imprescindible.', 'S', 101, 6),
    (2, 9, 'Relajante y adictivo.', 'S', 102, 8),
    (3, 8, 'Enorme, aunque el tercer acto se hace largo.', 'S', 104, 2);
```

¿Y MySQL?

El script de MariaDB sirve de base para **MySQL 8**, con algunos retoques: MySQL es todavía más estricto con los `CHECK` sobre columnas de claves ajenas que tienen acciones en cascada (afectaría a `ck_amistad_orden`), y los roles se activan con `SET DEFAULT ROLE ALL TO usuario`.

Resumen

- Los tres gestores implementan el modelo relacional con la misma lógica: tablas, claves primarias y ajenas, `UNIQUE`, `CHECK`, vistas, usuarios y roles.
- Cambian los **tipos** (`VARCHAR2` / `NVARCHAR` / `VARCHAR`, `NUMBER` / `INT` / `DECIMAL`), el **autonumérico** (`IDENTITY` / `IDENTITY(1,1)` / `AUTO_INCREMENT`) y la **organización** (usuario = esquema en Oracle; base de datos independiente en los otros dos).
- Algunas decisiones del diseño no se pueden implantar igual en todos los gestores (cascadas en SQL Server, `CHECK` con `SET NULL` en MariaDB): se buscan alternativas y **se documentan**.

RA 2 · REPASO

Errores ORA- frecuentes

Cuando una sentencia falla, Oracle devuelve un código `ORA-nnnnn` y un mensaje. Aprender a leerlos ahorra mucho tiempo: casi siempre dicen **exactamente** qué ha pasado. Esta página reúne los que más aparecen al crear una base de datos, agrupados por el momento en que suelen salir.

Cómo leer un error

1. Fíjate en el **código**: es lo único que no cambia con el idioma de la instalación. Los textos de los mensajes pueden variar ligeramente según el idioma y la versión.
2. Mira el **nombre de la restricción** entre paréntesis: si la nombraste bien (`CK_VIDEOJUEGO_PEGI`), ya sabes qué regla se ha incumplido.
3. En SQL Developer, la **línea y columna** del error señalan dónde está el problema de sintaxis.
4. Busca el código en la documentación de Oracle (*Database Error Messages*); en 23ai/26ai el propio mensaje incluye un enlace a la ayuda.

Al conectarse y crear usuarios

Código	Mensaje (resumido)	Causa habitual	Solución
ORA-12514	El listener no conoce el servicio solicitado	Nombre de servicio mal escrito, o la base de datos no está arrancada	Revisa el servicio: `XEPDB1` (21c) o `FREEPDB1` (23ai/26ai)
ORA-01017	Usuario o contraseña no válidos	Contraseña equivocada (distingue mayúsculas) o usuario creado en otro contenedor	Comprueba la contraseña y que te conectas a la PDB
ORA-65096	Nombre de usuario o rol común no válido	Estás en `CDB$ROOT` y el nombre no empieza por `C##`	Conéctate a la PDB; no añadas `C##`
ORA-01045	El usuario no tiene privilegio `CREATE SESSION`	Usuario recién creado sin permisos	`GRANT CREATE SESSION TO usuario;`
ORA-28000	La cuenta está bloqueada	Se bloqueó a mano o por demasiados intentos fallidos	`ALTER USER usuario ACCOUNT UNLOCK;` (como administrador)
ORA-01920	El nombre de usuario entra en conflicto con otro	El usuario ya existe (has ejecutado el script dos veces)	`DROP USER ... CASCADE` y repetir, o saltarte ese paso
ORA-01921	El nombre de rol entra en conflicto con otro	El rol ya existe	`DROP ROLE ...` o saltarte ese paso

Al crear o modificar tablas

Código	Mensaje (resumido)	Causa habitual	Solución
ORA-00955	El nombre ya lo está utilizando un objeto existente	La tabla (o vista, índice…) ya existe	Bórrala antes o usa otro nombre; en 23ai, `IF NOT EXISTS`
ORA-00942	La tabla o vista no existe	Nombre mal escrito; tabla de otro esquema sin prefijo; sin permisos; tabla creada con comillas	Revisa el nombre, añade `esquema.` o pide permisos
ORA-00904	Identificador no válido	Columna mal escrita, o una **palabra reservada** como nombre (`date` , `order` , `level` …)	Corrige el nombre; no uses palabras reservadas
ORA-00907	Falta el paréntesis derecho	Suele ser una coma de más o de menos, o un tipo mal escrito, no un paréntesis	Revisa la línea del error y la anterior
ORA-00902	Tipo de dato no válido	Tipo inexistente en Oracle (`INT(10)` , `DATETIME` , `BOOLEAN` en 21c…)	Usa `NUMBER(10)` , `DATE` , `CHAR(1)` …
ORA-02264	El nombre ya lo utiliza una restricción existente	Dos restricciones con el mismo nombre (aunque sean de tablas distintas)	Los nombres de restricción son únicos en el esquema: sigue el convenio `tipo_tabla_...`
ORA-02270	No hay clave única o primaria que coincida con esta lista de columnas	La FK apunta a columnas que no son PK ni UK de la tabla referida	Apunta a la clave primaria, o declara `UNIQUE` esas columnas
ORA-02436	Fecha o variable del sistema mal especificada en una restricción CHECK	`SYSDATE` , `USER` … dentro de un `CHECK`	Esa regla no es un `CHECK` : documentarla como restricción no representable
ORA-01758	La tabla debe estar vacía para añadir una columna obligatoria	`ADD` de una columna `NOT NULL` sin `DEFAULT`	Añade un `DEFAULT`
ORA-01441	No se puede reducir la longitud de la columna	Hay valores que no cabrían	Revisa los datos antes de reducir
ORA-02449	Claves únicas/primarias de la tabla referenciadas por claves ajenas	`DROP TABLE` de una tabla referenciada	Borra antes las hijas o usa `CASCADE CONSTRAINTS`
ORA-02266	Claves de la tabla referenciadas por claves ajenas activadas	`TRUNCATE` de una tabla referenciada	Vacía antes las hijas o desactiva sus claves ajenas

Código	Mensaje (resumido)	Causa habitual	Solución
ORA-02429	No se puede borrar el índice usado para una clave única o primaria	`DROP INDEX` de un índice de una restricción	Borra la restricción, no el índice
ORA-01950	Sin privilegios en el tablespace	El usuario no tiene cuota	`ALTER USER usuario QUOTA 100M ON users;`
ORA-01031	Privilegios insuficientes	Falta un privilegio de sistema (`CREATE VIEW` ...) o de objeto	Pide el privilegio al administrador o al propietario

> ⚠ **ORA-01031 al crear una vista**
>
> Un error muy típico: el usuario tiene `SELECT` sobre una tabla de otro esquema **a través de un rol**, puede consultarla sin problema, pero al crear una vista sobre ella obtiene ORA-01031. Para crear vistas (y procedimientos) sobre objetos ajenos, Oracle exige que el permiso se haya concedido **directamente al usuario**, no a través de un rol.

Al insertar o modificar datos

Código	Mensaje (resumido)	Qué restricción lo provoca	Ejemplo en PixelVault
ORA-00001	Restricción única violada	`PRIMARY KEY` o `UNIQUE`	Dos jugadores con el mismo *nick*
ORA-01400	No se puede insertar NULL	`NOT NULL` (o `''`, que en Oracle es `NULL`)	Un juego sin estudio
ORA-01407	No se puede actualizar a NULL	`NOT NULL` en un `UPDATE`	Quitar el título a un juego
ORA-02290	Restricción de control violada	`CHECK`	PEGI 15
ORA-02291	Clave principal no encontrada	`FOREIGN KEY` al insertar o modificar la hija	Un juego del estudio 99, que no existe
ORA-02292	Registro secundario encontrado	`FOREIGN KEY` al borrar la madre	Borrar un estudio con juegos
ORA-12899	Valor demasiado grande para la columna	Tamaño de `VARCHAR2` / `CHAR`	`tipo = 'GRATIS'` en un `VARCHAR2(4)`
ORA-01438	Valor mayor que la precisión permitida	Precisión de `NUMBER`	Un precio de 12 345,60 en un `NUMBER(6,2)`
ORA-01402	Violación de la cláusula WHERE de la vista WITH CHECK OPTION	`WITH CHECK OPTION`	Un juego PEGI 3 en `v_juegos_pegi18`

Código	Mensaje (resumido)	Qué restricción lo provoca	Ejemplo en PixelVault
ORA-42399	No se puede realizar DML en una vista de solo lectura	`WITH READ ONLY`	Modificar `v_catalogo`
ORA-32795	No se puede insertar en una columna de identidad siempre generada	`GENERATED ALWAYS AS IDENTITY`	Dar un `id_juego` a mano
ORA-01861 / **ORA-01843**	El literal no coincide con el formato / mes no válido	Fecha escrita como texto con otro formato	Usa `DATE '2023-08-03'` o `TO_DATE` con máscara

RA 2 · REPASO

Ejercicios resueltos

Doce ejercicios de dificultad creciente sobre otros casos del mundo de los videojuegos: una **liga de eSports**, un **juego de rol online** y una **web de *speedrun***. Intenta resolver cada uno antes de desplegar la solución, y si puedes, **ejecútalo en Oracle**: es la mejor forma de comprobarlo.

Bloque	Ejercicios	Apartados
Tipos y tablas	1 a 5	3, 4 y 5
Errores, orden y cambios	6 a 8	4 y 6
Vistas, usuarios y diagnóstico	9 a 11	8, 9 y errores ORA-
Otros gestores	12	12

Bloque 1 · Tipos de datos y tablas

Ejercicio 1 · Elegir tipos de datos

Una liga de eSports guarda de cada **torneo**: un código interno, el nombre, la fecha y hora de inicio, el premio total en euros (con céntimos, nunca más de un millón), el número de plazas (máximo 64), el código postal de la sede, si es en línea o presencial, y un reglamento que puede ocupar varias páginas. Elige el tipo de Oracle de cada dato y justifícalo.

Solución

Dato	Tipo	Por qué
Código interno	`NUMBER(10)` + `IDENTITY`	Clave subrogada, entero
Nombre	`VARCHAR2(100)`	Texto de longitud variable
Fecha y hora de inicio	`DATE`	En Oracle, `DATE` ya guarda la hora; `TIMESTAMP` solo si hicieran falta fracciones de segundo
Premio	`NUMBER(9,2)`	Importe exacto: hasta 9 999 999,99 cubre el millón con margen. Nunca `BINARY_FLOAT`
Plazas	`NUMBER(2)` + `CHECK (plazas BETWEEN 2 AND 64)`	Entero pequeño con un rango
Código postal	`VARCHAR2(10)`	Tiene cifras pero no se opera con él y puede empezar por 0
En línea	`CHAR(1)` + `CHECK (en_linea IN ('S','N'))`	Compatible con 21c; en 23ai/26ai, `BOOLEAN`
Reglamento	`CLOB`	Puede superar los 4000 bytes de un `VARCHAR2`

Ejercicio 2 · Una tabla con todas las restricciones

Crea la tabla **EQUIPO**: identificador autonumérico (Oracle lo genera siempre), nombre obligatorio y único, siglas de exactamente 3 letras, obligatorias y únicas (como `KOI` o `G2A`), país, fecha de fundación y número máximo de jugadores en plantilla, entre 5 y 10, que si no se indica vale 7.

Solución

```
CREATE TABLE equipo (
    id_equipo        NUMBER(10)    GENERATED ALWAYS AS IDENTITY,
    nombre           VARCHAR2(80)  NOT NULL,
    siglas           CHAR(3)       NOT NULL,
    pais             VARCHAR2(60),
    fecha_fundacion  DATE,
    max_jugadores    NUMBER(2)     DEFAULT 7 NOT NULL,
    CONSTRAINT pk_equipo           PRIMARY KEY (id_equipo),
    CONSTRAINT uk_equipo_nombre    UNIQUE (nombre),
    CONSTRAINT uk_equipo_siglas    UNIQUE (siglas),
    CONSTRAINT ck_equipo_max       CHECK (max_jugadores BETWEEN 5 AND 10),
    CONSTRAINT ck_equipo_siglas    CHECK (siglas = UPPER(siglas))
);
```

- `GENERATED ALWAYS` porque el enunciado dice que lo genera **siempre** Oracle.
- `CHAR(3)` es adecuado porque la longitud es fija. El `CHECK` con `UPPER` es un extra razonable para que no haya `koi` y `KOI` a la vez.
- `DEFAULT 7` + `NOT NULL`: si no se indica, vale 7, y nunca puede quedar vacío.

Ejercicio 3 · Clave ajena opcional y una trampa

Crea **JUGADOR_PRO** (identificador, alias único, país, fecha de nacimiento obligatoria, equipo actual y fecha de fichaje). Un jugador puede ser agente libre (sin equipo). Además: la fecha de fichaje debe estar vacía si y solo si el jugador no tiene equipo.

Después responde: ¿qué pasaría si la clave ajena hacia EQUIPO se declarase con `ON DELETE SET NULL`?

Solución

```
CREATE TABLE jugador_pro (
    id_pro            NUMBER(10)   GENERATED ALWAYS AS IDENTITY,
    alias             VARCHAR2(30) NOT NULL,
    pais              VARCHAR2(60),
    fecha_nacimiento  DATE         NOT NULL,
    id_equipo         NUMBER(10),                  -- NULL: agente libre
    fecha_fichaje     DATE,
    CONSTRAINT pk_jugador_pro        PRIMARY KEY (id_pro),
    CONSTRAINT uk_jugador_pro_alias  UNIQUE (alias),
    CONSTRAINT fk_jugador_pro_equipo FOREIGN KEY (id_equipo)
        REFERENCES equipo (id_equipo),
    CONSTRAINT ck_jugador_pro_fichaje CHECK (
        (id_equipo IS NULL     AND fecha_fichaje IS NULL) OR
        (id_equipo IS NOT NULL AND fecha_fichaje IS NOT NULL)
    )
);
```

La trampa. Con `ON DELETE SET NULL`, al borrar un equipo Oracle pondría `id_equipo = NULL` en sus jugadores, pero **no tocaría** `fecha_fichaje`. Las filas quedarían con equipo nulo y fecha rellena, incumpliendo `ck_jugador_pro_fichaje`, así que el `DELETE` del equipo fallaría con **ORA-02290**. Las acciones referenciales también tienen que respetar las demás restricciones. Por eso la dejamos sin acción: para borrar un equipo, antes hay que dar de baja a sus jugadores (poniendo a `NULL` las dos columnas).

Ejercicio 4 · Entidad débil y 1:1 con clave compuesta

En un juego de rol online, cada **cuenta** (identificador, email único, fecha de alta que por defecto es hoy) tiene **personajes**, cuyo nombre solo es único dentro de su cuenta. De cada personaje se guarda la clase (`GUERRERO` , `MAGO` o `ARQUERO`) y el nivel (de 1 a 60, empieza en 1). Si se borra una cuenta, se borran sus personajes. Cada **gremio** (identificador, nombre único) está liderado por exactamente un personaje, y un personaje lidera como mucho un gremio.

Crea las tres tablas. ¿Qué ocurre si se intenta borrar la cuenta de un líder de gremio?

Solución

```
CREATE TABLE cuenta (
    id_cuenta   NUMBER(10)    GENERATED ALWAYS AS IDENTITY,
    email       VARCHAR2(120) NOT NULL,
    fecha_alta  DATE          DEFAULT SYSDATE NOT NULL,
    CONSTRAINT pk_cuenta       PRIMARY KEY (id_cuenta),
    CONSTRAINT uk_cuenta_email UNIQUE (email)
);

CREATE TABLE personaje (
    id_cuenta  NUMBER(10)   NOT NULL,
    nombre     VARCHAR2(30) NOT NULL,
    clase      VARCHAR2(10) NOT NULL,
    nivel      NUMBER(2)    DEFAULT 1 NOT NULL,
    CONSTRAINT pk_personaje        PRIMARY KEY (id_cuenta, nombre),  -- débil
    CONSTRAINT fk_personaje_cuenta FOREIGN KEY (id_cuenta)
        REFERENCES cuenta (id_cuenta) ON DELETE CASCADE,
    CONSTRAINT ck_personaje_clase  CHECK (clase IN ('GUERRERO', 'MAGO', 'ARQUERO')),
    CONSTRAINT ck_personaje_nivel  CHECK (nivel BETWEEN 1 AND 60)
);

CREATE TABLE gremio (
    id_gremio        NUMBER(10)   GENERATED ALWAYS AS IDENTITY,
    nombre           VARCHAR2(60) NOT NULL,
    id_cuenta_lider  NUMBER(10)   NOT NULL,
    nombre_lider     VARCHAR2(30) NOT NULL,
    CONSTRAINT pk_gremio        PRIMARY KEY (id_gremio),
    CONSTRAINT uk_gremio_nombre UNIQUE (nombre),
    CONSTRAINT fk_gremio_lider  FOREIGN KEY (id_cuenta_lider, nombre_lider)
        REFERENCES personaje (id_cuenta, nombre),
    CONSTRAINT uk_gremio_lider  UNIQUE (id_cuenta_lider, nombre_lider)   -- 1:1
);
```

- La clave de PERSONAJE es compuesta, así que la clave ajena de GREMIO también lo es (**dos columnas**).
- `NOT NULL` porque todo gremio tiene líder; `UNIQUE` porque un personaje lidera como mucho uno.

Borrar la cuenta de un líder: el borrado en cascada intentaría borrar el personaje, pero GREMIO lo referencia sin `ON DELETE` , así que todo el `DELETE` falla con **ORA-02292**. Primero hay que cambiar el líder del gremio (o borrar el gremio).

Ejercicio 5 · Ternaria con un lado 1

Crea la tabla **PARTICIPA**: un jugador profesional participa en un torneo con un único equipo (en cada torneo solo con uno) y lleva un dorsal del 1 al 99. En un mismo torneo, dos jugadores del mismo equipo no pueden llevar el mismo dorsal. Supón que existen las tablas JUGADOR_PRO, TORNEO (`id_torneo`) y EQUIPO.

Solución

```
CREATE TABLE participa (
    id_pro     NUMBER(10) NOT NULL,
    id_torneo  NUMBER(10) NOT NULL,
    id_equipo  NUMBER(10) NOT NULL,
    dorsal     NUMBER(2)  NOT NULL,
    CONSTRAINT pk_participa         PRIMARY KEY (id_pro, id_torneo),
    CONSTRAINT fk_participa_pro     FOREIGN KEY (id_pro)
        REFERENCES jugador_pro (id_pro),
    CONSTRAINT fk_participa_torneo  FOREIGN KEY (id_torneo)
        REFERENCES torneo (id_torneo),
    CONSTRAINT fk_participa_equipo  FOREIGN KEY (id_equipo)
        REFERENCES equipo (id_equipo),
    CONSTRAINT ck_participa_dorsal  CHECK (dorsal BETWEEN 1 AND 99),
    CONSTRAINT uk_participa_dorsal  UNIQUE (id_torneo, id_equipo, dorsal)
);

CREATE INDEX ix_participa_torneo ON participa (id_torneo);
CREATE INDEX ix_participa_equipo ON participa (id_equipo);
```

- Como fijados jugador y torneo el equipo es uno, la clave primaria es `(id_pro, id_torneo)` y `id_equipo` es una clave ajena normal `NOT NULL`.
- La regla del dorsal parece una restricción «difícil», pero es una **clave alternativa compuesta**.
- Los índices son para las claves ajenas que no son el inicio de la clave primaria. (`uk_participa_dorsal` ya empieza por `id_torneo`, así que `ix_participa_torneo` sería prescindible.)

Bloque 2 · Errores, orden y cambios

Ejercicio 6 · Encuentra los errores

Esta sentencia, para Oracle XE 21c, tiene **seis** problemas. Encuéntralos, di qué error daría cada uno y escribe la versión correcta. Existen CORREDOR (`id_corredor`) y CATEGORIA, cuya clave es `(id_juego, nombre_cat)`.

```
CREATE TABLE marca (
    id_marca     NUMBER(10) GENERATED ALWAYS AS IDENTITY,
    tiempo       NUMBER(8,3) NOT NULL
    date         DATE DEFAULT SYSDATE,
    video_url    VARCHAR2 NOT NULL,
    verificada   BOOLEAN,
    id_corredor  NUMBER(10) NOT NULL,
    id_juego     NUMBER(10),
    nombre_cat   VARCHAR2(40),
    CONSTRAINT pk_marca PRIMARY KEY (id_marca),
    CONSTRAINT ck_marca_fecha CHECK (date <= SYSDATE),
    CONSTRAINT fk_marca_categoria FOREIGN KEY (nombre_cat)
        REFERENCES categoria (nombre_cat),
    CONSTRAINT fk_marca_corredor FOREIGN KEY (id_corredor) REFERENCES corredor
);
```

Solución

#	Problema	Error
1	Falta la coma después de `tiempo NUMBER(8,3) NOT NULL`	ORA-00907
2	`date` es palabra reservada	ORA-00904
3	`VARCHAR2` sin tamaño	ORA-00906
4	`BOOLEAN` no existe en 21c (sí en 23ai/26ai)	ORA-00902
5	`SYSDATE` dentro de un `CHECK`	ORA-02436
6	La clave ajena apunta solo a `nombre_cat`, que no es la clave de CATEGORIA	ORA-02270

Además, `id_juego` y `nombre_cat` deberían ser `NOT NULL` si toda marca pertenece a una categoría. Y `REFERENCES corredor` sin columnas **es correcto**: Oracle entiende que apunta a su clave primaria (aunque escribirla es más claro).

```
CREATE TABLE marca (
    id_marca     NUMBER(10)    GENERATED ALWAYS AS IDENTITY,
    tiempo       NUMBER(8,3)   NOT NULL,
    fecha        DATE          DEFAULT SYSDATE NOT NULL,
    video_url    VARCHAR2(300) NOT NULL,
    verificada   CHAR(1)       DEFAULT 'N' NOT NULL,
    id_corredor  NUMBER(10)    NOT NULL,
    id_juego     NUMBER(10)    NOT NULL,
    nombre_cat   VARCHAR2(40)  NOT NULL,
    CONSTRAINT pk_marca            PRIMARY KEY (id_marca),
    CONSTRAINT ck_marca_verificada CHECK (verificada IN ('S', 'N')),
    CONSTRAINT ck_marca_tiempo     CHECK (tiempo > 0),
    CONSTRAINT fk_marca_categoria  FOREIGN KEY (id_juego, nombre_cat)
        REFERENCES categoria (id_juego, nombre_cat),
    CONSTRAINT fk_marca_corredor   FOREIGN KEY (id_corredor)
        REFERENCES corredor (id_corredor)
);
```

La regla «la fecha no puede ser futura» no es un `CHECK`: se documenta como restricción no representable y se implementa con un disparador.

Ejercicio 7 · Orden de creación y de borrado

Para la web de *speedrun* hay que crear JUEGO, CATEGORIA (débil de JUEGO), CORREDOR y MARCA (del ejercicio 6). Escribe el script completo en el orden correcto, con JUEGO (`id_juego` , `titulo`), CATEGORIA (`id_juego` , `nombre_cat` , `descripcion`) y CORREDOR (`id_corredor` , `alias` único), y después el script de borrado.

Solución

Niveles: JUEGO y CORREDOR no dependen de nadie (nivel 0); CATEGORIA depende de JUEGO (nivel 1); MARCA depende de CATEGORIA y CORREDOR (nivel 2).

```
CREATE TABLE juego (
    id_juego  NUMBER(10)    GENERATED BY DEFAULT AS IDENTITY,
    titulo    VARCHAR2(150) NOT NULL,
    CONSTRAINT pk_juego PRIMARY KEY (id_juego)
);

CREATE TABLE corredor (
    id_corredor  NUMBER(10)   GENERATED BY DEFAULT AS IDENTITY,
    alias        VARCHAR2(30) NOT NULL,
    CONSTRAINT pk_corredor       PRIMARY KEY (id_corredor),
    CONSTRAINT uk_corredor_alias UNIQUE (alias)
);

CREATE TABLE categoria (
    id_juego     NUMBER(10)    NOT NULL,
    nombre_cat   VARCHAR2(40)  NOT NULL,
    descripcion  VARCHAR2(400),
    CONSTRAINT pk_categoria       PRIMARY KEY (id_juego, nombre_cat),
    CONSTRAINT fk_categoria_juego FOREIGN KEY (id_juego)
        REFERENCES juego (id_juego) ON DELETE CASCADE
);

-- MARCA: la versión corregida del ejercicio 6
```

Borrado: orden inverso

```
DROP TABLE marca     PURGE;
DROP TABLE categoria PURGE;
DROP TABLE corredor  PURGE;
DROP TABLE juego     PURGE;
```

Ejercicio 8 · Cambios en la estructura

Sobre las tablas EQUIPO y JUGADOR_PRO de los ejercicios 2 y 3 (ya con datos), escribe las sentencias para:

1. Añadir a EQUIPO una columna obligatoria `activo` (`'S'` / `'N'`), que valga `'S'` para los equipos existentes.
2. Permitir alias de hasta 40 caracteres.
3. Añadir a JUGADOR_PRO un `rol` opcional (`CAPITAN`, `TITULAR` o `SUPLENTE`).
4. Cambiar el nombre de la columna `pais` de JUGADOR_PRO por `nacionalidad`.
5. Eliminar la columna `fecha_fundacion` de EQUIPO.
6. Desactivar temporalmente la restricción del número máximo de jugadores.
7. Añadir un comentario a la tabla EQUIPO.

Solución

```sql
-- 1. Con DEFAULT: si no, ORA-01758 al haber filas
ALTER TABLE equipo ADD (activo CHAR(1) DEFAULT 'S' NOT NULL);
ALTER TABLE equipo ADD CONSTRAINT ck_equipo_activo CHECK (activo IN ('S', 'N'));

-- 2.
ALTER TABLE jugador_pro MODIFY (alias VARCHAR2(40));

-- 3.
ALTER TABLE jugador_pro ADD (rol VARCHAR2(8));
ALTER TABLE jugador_pro ADD CONSTRAINT ck_jugador_pro_rol
    CHECK (rol IN ('CAPITAN', 'TITULAR', 'SUPLENTE'));

-- 4.
ALTER TABLE jugador_pro RENAME COLUMN pais TO nacionalidad;

-- 5.
ALTER TABLE equipo DROP COLUMN fecha_fundacion;

-- 6.
ALTER TABLE equipo DISABLE CONSTRAINT ck_equipo_max;

-- 7.
COMMENT ON TABLE equipo IS 'Equipos inscritos en la liga';
```

En el 3, `rol` es opcional, así que el `CHECK` admite `NULL` (con `NULL` la condición es desconocida y se da por cumplida).

Bloque 3 · Vistas, usuarios y diagnóstico

Ejercicio 9 · Vistas

Con las tablas EQUIPO y JUGADOR_PRO, crea:

1. `v_equipos_espanoles` : los equipos cuyo país es `'España'` , de forma que a través de ella no se puedan insertar equipos de otro país.
2. `v_plantillas` : por cada jugador con equipo, las siglas del equipo, el alias del jugador y su edad, de solo lectura.
3. `v_agentes_libres` : los jugadores sin equipo, mostrando solo alias y nacionalidad.

¿Cuáles son actualizables?

Solución

```
CREATE OR REPLACE VIEW v_equipos_espanoles AS
SELECT id_equipo, nombre, siglas, pais, max_jugadores
  FROM equipo
 WHERE pais = 'España'
  WITH CHECK OPTION CONSTRAINT ck_v_equipos_espanoles;

CREATE OR REPLACE VIEW v_plantillas AS
SELECT e.siglas,
       j.alias,
       TRUNC(MONTHS_BETWEEN(SYSDATE, j.fecha_nacimiento) / 12) AS edad
  FROM jugador_pro j
  JOIN equipo e ON e.id_equipo = j.id_equipo
  WITH READ ONLY;

CREATE OR REPLACE VIEW v_agentes_libres AS
SELECT id_pro, alias, nacionalidad
  FROM jugador_pro
 WHERE id_equipo IS NULL;
```

- `v_equipos_espanoles` es actualizable (una tabla, sin agregados) y el `CHECK OPTION` impide insertar equipos de otro país.
- `v_plantillas` no lo es: es de solo lectura por decisión, y además `edad` es una expresión.
- `v_agentes_libres` es actualizable. Pero ojo: un `INSERT` a través de ella fallaría si JUGADOR_PRO tiene columnas `NOT NULL` que la vista no incluye (`fecha_nacimiento`): ORA-01400.

Ejercicio 10 · Usuarios y privilegios

La liga necesita:

- Un propietario `LIGA` con 50 MB en `USERS` que pueda crear tablas y vistas.
- Un rol `ROL_ARBITRO` : consulta las tablas EQUIPO, JUGADOR_PRO y PARTICIPA, y solo puede **modificar el dorsal** en PARTICIPA.
- Un rol `ROL_PUBLICO` : solo consulta `v_plantillas` .
- Los usuarios `ARBITRO1` (con `ROL_ARBITRO` , que debe cambiar la contraseña al entrar) y `WEB_LIGA` (con `ROL_PUBLICO`).

Indica qué parte ejecuta SYSTEM y qué parte LIGA.

Solución

Como SYSTEM, en la PDB

```
CREATE USER liga IDENTIFIED BY "Liga_2026"
    DEFAULT TABLESPACE users
    QUOTA 50M ON users;
GRANT CREATE SESSION, CREATE TABLE, CREATE VIEW TO liga;

CREATE ROLE rol_arbitro;
CREATE ROLE rol_publico;

CREATE USER arbitro1 IDENTIFIED BY "Temporal_2026" PASSWORD EXPIRE;
CREATE USER web_liga IDENTIFIED BY "WebLiga_2026";
GRANT CREATE SESSION TO arbitro1, web_liga;
GRANT rol_arbitro TO arbitro1;
GRANT rol_publico TO web_liga;
```

Como LIGA, una vez creadas sus tablas y vistas

```
GRANT SELECT ON equipo      TO rol_arbitro;
GRANT SELECT ON jugador_pro TO rol_arbitro;
GRANT SELECT, UPDATE (dorsal) ON participa TO rol_arbitro;

GRANT SELECT ON v_plantillas TO rol_publico;
```

Si `ARBITRO1` intenta `UPDATE liga.participa SET id_equipo = 3 ...`, obtendrá ORA-01031: solo puede modificar `dorsal`.

Ejercicio 11 · Diagnóstico de errores

Explica la causa y la solución de cada situación:

1. Como SYSTEM, al ejecutar `CREATE USER liga ...` sale ORA-65096.
2. `LIGA` crea la tabla EQUIPO sin problema, pero al insertar la primera fila obtiene ORA-01950.
3. `WEB_LIGA` ejecuta `SELECT * FROM v_plantillas` y obtiene ORA-00942.
4. `LIGA` intenta `DROP TABLE equipo` y obtiene ORA-02449.
5. Un alumno ejecuta el script de tablas por segunda vez y la primera sentencia da ORA-00955.
6. `CREATE TABLE participa (...)` da ORA-00942 en la línea de la clave ajena hacia TORNEO.

Solución

1. Está conectado a la raíz `CDB$ROOT`, donde solo se crean usuarios comunes (`C##...`). Debe conectarse a la PDB (`XEPDB1` o `FREEPDB1`).
2. El usuario no tiene cuota en el tablespace (con la creación diferida de segmentos, la tabla vacía no ocupa espacio hasta la primera fila). Solución: `ALTER USER liga QUOTA 50M ON users;`.
3. La vista es de `LIGA`; sin prefijo, Oracle la busca en el esquema de `WEB_LIGA`. Debe escribir `liga.v_plantillas` (o `ALTER SESSION SET CURRENT_SCHEMA = liga`). Si aun así falla con ORA-00942, es que no tiene el permiso: revisar el `GRANT` al rol.
4. Otras tablas (JUGADOR_PRO, PARTICIPA) tienen claves ajenas que apuntan a EQUIPO. Hay que borrarlas antes, o usar `DROP TABLE equipo CASCADE CONSTRAINTS` (que elimina esas claves ajenas, no las tablas).
5. La tabla ya existe. Hay que ejecutar antes el script de borrado (o, en 23ai/26ai, usar `IF NOT EXISTS`).
6. La tabla TORNEO no existe todavía: se está creando PARTICIPA antes que la tabla a la que referencia. Hay que respetar el orden de dependencias.

Bloque 4 · Otros gestores

Ejercicio 12 · Traducir a SQL Server y MariaDB

Traduce la tabla EQUIPO del ejercicio 2 a SQL Server y a MariaDB.

Solución

SQL Server

```
CREATE TABLE equipo (
    id_equipo        INT          IDENTITY(1,1) NOT NULL,
    nombre           NVARCHAR(80) NOT NULL,
    siglas           CHAR(3)      NOT NULL,
    pais             NVARCHAR(60) NULL,
    fecha_fundacion  DATE         NULL,
    max_jugadores    TINYINT      NOT NULL
        CONSTRAINT df_equipo_max DEFAULT (7),
    CONSTRAINT pk_equipo        PRIMARY KEY (id_equipo),
    CONSTRAINT uk_equipo_nombre UNIQUE (nombre),
    CONSTRAINT uk_equipo_siglas UNIQUE (siglas),
    CONSTRAINT ck_equipo_max    CHECK (max_jugadores BETWEEN 5 AND 10)
);
```

MariaDB

```
CREATE TABLE equipo (
    id_equipo        INT         NOT NULL AUTO_INCREMENT,
    nombre           VARCHAR(80) NOT NULL,
    siglas           CHAR(3)     NOT NULL,
    pais             VARCHAR(60),
    fecha_fundacion  DATE,
    max_jugadores    TINYINT     NOT NULL DEFAULT 7,
    CONSTRAINT pk_equipo        PRIMARY KEY (id_equipo),
    CONSTRAINT uk_equipo_nombre UNIQUE (nombre),
    CONSTRAINT uk_equipo_siglas UNIQUE (siglas),
    CONSTRAINT ck_equipo_max    CHECK (max_jugadores BETWEEN 5 AND 10)
) ENGINE = InnoDB;
```

El `CHECK` de mayúsculas del ejercicio 2 se omite: en SQL Server y MariaDB, con la intercalación por defecto (que no distingue mayúsculas), `siglas = UPPER(siglas)` sería siempre verdadero, y además el `UNIQUE` ya trata `koi` y `KOI` como iguales.

RA 2 · PRÁCTICA EVALUABLE

Práctica: la base de datos de RetroRioja

RetroRioja es una tienda (ficticia) de Logroño que compra y vende consolas y videojuegos retro de segunda mano. Ya tiene hecho el diseño de su base de datos. Tu trabajo es **implementarlo en Oracle** siguiendo todo lo aprendido en este RA.

Criterios de evaluación que se trabajan: CE 2a CE 2b CE 2c CE 2d CE 2e CE 2f CE 2g CE 2h

El esquema relacional

```
CONSOLA(id_consola, nombre UNIQUE, fabricante, anio_lanzamiento)
JUEGO(id_juego, titulo, anio, id_consola → CONSOLA NOT NULL)
EJEMPLAR(id_juego → JUEGO, num_ejemplar, estado, precio, con_caja) — entidad débil
CLIENTE(id_cliente, dni UNIQUE, nombre, apellidos, email UNIQUE, telefono,
   fecha_alta)
SOCIO(id_cliente → CLIENTE, num_socio UNIQUE, puntos) — subtipo de CLIENTE
VENTA(num_venta, fecha, id_cliente → CLIENTE NOT NULL)
LINEA_VENTA(num_venta → VENTA, id_juego, num_ejemplar, precio_venta) — (id_juego,
   num_ejemplar) → EJEMPLAR, UNIQUE
```

Reglas de negocio

1. Los identificadores `id_consola`, `id_juego`, `id_cliente` y `num_venta` los genera la base de datos.
2. Los ejemplares se numeran dentro de cada juego (ejemplar 1, 2, 3… de *Super Mario World*).
3. El `estado` de un ejemplar solo puede ser `NUEVO`, `BUENO`, `USADO` o `REPARAR`.
4. Precios mayores o iguales que 0 y con céntimos; el precio de un ejemplar no supera los 5000 €.
5. `con_caja` indica si el ejemplar conserva la caja original (sí/no). Por defecto, no.
6. El año de un juego no puede ser anterior a 1970.
7. La fecha de alta de un cliente y la de una venta son, por defecto, la del día.
8. Los puntos de un socio empiezan en 0 y nunca son negativos.
9. **Cada ejemplar se vende como mucho una vez.**
10. Si se borra un juego del catálogo, se borran sus ejemplares. Si se borra un cliente, se borra su ficha de socio. Si se borra una venta, se borran sus líneas, pero los dependientes **no** pueden borrar ventas.
11. No se puede borrar una consola que tenga juegos, ni un cliente con ventas.

Lo que tienes que hacer

1. Usuarios y roles · `01_usuarios.sql` (como SYSTEM)

- Propietario `RETRO` con 50 MB en `USERS`, con los privilegios de sistema **justos** para crear tablas, vistas y secuencias.
- Roles `ROL_DEPENDIENTE` y `ROL_GESTORIA`.
- Usuarios `DEPENDIENTE1` (con `ROL_DEPENDIENTE`, que debe cambiar la contraseña la primera vez) y `GESTORIA` (con `ROL_GESTORIA`).

2. Tablas · `02_tablas.sql` (como RETRO)

- Las 7 tablas con los **tipos de datos** adecuados.
- Todas las restricciones, **con nombre** según el convenio (`pk_`, `fk_`, `uk_`, `ck_`), que implanten las reglas 1 a 11.
- Comentarios (`COMMENT ON`) en al menos tres tablas.

3. Índices · `03_indices.sql`

- Índices para las claves ajenas que no estén ya cubiertas por otro índice.

4. Vistas · `04_vistas.sql`

- `v_stock`: ejemplares **no vendidos**, con título, consola, estado y precio. De solo lectura.
- `v_ventas`: para la gestoría, número de venta, fecha y total de cada venta, **sin datos personales** de los clientes.
- `v_juegos_nintendo`: juegos de consolas del fabricante Nintendo, de forma que a través de ella no se puedan insertar juegos de otras consolas. (Pista: la consulta necesita una subconsulta sobre CONSOLA en el `WHERE`).

5. Privilegios · `05_privilegios.sql` (como RETRO)

- `ROL_DEPENDIENTE`: consulta todas las tablas y `v_stock`; registra ventas y sus líneas; da de alta clientes y socios; puede cambiar **solo** el `estado` y el `precio` de los ejemplares.
- `ROL_GESTORIA`: solo consulta `v_ventas`.

6. Datos y pruebas · `06_datos.sql` y `07_pruebas.sql`

- Al menos 3 consolas, 6 juegos, 10 ejemplares, 4 clientes (2 de ellos socios) y 3 ventas con varias líneas.
- Un script con **al menos 10 sentencias que deban fallar**, cada una con un comentario que indique qué regla prueba y qué error ORA- se espera. Al menos una debe probar la regla 9 y otra, un privilegio.

7. Verificación y memoria

- Consultas al **diccionario de datos** que demuestren que existen las 7 tablas, sus restricciones (con su `DELETE_RULE`) y sus índices.
- Un **documento breve** (máximo 3 páginas) que explique:

- Por qué has elegido cada tipo de dato que no sea evidente.
- Qué acción `ON DELETE` has puesto a cada clave ajena y por qué.
- Cómo has implantado la regla 9.
- Qué reglas **no** has podido implantar con restricciones y cómo se podrían implementar.

Entrega

Un único zip `apellido_nombre_retrorioja.zip` con los scripts numerados, el documento en PDF y capturas de SQL Developer donde se vean las tablas de `RETRO` y la ejecución de las pruebas. Los scripts deben poder ejecutarse **en orden, sin errores**, en una PDB limpia (salvo `07_pruebas.sql`, que debe dar exactamente los errores previstos).

Criterios de corrección

Criterio	CE	Excelente	Adecuado	Insuficiente	Peso
Usuarios, roles y privilegios	2g	Mínimo privilegio exacto; `UPDATE` por columnas; `PASSWORD EXPIRE`	Funciona pero con algún privilegio de más	Usa `DBA`, `ANY` o conexión como propietario	15 %
Tipos de datos	2a, 2c	Todos adecuados y justificados (dinero, fechas, códigos)	Algún tipo mejorable	Fechas o importes como texto, tamaños absurdos	10 %
Tablas y claves	2b, 2d	PK y FK correctas, incluidas la compuesta y la del subtipo; orden correcto	Algún fallo menor en claves	Faltan claves o el script no se ejecuta en orden	20 %
Restricciones	2e	Reglas 1-11 implantadas, todas con nombre; regla 9 con `UNIQUE`	Falta alguna regla o algún nombre	Faltan varias reglas o no tienen nombre	20 %
Índices	2e	Todas las FK no cubiertas, sin índices redundantes	Falta alguno	No hay índices	5 %
Vistas	2f	Las tres correctas, con `READ ONLY` y `CHECK OPTION` donde toca	Alguna vista incompleta	No hay vistas o no funcionan	10 %
Pruebas y verificación	2h	10+ pruebas bien comentadas; consultas al diccionario completas	Menos pruebas o sin comentar	Sin pruebas	10 %

Criterio	CE	Excelente	Adecuado	Insuficiente	Peso
Memoria	2e, 2h	Justifica decisiones y documenta lo no implantable	Justificación parcial	Sin memoria o sin justificar	10 %

RA 2 · REPASO

Autoevaluación

Treinta preguntas para comprobar lo que has aprendido. Responde cada una antes de desplegar la solución. Si fallas alguna, vuelve al apartado indicado.

Almacenamiento y herramientas

1. Al ejecutar `CREATE USER pixelvault ...` como SYSTEM obtienes ORA-65096. ¿Qué ocurre?

- a) La contraseña no cumple los requisitos
- b) Estás conectado a la raíz del contenedor (`CDB$ROOT`) y no a la PDB
- c) El usuario ya existe
- d) SYSTEM no tiene permiso para crear usuarios

Solución

b) En la raíz solo se crean usuarios comunes (`C##...`). Hay que conectarse al servicio de la PDB: `XEPDB1` o `FREEPDB1` . Repasa el apartado 1.2.

2. Ordena de mayor a menor: bloque, tablespace, extensión, segmento.

Solución

Tablespace → segmento → extensión → bloque. Un tablespace contiene segmentos (uno por tabla o índice); cada segmento crece por extensiones, que son grupos de bloques contiguos. Repasa el apartado 1.3.

3. En Oracle, ¿qué relación hay entre un usuario y un esquema?

Solución

Son lo mismo visto desde dos lados: el usuario es la cuenta, y su esquema es el conjunto de objetos que le pertenecen, con su mismo nombre. Por eso la tabla VIDEOJUEGO de PIXELVAULT se llama `pixelvault.videojuego` . Repasa el apartado 1.5.

4. Un usuario sin cuota en ningún tablespace ejecuta `CREATE TABLE` y después un `INSERT` . ¿Qué pasa?

- a) Falla el `CREATE TABLE` con ORA-01031
- b) La tabla se crea, pero el primer `INSERT` falla con ORA-01950
- c) Todo funciona: la cuota solo limita el tamaño máximo
- d) La tabla se crea en el tablespace `SYSTEM`

Solución

b) La tabla vacía no ocupa espacio todavía; al insertar la primera fila, Oracle necesita espacio y el usuario no tiene cuota. Repasa el apartado 1.3.

5. Tienes filas insertadas sin confirmar y ejecutas un `CREATE TABLE`. Después haces `ROLLBACK`. ¿Qué ocurre con las filas?

Solución

Se quedan: el `CREATE TABLE`, como toda sentencia DDL, hace un `COMMIT` implícito, así que las filas ya estaban confirmadas cuando llegó el `ROLLBACK`. Repasa el apartado 2.1.

6. En SQL Developer, ¿qué diferencia hay entre `^ Ctrl` + `Enter ⏎` y `F5`?

Solución

`^ Ctrl` + `Enter ⏎` ejecuta **solo la sentencia** donde está el cursor y muestra el resultado en una rejilla; `F5` ejecuta **todo el script** y muestra la salida como texto. Repasa el apartado 2.2.

Tipos de datos

7. ¿Qué tipo usarías para el código postal?

- a) `NUMBER(5)`
- b) `VARCHAR2(10)`
- c) `DATE`
- d) `BINARY_FLOAT`

Solución

b) No se opera con él, puede empezar por `0` (`01001`) y en otros países lleva letras. Repasa el apartado 3.5.

8. ¿Qué ocurre al insertar `12345.6` en una columna `NUMBER(6,2)`?

Solución

Error **ORA-01438**: `NUMBER(6,2)` admite 6 dígitos en total, 2 de ellos decimales, así que la parte entera tiene como máximo 4 dígitos (hasta 9999,99). Repasa el apartado 3.2.

9. Guardas `SYSDATE` en una columna `DATE` y luego buscas `WHERE fecha = DATE '2026-09-25'` el mismo día. No aparece la fila. ¿Por qué?

Solución

Porque `DATE` guarda también la hora, y `SYSDATE` la incluye. El literal `DATE '2026-09-25'` es ese día a las 00:00:00, que no coincide. Se soluciona guardando `TRUNC(SYSDATE)` o buscando con `TRUNC(fecha) = ...`. Repasa el apartado 3.3.

10. En Oracle XE 21c, ¿cómo se guarda un dato de tipo sí/no?

- a) Con el tipo `BOOLEAN`
- b) Con el tipo `BIT`
- c) Con `CHAR(1)` y un `CHECK (columna IN ('S','N'))`
- d) Con `VARCHAR2(5)` para guardar `'TRUE'` o `'FALSE'` sin restricciones

Solución

c) `BOOLEAN` no existe en tablas hasta 23ai, y `BIT` es de SQL Server. La opción d) permitiría cualquier texto. Repasa el apartado 3.4.

11. En Oracle, ¿qué se guarda al insertar `''` en una columna `VARCHAR2` ?

Solución

`NULL`. En Oracle la cadena vacía es lo mismo que `NULL` ; si la columna es `NOT NULL` , la inserción falla con ORA-01400. En SQL Server y MariaDB no ocurre. Repasa el apartado 3.1.

Tablas y claves

12. ¿Por qué conviene poner nombre a todas las restricciones?

Solución

Porque el nombre aparece en los mensajes de error y en el diccionario de datos. `ORA-02290 ... (PIXELVAULT.CK_VIDEOJUEGO_PEGI)` dice qué regla se ha incumplido; `SYS_C008412` no dice nada. Además, permite desactivarlas o borrarlas por su nombre. Repasa el apartado 4.1.

13. ¿Qué hace Oracle automáticamente al declarar una `PRIMARY KEY` ?

- a) Solo comprueba que no hay duplicados
- b) Añade `NOT NULL` , comprueba que no hay duplicados y crea un índice único
- c) Crea una secuencia para generar los valores
- d) Crea claves ajenas en las tablas relacionadas

Solución

b) La clave primaria no genera valores (para eso están `IDENTITY` y las secuencias) ni crea nada en otras tablas. Repasa el apartado 4.3.

14. Diferencia entre `GENERATED ALWAYS AS IDENTITY` y `GENERATED BY DEFAULT AS IDENTITY`.

Solución

Con `ALWAYS`, Oracle genera siempre el valor y da error (ORA-32795) si intentas darlo tú. Con `BY DEFAULT`, lo genera solo si no lo das. En PixelVault usamos `BY DEFAULT` para poder cargar identificadores conocidos, y después sincronizamos con `START WITH LIMIT VALUE`. Repasa el apartado 4.4.

15. ¿Qué debe cumplir la columna (o columnas) a la que apunta una clave ajena?

Solución

Debe ser la **clave primaria** o una clave `UNIQUE` de la tabla referida, con el mismo número de columnas y tipos compatibles. Si no, ORA-02270. Repasa el apartado 4.5.

16. En PixelVault, la clave ajena de LINEA_PEDIDO apunta a JUEGO_PAGO y no a VIDEOJUEGO. ¿Qué consigue?

Solución

Que solo se puedan vender juegos de pago: un juego *free-to-play* no tiene fila en JUEGO_PAGO, así que no puede aparecer en una línea de pedido (ORA-02291). La regla de negocio queda en la estructura. Repasa el apartado 4.5.

17. ¿Qué acción referencial **no** existe en Oracle?

- a) Restringir (sin cláusula)
- b) `ON DELETE CASCADE`
- c) `ON DELETE SET NULL`
- d) `ON UPDATE CASCADE`

Solución

d) Oracle no propaga cambios de la clave primaria. Repasa el apartado 4.5.

18. Al crear una tabla, Oracle responde ORA-00942 en la línea de una clave ajena. ¿Causa más probable?

Solución

La tabla a la que apunta la clave ajena **todavía no existe**: el script no respeta el orden de dependencias. Hay que crear primero las tablas referenciadas. Repasa el apartado 4.6.

Restricciones e índices

19. ¿Cuál de estas reglas **no** se puede implantar con un `CHECK`?

- a) `pegi IN (3, 7, 12, 16, 18)`
- b) `id_jugador < id_amigo`
- c) La fecha de nacimiento es anterior a hoy
- d) `nota BETWEEN 1 AND 10`

> ✓ **Solución**
>
> **c)** Un `CHECK` no puede usar `SYSDATE` (ORA-02436), porque su resultado cambiaría con el tiempo. Repasa el apartado 5.4.

20. Una columna `precio NUMBER(6,2)` tiene `CHECK (precio >= 0)` pero no `NOT NULL`. ¿Se puede insertar una fila con precio `NULL`?

> ✓ **Solución**
>
> **Sí.** Con `NULL`, la condición es desconocida, y un `CHECK` se da por cumplido si no es falso. Para prohibirlo hace falta también `NOT NULL`. Repasa el apartado 5.4.

21. ¿Puede una columna `UNIQUE` tener varios `NULL` en Oracle? ¿Y en SQL Server?

> ✓ **Solución**
>
> En Oracle, **sí**: `NULL` no es igual a nada, así que varios `NULL` no se consideran repetidos. En SQL Server, un `UNIQUE` normal solo admite un `NULL`; para admitir varios se usa un índice único filtrado. Repasa los apartados 5.3 y 12.1.

22. ¿Crea Oracle automáticamente un índice para cada clave ajena? ¿Por qué conviene tenerlo?

> ✓ **Solución**
>
> **No.** Conviene porque las combinaciones (`JOIN`) se hacen por la clave ajena y porque, al borrar una fila de la tabla referenciada, Oracle busca las filas hijas: sin índice recorre toda la tabla hija y la bloquea. Repasa el apartado 7.2.

23. Tienes un índice sobre `(id_jugador, id_juego)`. ¿Sirve para una consulta que filtra solo por `id_juego`?

> ✓ **Solución**
>
> No, o muy mal: el índice está ordenado primero por jugador. Sirve para buscar por `id_jugador` o por los dos. Por eso PixelVault tiene además `ix_biblioteca_juego`. Repasa el apartado 7.3.

Modificar, vistas, usuarios y diccionario

24. Añades a una tabla con filas una columna `activo CHAR(1) NOT NULL` y obtienes ORA-01758. ¿Cómo se arregla?

> ✓ **Solución**
>
> Añadiendo un `DEFAULT`: `ALTER TABLE usuario ADD (activo CHAR(1) DEFAULT 'S' NOT NULL);`. Así las filas existentes reciben `'S'`. Repasa el apartado 6.2.

25. ¿Qué diferencia hay entre `TRUNCATE TABLE t` y `DELETE FROM t`?

> ✓ **Solución**
>
> `DELETE` es DML: borra fila a fila, admite `WHERE`, respeta las acciones `ON DELETE` y se puede deshacer con `ROLLBACK`. `TRUNCATE` es DDL: vacía toda la tabla de golpe, es muy rápido, hace `COMMIT` implícito y **no** se puede deshacer. Repasa el apartado 6.6.

26. ¿Por qué la edad del jugador se calcula en una vista y no en una columna de la tabla?

> ✓ **Solución**
>
> Porque es un **atributo derivado** que depende de la fecha actual. Guardada, dejaría de ser correcta; y una columna virtual no puede usar `SYSDATE`. La vista la calcula en el momento de consultar. Repasa el apartado 8.3.

27. ¿Qué hace `WITH CHECK OPTION` en una vista?

- a) Impide cualquier modificación a través de la vista
- b) Comprueba las restricciones `CHECK` de la tabla
- c) Impide insertar o modificar a través de la vista filas que no cumplirían su `WHERE`
- d) Obliga a tener permisos sobre la tabla para usar la vista

> ✓ **Solución**
>
> **c)** La opción a) es `WITH READ ONLY`. Repasa el apartado 8.4.

28. Diferencia entre un privilegio de sistema y uno de objeto. Pon un ejemplo de cada.

> ✓ **Solución**
>
> Un privilegio de **sistema** permite una acción en la base de datos (`CREATE TABLE`, `CREATE SESSION`); uno de **objeto** permite una operación sobre un objeto concreto (`SELECT ON pixelvault.videojuego`). Repasa los apartados 9.3 y 9.4.

29. El analista, que no tiene ningún permiso sobre la tabla USUARIO, ejecuta `SELECT * FROM pixelvault.usuario`. ¿Qué error obtiene y por qué ese?

Solución

ORA-00942 (la tabla o vista no existe). Oracle no revela la existencia de objetos a quien no tiene ningún permiso sobre ellos. Si tuviera algún permiso pero no el necesario, el error sería ORA-01031. Repasa el apartado 9.8.

30. ¿En qué vista del diccionario consultarías la acción `ON DELETE` de las claves ajenas de tus tablas? Escribe la consulta para VIDEOJUEGO.

Solución

En `USER_CONSTRAINTS`, columna `DELETE_RULE`:

```
SELECT constraint_name, delete_rule
  FROM user_constraints
 WHERE table_name = 'VIDEOJUEGO'
   AND constraint_type = 'R';
```

El nombre de la tabla va en mayúsculas. Repasa el apartado 10.3.

RA 2 · REPASO

Glosario del RA 2

Los términos clave, por orden alfabético.

Bloque de datos

: Unidad mínima de lectura y escritura de Oracle (8 KB por defecto). Guarda varias filas.

CDB (base de datos contenedora)

: Base de datos que aloja una o varias PDB. Su raíz, `CDB$ROOT`, no se usa para crear usuarios de aplicación.

`CHECK`

: Restricción que comprueba una condición sobre los valores de la propia fila. No puede consultar otras filas ni usar `SYSDATE`.

Clave ajena (`FOREIGN KEY`)

: Restricción que obliga a que los valores de una o varias columnas existan como clave primaria o única en otra tabla (o en la misma). Implementa la propagación de clave del diseño.

Clave primaria (`PRIMARY KEY`)

: Restricción que identifica cada fila: no admite nulos ni repetidos. Oracle crea para ella un índice único.

`COMMIT` implícito

: Confirmación automática de la transacción que Oracle hace antes y después de cada sentencia DDL.

Cuota

: Espacio máximo que un usuario puede ocupar en un tablespace (`QUOTA 100M ON users`).

DCL (lenguaje de control de datos)

: Parte de SQL que gestiona los permisos: `GRANT` y `REVOKE`.

DDL (lenguaje de definición de datos)

: Parte de SQL que crea, modifica y borra la estructura: `CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`, `COMMENT`.

`DEFAULT`

: Valor que toma una columna cuando el `INSERT` no la indica.

Diccionario de datos

Conjunto de tablas y vistas internas en las que Oracle describe todos los objetos de la base de datos (`USER_TABLES`, `USER_CONSTRAINTS`...).

Esquema

Conjunto de objetos que pertenecen a un usuario. En Oracle tiene el mismo nombre que el usuario.

Fichero de datos

Fichero del disco (`.dbf`) donde se guarda físicamente el contenido de un tablespace.

`IDENTITY`

Columna numérica que genera automáticamente sus valores (`GENERATED ALWAYS` o `BY DEFAULT AS IDENTITY`).

Índice

Estructura, normalmente un árbol B, que guarda valores ordenados con el ROWID de su fila para acelerar las búsquedas.

Índice basado en función

Índice sobre una expresión, como `UPPER(titulo)`.

Instancia

Memoria (SGA) y procesos de Oracle que atienden a los usuarios y trabajan con los ficheros de la base de datos.

`NOT NULL`

Restricción que hace obligatoria una columna.

Papelera de reciclaje

Lugar al que van las tablas borradas con `DROP TABLE` sin `PURGE`. Se recuperan con `FLASHBACK TABLE ... TO BEFORE DROP`.

PDB (base de datos enchufable)

Base de datos independiente dentro de una CDB. En las ediciones gratuitas: `XEPDB1` (21c) o `FREEPDB1` (23ai/26ai).

Principio de mínimo privilegio

Cada usuario debe tener solo los permisos imprescindibles para su tarea.

Privilegio de objeto

Permiso para una operación sobre un objeto concreto: `SELECT ON videojuego`, `INSERT ON pedido`...

Privilegio de sistema

Permiso para una acción en la base de datos: `CREATE SESSION`, `CREATE TABLE`, `CREATE VIEW`…

Restricción (*constraint*)

Regla que el gestor comprueba en cada inserción, modificación o borrado: `PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, `NOT NULL` y `CHECK`.

Restricción diferible

Restricción declarada `DEFERRABLE` que se puede comprobar al hacer `COMMIT` en lugar de al final de cada sentencia.

Rol

Conjunto de privilegios con nombre que se concede a usuarios.

ROWID

Dirección física de una fila: fichero, bloque y posición.

Script

Fichero `.sql` con sentencias que se ejecutan en orden.

Secuencia

Objeto que genera números consecutivos (`NEXTVAL`), independiente de las tablas.

Segmento

Espacio que ocupa un objeto (una tabla, un índice) dentro de un tablespace. Crece por extensiones.

Tablespace

Contenedor lógico de almacenamiento donde se guardan tablas e índices. Se apoya en uno o varios ficheros de datos.

Tipo de dato

Conjunto de valores que admite una columna y cómo se almacenan: `VARCHAR2`, `NUMBER`, `DATE`…

`UNIQUE`

Restricción que impide valores repetidos. Implementa las claves alternativas. En Oracle admite varios `NULL`.

Usuario

Cuenta con la que se conecta una persona o un programa. En Oracle, cada usuario tiene su esquema.

Vista

Consulta guardada con nombre que se usa como una tabla. No almacena datos.

`WITH CHECK OPTION`

Cláusula de una vista que impide insertar o modificar a través de ella filas que no cumplirían su `WHERE`.

`WITH READ ONLY`

Cláusula de una vista que impide modificar datos a través de ella.